Modeling State to Improve Defensive Cyberattack Strategies

by

Vivin Suresh Paliath

A Dissertation Presented in Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy

Approved September 2023 by the
Graduate Supervisory Committee:

Adam Doupé, Co-Chair
Yan Shoshitaishvili, Co-Chair
Ruoyu Wang
Paulo Shakarian

ARIZONA STATE UNIVERSITY

December 2023

©2023 Vivin Suresh Paliath

All Rights Reserved

ABSTRACT

Human civilization within the last two decades has largely transformed into an online one, with many of its associated activities taking place on computers and complex networked systems – their analog and real-world equivalents having been rendered obsolete. These activities run the gamut from the ordinary and mundane, like ordering food, to complex and large-scale, such as those involving critical infrastructure or global trade and communications. Unfortunately, the activities of human civilization also involve criminal, adversarial, and malicious ones with the result that they also now have their digital equivalents. Ransomware, malware, and targeted cyberattacks are a fact of life today and are instigated not only by organized criminal gangs, but adversarial nation-states and organizations as well. Needless to say, such actions result in disastrous and harmful real-world consequences.

As the complexity and variety of software has evolved, so too has the ingenuity of attacks that exploit them; for example modern cyberattacks typically involve sequential exploitation of multiple software vulnerabilities. Compared to a decade ago, modern software stacks on personal computers, laptops, servers, mobile phones, and even Internet of Things (IoT) devices involve a dizzying array of interdependent programs and software libraries, with each of these components presenting attractive attack-surfaces for adversarial actors. However, the responses to this still rely on paradigms that can neither react quickly enough nor scale to increasingly dynamic, ever-changing, and complex software environments. Better approaches are therefore needed, that can assess system readiness and vulnerabilities, identify potential attack vectors and strategies (including ways to counter them), and proactively detect vulnerabilities in complex software before they can be exploited.

In this dissertation, I first present a mathematical model and associated algorithms

to identify attacker strategies for sequential cyberattacks based on attacker state, attributes and publicly-available vulnerability information. Second, I extend the model and design algorithms to help identify defensive courses of action against attacker strategies. Finally, I present my work to enhance the ability of coverage-based fuzzers to identify software vulnerabilities by providing visibility into complex, internal program-states.

DEDICATION

To my wife Aparna, and to our dogs Arizona and Bear; I could not have done this without your love, strength, patience, and support. My love for you is eternal.

To my grandparents; Ithitha, Ammumma, and Paliath Ammumma, I dearly wish you were alive to see me get my doctorate but I know you are proud wherever you are.

To my parents Suresh and Mohana, my sister Keerthi, my uncle and aunt Ram and Hema, my cousins Dipu Cheta and Prem, my wife's parents Valsan and Nalini, and to Anoop and Nivedya; for supporting me and always being there for me.

To my dearest friends Michael, Josh, Suhrid, Vishal, Nicole, Tawnya, and Jignya; for believing in me even when I did not, and for reminding me to take time for myself.

To my younger self, whose optimistic hubris and persistence helped me succeed.

ACKNOWLEDGMENTS

I am profoundly thankful to my advisor and committee co-chair, Dr. Adam Doupé. His unwavering support, patience, and constructive feedback have been foundational in my academic journey. Even while juggling a full-time career with my PhD, his adaptability and willingness to assist me at any time were invaluable. His empathetic guidance, especially during the most strenuous stretches of my PhD, reassured me immensely, and his suggestions to prioritize my well-being were greatly appreciated.

My deepest appreciation extends to my committee co-chair, Dr. Yan Shoshitaishvili. His fervent support and keen insights into my research, along with his invaluable advice, have greatly enriched my work. The enhancements in the final outcome of my research are significantly attributed to his contributions.

I wish to express my profound gratitude to Dr. Ruoyu Wang, a committee member who consistently inspired me to approach my research with a discerning eye. His invaluable critiques undeniably elevated the caliber of my work.

Dr. Paulo Shakarian, my former advisor and committee member, has my heartfelt thanks for his foundational support early in my academic endeavor. His encouragement was pivotal, fueling my determination to see my PhD journey to its conclusion.

A special note of gratitude is reserved for Dr. Tiffany Bao, whose insightful feedback was instrumental in refining my research. Her keen observations and suggestions undeniably enhanced the quality of my work.

A special acknowledgment to the Pat Tillman Foundation for bestowing upon me the honor of being a Tillman Scholar. Their financial support was instrumental in my PhD journey. The fraternity of Tillman Scholars they introduced me to has been an immense source of strength and encouragement.

My lab colleagues and peers, particularly Dr. Erik Trickel, have been a cornerstone

of support. Their expert advice and insights were essential throughout my research and the dissertation process. I extend my gratitude to the Kubernetes team at SEFCOM lab. Their assistance, from infrastructural aid to patiently answering my plethora of queries, was crucial. A nod of appreciation to my Computer Science professors from my formative academic years; their teachings have been the bedrock of my success.

The support structure provided by my family and friends has been monumental. Their unwavering faith in me, even during the most challenging times of my PhD journey, has been a beacon. My dogs, Arizona and Bear, with their boundless affection and positivity, have been a constant source of joy, alleviating the stresses of my PhD journey.

Communities such as StackOverflow, LaTeX Stack Exchange, and various resourceful individuals on GitHub, Google Groups, and forums deserve my gratitude. Their expertise and solutions were pivotal in my research and the formatting of my academic works, including this dissertation.

As a token of appreciation, I'd also like to acknowledge the Star Trek series and numerous video game creators. Their creations were my go-to respite, providing the relaxation I often sought during intense research periods.

TABLE OF CONTENTS

Page

LIST OF TABLES ix

LIST OF FIGURES x

CHAPTER

1 INTRODUCTION 1

2 BACKGROUND 6

2.1 Cybersecurity Threat Modeling and Mitigation 6

2.2 Greybox Fuzzing 7

3 REASONING ABOUT SEQUENTIAL CYBERATTACKS 10

3.1 Model 10

3.1.1 Fixed-Point Operator 15

3.1.2 Fixed-Point Operator Complexity and Memoization 21

3.1.3 Preferred Attack Strategy 25

3.2 Algorithms 29

3.2.1 Pruning Exploits 30

3.2.2 PAS-A* 32

3.2.3 Enhancing PAS-A* Using Fixed-Point Operator Results 35

3.2.4 Improving Fixed-Point Operator Performance 39

3.3 Experiments and Results 40

3.4 Related Work 47

3.5 Conclusion 48

4 DEFENDING AGAINST SEQUENTIAL CYBERATTACKS 49

4.1 Defender Problem 49

4.1.1 Preferred Defense Strategy 52

CHAPTER Page

4.2 Algorithms 56

4.2.1 Pruning Vulnerabilities 57

4.2.2 A* Based Approach 61

4.3 Experiments and Results 65

4.4 Related Work 72

4.5 Conclusion 74

5 AUTOMATIC IDENTIFICATION OF STATE-REPRESENTATIVE VARIABLES TO PROVIDE DEEP-STATE FEEDBACK 76

5.1 Background 79

5.2 Overview 82

5.3 Trace Collection 84

5.4 Semantic Categories 85

5.5 Semantic Category Heuristics 87

5.5.1 Non-Semantic Categories 87

5.5.2 Semantic Categories 88

5.6 Identification of Semantic Categories 92

5.7 Feedback Instrumentation 94

5.7.1 Permutation Feedback 95

5.7.2 Maximization Feedback 96

5.7.3 Related-Variable Maximization Feedback (RVMF) 97

5.7.4 Related-Variable Combination Feedback (RVCF) 98

5.8 Evaluation 99

5.8.1 Identification of Semantic Categories 100

5.8.2 Exploring Complex States 103

CHAPTER Page

5.8.2.1 Traversing the Maze 103

5.8.2.2 Super Mario Bros 105

5.8.2.3 State Exploration Evaluation 107

5.8.3 Coverage Evaluation 109

5.8.4 Vulnerability Detection 114

5.8.5 Feedback Instrumentation Evaluation 115

5.9 Discussion, Limitations, and Future Work 117

5.10 Related Work 119

5.11 Conclusion 122

6 CONCLUSION 123

REFERENCES 125

LIST OF TABLES

Table Page

1. Number of Obtained Capabilities, and Size and Cost of Optimal Solution Based on Attacker Type 42
2. Solution Costs and Sizes for Attackers of Different Skill-Levels 47
3. Patched Vulnerabilities, Solution Costs, and Sizes for Different Attacker Types 70
4. Patched Vulnerabilities for Different Attacker Types 71
5. Sample of Classified Variables with Labels from Various Real-World Programs 102
6. Comparison Between State-Representative Variables Manually Identified by IJON and Automatically Identified by SandPuppy 103
7. Comparison of Number of Integers SandPuppy Identified as State-Representative and the Number of IJON Annotations 103
8. Median Solution Times for All Three Mazes (5 Runs) 105
9. SandPuppy, AFL, and IJON Playing Super Mario Bros (3 Runs Each) 106
10. Number of Distinct Command Subsequences and Total Unique Sequences Found by All Fuzzers in Libtpms 108
11. Number of Variables Detected After Each Trace-Generation Step of Each Run 111
12. Improvement Over Baseline Coverage for All Fuzzers 112
13. Bugs Discovered by Each Fuzzer in PcapPlusPlus 115
14. Number of Distinct Command Subsequences and Total Unique Sequences with Parameter Size Found for Each Feedback-Evaluation Run in Libtpms 116
15. Coverage (Percentage of Basic Blocks Hit) for Each Feedback-Evaluation Run in Libtpms 116

LIST OF FIGURES

Figure Page

1. Example of a Trie Used to Cache Results of the Fixed-Point Operator 40
2. Runtime vs. Number of Exploits for Non-Memoized and Memoized Exploit Operator 41
3. Box Plot of Runtimes and Solution Costs for PAS-DFS and PAS-DFS-Pruned 43
4. Mean Runtime vs. Budget for PAS-DFS and PAS-DFS-Pruned 44
5. Box Plot of Runtimes for All PAS-A* Algorithms Against Different Attacker Types 46
6. Runtime Box Plots for PDS-DFS and PDS-DFS-Pruned With and Without Memoization 68
7. Box Plot of Runtimes for All PDS-A* Algorithms 69
8. Comparison of Edge-Transition Based and Player-Location Based Coverage-Maps for a Maze Program 82
9. SandPuppy Architecture 83
10. Comparison Of Edge-Transition Based and Permutation-Based Coverage-Maps for a Message-Processing Program 96
11. SandPuppy's Classification on Test-Bed Program Variables vs. k-Means and GMM 101
12. Unique Command-Sequences With Parameter Size Found Over Time in Libtpms 108
13. Heatmap of Counts of Various Sequence Lengths Found Over Time by All Fuzzers in Libtpms 109
14. Rarebug Solution Times for All Fuzzers (Average of 5 Runs) 110

Chapter 1

INTRODUCTION

The average cost of a data breach in 2022 was estimated at $4.35 million [1]. By 2025, cybercrime is expected to cost the world $10.5 trillion annually [2]. As an increasing amount of human activity moves online, the consequences of these attacks now correspondingly impact both an increasing number of people and multiple facets of their lives. For example, since 2007 there have been approximately 400 cyberattacks against financial institutions including numerous banks and credit reporting agencies, with almost half of these attacks having happened in the last 3 years [3]. In 2022 alone, nearly 50 million Americans had their health records exposed due to data breaches [4].

Such attacks are not limited to cybercrime; cyberattacks are now part of the arsenal of many nation states who regularly employ them to infiltrate adversary systems for the purposes of espionage or sabotage. Since the Stuxnet attack in 2010 [5], the scale and frequency of cyberattacks against government organizations has only increased. Compared to the same period in 2021, the number of cyberattacks against government agencies increased by 95% during the second half of 2022 [6].

These attacks are not just conducted by nation states, sophisticated hacker groups, or criminal organizations, but by low-skill attackers and "hacktivists" as well who are able to purchase exploit kits, ransomware, and malware on the darkweb [7]. The low prices of these kits and their relative ease of use is what makes them so attractive. Remote access trojans (RATs) can be purchased for as little as $5. Affordable phishing kits are also available for as little as $7. More sophisticated kits are slightly more expensive, though not by much; distributed denial-of-service (DDoS) attack kits can

be purchased for $10 and basic ransomware kits start at $15. Novice hackers can even purchase hacking tools for as little $14 to help them craft their own kits and malware [8]. Analysis of these tools, especially the more sophisticated ones, have revealed that they target multiple vulnerabilities through chained attacks [9].

However, despite the easy availability of such malware and exploit kits and their potential threat to computer-systems, existing paradigms that model cyber-adversarial behavior [10]–[12] do not factor in the interdependent nature of vulnerabilities or are too computationally intensive [13], [14]; this limits their effectiveness when applied to real-world situations. Such a model would make it possible for security experts and analysts to simulate cyberattacks against a model of an existing system in order to assess its degree of vulnerability and ascertain the potential consequences of attacks.

To address this problem, I introduce a mathematical formalism in Chapter 3 that models sequential cyberattacks based on vulnerability dependencies. The formalism is also able to take into account the initial capabilities of attackers and determine the maximal set of additional capabilities they are able to gain by exploiting vulnerable systems. I also introduce efficient A*-based algorithms that can identify an attacker's optimal strategy based on their budget and their initial set of capabilities. Through a suite of experiments based on real-world vulnerability data, I demonstrate that this model works well in practice and is capable of efficiently producing constructive results.

The failure to consider vulnerability dependencies also affects strategies aimed at defending systems from cyberattacks. While keeping systems up to date with security patches is still a generally good strategy, it is often not feasible for legacy systems. In addition, some patching solutions may require systems to be unavailable while they are patched, which may not possible in certain scenarios. There exist approaches

to derive effective patching strategies [15]–[17], but these are stochastic and largely based on metrics such as vulnerability discovery-rate, attack rates, and aspects of the vulnerability life-cycle itself. Hence, these approaches are far more general and when identifying effective patching-strategies, do not take into account vulnerability dependencies or the specific capabilities that attackers wish to gain.

To address this shortcoming, in Chapter 4 I extend the formalism introduced in Chapter 3, allowing it to model the effects of patching vulnerabilities. I then introduce efficient A*-based algorithms that are able to determine the optimal patching-strategy for a defender, who aims to deny the attacker specific capabilities. Once again, using real world data, I demonstrate that this approach works well in practice to determine efficient defender actions for not just a single system, but a network of systems as well.

While these approaches are useful, especially in the case of active and deployed systems, it is important to note that they are fundamentally *reactive*; that is, they rely on vulnerabilities having been disclosed. Disclosure can occur in two ways: vulnerabilities are either detected by security researchers or from analysis of malware in the wild. The former is preferable; if the discovery of vulnerabilities in existing software can be enhanced, security experts and software developers can address them before they make it into the wild, where they can be exploited by hackers.

In general, a vulnerability is any aspect of a computer system that allows for breaches in its security policy, and often such vulnerabilities are caused by software bugs [18]. There are numerous software-engineering paradigms and approaches that aim to minimize software-bugs, such as test-driven development [19]. However, writing test cases rely on identifying problematic inputs and verifying that the program responds to them appropriately; either with an error response or at least failing

without exposing a vulnerability. Since it is impossible to explicitly enumerate all possible invalid inputs, an approach called "fuzzing" can been used, which provides randomized inputs to a program in an attempt to make it crash [20]. The input and the state of the program after the crash can then be examined to ensure that there are no vulnerabilities (like buffer overflows) that can be exploited. While random fuzzing has allowed the discovery of many vulnerabilities, the process is fundamentally stochastic. Thus, not finding a bug in software that was randomly fuzzed does not necessarily imply that one or more do not exist.

To address this fundamental issue, approaches were designed to provide greater feedback to the fuzzing process, in order to make it more efficient and to increase the likelihood of discovering vulnerabilities. AFL, a coverage-based fuzzer, was the first to use this new paradigm [21], employing a genetic algorithm that guides it towards generating inputs that maximize branch coverage.

Due to AFL's success, modern fuzzing research has largely focused on maximizing *code coverage*, reasoning that enhancing code-coverage triggers more vulnerabilities. Interestingly, increased code coverage is not always correlated with discovered vulnerabilities: Shoshitaishvili et al. showed that fuzzers could use human-assisted input mutation to find more vulnerabilities—*while triggering strictly less code*—than standard (automated) code-coverage guided fuzzers [22]. One explanation is that not all code coverage is equal, and that coverage alone does not imply good exploration of a, program's *state space*.

Ijon furthered this concept using manual source-code annotations to extend traditional code coverage feedback with *state-representative variables* [23]. The resulting *state-coverage guided fuzzer* showed promise in—with human help—triggering program states that standard code-coverage guided fuzzers cannot reach.

The next clear step in exploring the impact of state coverage on analysis outcomes is, of course, *automatic* reasoning about state-representative variables. To address this problem, I develop an approach called SandPuppy, that can automate the process of identifying and using state-representative variables in fuzzing to increase state coverage, and thus detecting *state-dependent* bugs. Specifically, my approach attempts to identify different types of state-representative variables in program source, and instruments them to provide semantically-corresponding feedback to the fuzzer, with the aim to increase exploration of program state-space. To investigate the viability of this hypothesis, I conduct the following experiments: (1) compare state-representative variables identified by SandPuppy to those identified by human experts in Ijon, (2) compare exploration of deep program-state by SandPuppy against current state-of-the-art approaches, (3) compare basic-block coverage by SandPuppy against state-of-the-art approaches, and (4) compare vulnerability discovery by SandPuppy against state-of-the art approaches. The results from these experiments convincingly demonstrate that my approach is able to identify state-representative variables, and that the feedback from these variables leads to a better and deeper exploration of program state-space compared to current state-of-the-art techniques. Consequently SandPuppy is also able to improve coverage and vulnerability detection compared to those techniques.

Chapter 2

BACKGROUND

2.1 Cybersecurity Threat Modeling and Mitigation

Due to the persistent and increasing threat of cyberattacks, cybersecurity has been an extremely active field of research featuring a diversity of approaches. These approaches generally try to model attacks/attacker behavior and identify defensive strategies to prevent or mitigate such attacks.

Attacker-Defender Models. Over the years, attacker-defender models have proved useful in simulating attacker and defender actions. These models have been used to address various security problems, such as optimal network-security hardening and resource-allocation for malicious packet-detection. Research aimed at addressing power-grid and critical-infrastructure vulnerabilities have been able to successfully model attacker-initiated cascading-failures in these systems [24], [25] as well as identify defender strategies to mitigate such failures. The act of exploitation however, is viewed as binary (either a system/node is exploited or it is not) and these approaches ignore the mechanics and details of the attacks; specifically the motivation of the attacker to gain specific capabilities on the target system(s).

Game-theoretic Approaches. Game theory has been applied to security situations, and was first popularized by its application to airport security [26]. Its use has been extended to multiple other scenarios, including the placement of security

checkpoints [27], assignment of air marshals to flights [28], and more pertinently, to identify attacker and defender strategies in cybersecurity [29]–[31]. While useful, a drawback of these models is their high-level approach, which ignores the discrete nature of capabilities and how they affect an attacker's ability to compromise systems.

Sequential Cyberattacks. Approaches involving the use of exploit preconditions and postconditions to model sequential-attacks were previously introduced in [13], [14], where Petri nets [32] were used to model cyberattacks. These approaches are computationally-intensive, which makes it difficult to use them in practice. Given the popularity of sequential cyberattacks, it is clear that computationally-efficient approaches that provide constructive results are necessary.

Effective Patching Strategies. The identification of effective patching strategies have been explored in [15]–[17]. The issue once again, is that these approaches overlook the discrete nature of capabilities and the motivations of attackers to gain *specific* capabilities on target systems. Accounting for this information would allow more targeted defense and contingency strategies.

2.2 Greybox Fuzzing

Due to its success, AFL effectively spawned an entirely new field of fuzzing research with significant interest from both academia [23], [33]–[38] and industry [39]–[41]. In general, such approaches are called "greybox fuzzing" in contrast to the earlier "blackbox" or "dumb" fuzzing approaches, because they provide greater visibility into

the internal state of the program being fuzzed.

Coverage-Guided Fuzzing. As previously mentioned in Chapter 1, modern fuzzing research has largely focused on increasing code-coverage. This is usually done by satisfying previously-failing program branch conditions, whether by solving them using symbolic techniques [37], [38], identifying specific parts of program input that should be stochastically mutated to increase the chance of satisfying conditions [33], [34], or modifying the conditions themselves [35]. As a result of these interventions, code coverage is increased and additional bugs are found. A persistent drawback of these approaches however, is their lack of visibility into deep and complex states.

Domain-Specific Fuzzing. This form of fuzzing takes a narrower and more focused approach than coverage-guided fuzzing and has been successfully applied to certain deep-state fuzzing problems [42]–[46]. PerfFuzz [43] and SlowFuzz guide the fuzzer towards pathological inputs that could expose algorithmic-complexity vulnerabilities, while MemFuzz [45] augments coverage information by accounting for memory accesses. While successful, the highly focused nature of these approaches limit their applicability as target programs may not expose the necessary state.

State-Aware Fuzzing. Recent work in this area has made progress toward detecting state-representative variables [47]–[49], but either uses such variables purely to increase code coverage (rather than *state* coverage) [47] or is only applicable to specific types of state-representative variables [48] or software [49]. A more generic approach that provides *semantically-appropriate* feedback based on the type of state-representative

variable would allow for more targeted exploration of deep and complex program-states across a variety of programs.

Chapter 3

REASONING ABOUT SEQUENTIAL CYBERATTACKS

Contemporary cyber-threat actors rely on a variety of malware and exploits purchased through various channels such as the darkweb. These exploits are usually part of sophisticated "exploit kits" that target vulnerabilities and leverage their interdependencies to form "chained attacks". In this chapter, I introduce a formalism that allows modeling adversarial-actions against a system with possibly-interdependent vulnerabilities using associated exploits. I present an operator that I show has a least fixed-point, which corresponds to the precise set of capabilities obtained by the attacker under the framework and is efficient to compute. I address the problem of determining the optimal/most-dangerous strategy for a cyber-adversary with respect to this model and find it to be NP-Complete. I also present a suite of provably-correct algorithms based on A* search to solve this problem. I develop an admissible heuristic, and show how I enhance algorithm performance by incorporating fixed-point operator results and by using a memoization approach that I also introduce. With these techniques, I see an average performance-improvement of 82% over standard A^*. I demonstrate the performance of these algorithms and the validity of the model through a suite of experiments using both simulated and actual vulnerability-data.

3.1 Model

In this section, I define a formal mathematical-model to capture the semantics of chained attacks that leverage vulnerability and exploit interdependencies. The

model assumes that there is an attacker with an initial set of "capabilities". These capabilities represent system functionality that the attacker has access to, and also represent different actions that the attacker can take on or against the system. Using different sets of initial capabilities, I can model various attacker-types, such as unauthenticated/authenticated remote-attackers, local users, etc. The set of initial capabilities can also reflect prior knowledge gained by scanning or profiling a target system. Using their capabilities the attacker employs a sequence of exploits, where each subsequent exploit uses capabilities provided by a previous one. Through this iterative approach, the attacker gradually compromises the system until they have achieved their objective.

I first define a set V, which is the set of all vulnerabilities, and a set C, which is the set of capabilities supported by a component on the information systems network. I use the notation 2^C to represent the powerset of C. Following the intuition of [14], [50]–[53], I assume each exploit has two sets of capabilities: C^r representing preconditions and C^g representing postconditions. These capabilities can represent high-level interactions (e.g., HTTP requests), local commands on the system, and malicious capabilities gained by successfully using an exploit:

Definition 3.1. Given a set $C^r \subseteq C$ of capabilities required to use an exploit for a vulnerability $v \in V$, and a set $C^g \subseteq C$ of capabilities gained by using that exploit, the set of exploits E is a set of tuples of the form (C^r, C^g, v).

This formalism captures the necessary conditions required to use a particular vulnerability exploit, and also models what the attacker gains by using the exploit. It also lets me model the attack surface of the system. For example, I can model an "exploit" that grants privileged capabilities with the precondition that the user is able to escalate their privilege. Similarly, a "passive" attack-surface can also be modeled as

an "exploit" with no required capabilities; this is useful to model services running on a system, such as web servers. I acknowledge that deriving these capability sets from CVE descriptions is challenging. However, there are approaches [54], [55] that have solved this problem with some success, which can be easily adapted to work with my model. Note that it is possible for multiple exploits to target a single vulnerability.

Example 3.1. *The identities of most vulnerabilities have been standardized and they are collectively known as* Common Vulnerabilities and Exposures *(CVE) [56]; these vulnerabilities themselves are uniquely identified by a CVE number, and each entry contains detailed information which includes the manner in which the vulnerability can be exploited (i.e., the attack vector, and what commands to send), and the capabilities gained by an attacker from doing so (e.g., "remote code-execution", "denial of service"). A set of CVE's (identified by their CVE number) is therefore a good instantiation of the set V. The set V can also contain other "vulnerabilities"; consider that any accessible machine presents an attack surface of some sort, which can be considered to be a vulnerability as well. Elements in set C are abstractions of available functionality on a system. As such they do not necessarily represent an actual command recognized by the machine's operating-system; an element could also represent a series of commands, a capability obtained by exploiting a vulnerability, a high-level interaction like a Hyper-Text Transfer Protocol (HTTP) request, or even a physical capability, such as inserting a thumbdrive. For example:*

- *If I consider a Linux server with an un-patched Shellshock [57] vulnerability, running version 2.6 of the Linux kernel (susceptible to a privilege-escalation exploit [58]), the set V for this machine can be defined as* {`attack-surface`, `2014-6271`, `2009-1185`}*, and the set C defined as* {`get_request`, `ls`, `cd`, `uname`, `esc_priv`, `shutdown`, `reboot`, ...}.

- *If I consider a Siemens Simatic S7-300 Programmable Logic Controller (PLC) with some unpatched vulnerabilities, the set V for this machine can be defined as* $\{$`attack-surface`, `2016-9159`, `2016-9158`, `2016-3949`$\}$, *and the set C defined as* $\{$`get_request`, `esc_priv`, `denial_svc`$\}$.

While I use CVEs to uniquely identify vulnerabilities in this chapter, the model is not necessarily dependent on them and can handle any kind of vulnerability, including zero-days.

I assume that vulnerability dependencies can be derived based on capabilities; exploiting a vulnerability can provide capabilities that can be subsequently used to exploit a different vulnerability. I now define an exploit chain as follows:

Definition 3.2. Given some set $E' \subseteq E$, and initial set of capabilities $C_0 \subseteq C$, an **exploit chain** (denoted $E_{C_0,E'}$) is a subset of E':

$$\langle e_n \mid n \in \mathbb{N} \rangle = \langle (C_1^r, C_1^g, v_1), \ldots, (C_i^r, C_i^g, v_i), \ldots, (C_n^r, C_n^g, v_n) \rangle$$

where:

$$C_1^r \subseteq C_0$$

and for each consecutive pair $(C_i^r, C_i^g, v_i), (C_j^r, C_j^g, v_j)$ I have:

$$C_j^r \subseteq C_0 \cup \bigcup_{k<j} C_k^g$$

Example 3.2. *Assume that the attacker has a set of exploits E' that consists of the following tuples:*

- $(\{$`esc_priv`$\}, \{$`shutdown`, `reboot`, $\ldots\}$, `attack-surface`$)$
- $(\{$`tkey_query`$\}, \{$`dos`$\}$, `2015-5477`$)$
- $(\{$`get_request`$\}, \{$`ls`, `cd`, `uname`, `mmap`, $\ldots\}$, `2014-6271`$)$
- $(\{$`uname`$\}, \{$`esc_priv`$\}$, `2009-1185`$)$

- $(\{\texttt{mmap}\}, \{\texttt{esc_priv}\}, \texttt{2009-2692})$

Assuming that the attacker's initial set of capabilities $C_0 = \{\texttt{get_request}\}$*, and that they want to gain the capability to reboot and shutdown the machine, an exploit chain that would lead them to those capabilities is:*

1. $(\{\texttt{get_request}\}, \{\texttt{ls}, \texttt{cd}, \texttt{uname}, \texttt{mmap}, \ldots\}, \texttt{2014-6271})$
2. $(\{\texttt{uname}\}, \{\texttt{esc_priv}\}, \texttt{2009-1185})$
3. $(\{\texttt{esc_priv}\}, \{\texttt{shutdown}, \texttt{reboot}, \ldots\}, \texttt{attack-surface})$

Note that for a given E' *and* C_0 *there can be multiple exploit-chains that allow an attacker to obtain a particular set of capabilities. This means that an attacker has multiple options at their disposal. For example, the attacker can also use the following exploit chain to gain the ability to reboot and shutdown the machine:*

1. $(\{\texttt{get_request}\}, \{\texttt{ls}, \texttt{cd}, \texttt{uname}, \texttt{mmap}, \ldots\}, \texttt{2014-6271})$
2. $(\{\texttt{mmap}\}, \{\texttt{esc_priv}\}, \texttt{2009-2692})$
3. $(\{\texttt{esc_priv}\}, \{\texttt{shutdown}, \texttt{reboot}, \ldots\}, \texttt{attack-surface})$

An attacker can use different exploit chains to achieve various goals; there may even be multiple exploit-chains that lead to the same goal. By accounting for all these chains, I can identify the complete set of capabilities that the attacker can gain; I call this the **obtained set of capabilities**. The intuition is that this set establishes an upper bound on the set of capabilities an attacker can gain, for a given set of initial capabilities and exploits. It easily translates to the maximum amount of damage an attacker can cause to the system.

Definition 3.3. Given a set of exploits $E' \subseteq E$ and a set of initial capabilities $C_0 \subseteq C$, the **obtained set of capabilities** (denoted C^*_{E',C_0}) consists of all capabilities in C_0,

in addition to any capability $c \in C$ for which there exists an exploit chain $E_{C_0,E'}$ leading to c, and does not contain any capability $c' \in C$ such that $c' \notin C_0$, and for which there exists no exploit chain $E_{C_0,E'}$ leading to c'.

Essentially, given some initial set of capabilities C_0, the attacker can apply a sequence of exploits constructed from E' to eventually gain a certain set of capabilities. Hence for every capability $c \in C^*_{E',C_0}$, there is a subsequence $\langle (C_1^r, C_1^g, v_1), \ldots, (C_i^r, C_i^g, v_i) \rangle$ such that $c \in C_i^g$. In addition, if there is a capability $c \notin C^*_{E',C_0}$ the implication is that there is no exploit chain that can lead the attacker to that capability, making it impossible to obtain.

3.1.1 Fixed-Point Operator

I now design an operator that models exploit application and prove that it has a fixed point, that corresponds to the **obtained set of capabilities**. I also show that the operator converges in a polynomial number of steps and investigate some other useful properties. A key assumption that I make here is that exploit application is monotonic; applying an exploit does not remove a previously-obtained capability. This assumption is quite common in related work [50], [59], [60] in this area. In [61], the authors also note that non-monotonic attacks can be treated as monotonic, if certain low-level details are ignored. As such information is not provided, or difficult to obtain from CVE and exploit descriptions, the monotonicity assumption is reasonable.

When an attacker wants to compromise a system through a chained attack, they start with some initial vulnerability exploit. Using the capabilities provided by this initial exploit, they go on to exploit a different vulnerability. By repeating

this process, the attacker cumulatively gains capabilities allowing them to exploit additional vulnerabilities, eventually leading them to their desired goal:

Definition 3.4. I introduce the notion of exploit application via an "exploit-application operator":

i. Given $E' \subseteq E$ and $C_0 \in 2^C$ I define the exploit-application operator $\mathbf{T}_{E'} : 2^C \rightarrow 2^C$ as:

$$\mathbf{T}_{E'}(C_0) = C_0 \cup \bigcup \{C^g \mid (C^r, C^g, v) \in E' \wedge C^r \subseteq C_0\}$$

ii. Given some $i \in \mathbb{N}$, I define the ith application of $\mathbf{T}_{E'}$ on $C_0 \subseteq C$ as:

$$\mathbf{T}_{E'}\uparrow_i(C_0) = \mathbf{T}_{E'}(\mathbf{T}_{E'}\uparrow_{i-1}(C_0))$$

where:

$$\mathbf{T}_{E'}\uparrow_1(C_0) = \mathbf{T}_{E'}(C_0)$$

iii. $\mathbf{T}^*_{E'}$, the fixed point of $\mathbf{T}_{E'}$ is defined as:

$$\mathbf{T}^*_{E'}(C_0) = \mathbf{T}_{E'}\uparrow_i(C_0)$$

where:

$$\mathbf{T}_{E'}\uparrow_i(C_0) = \mathbf{T}_{E'}\uparrow_{i+1}(C_0)$$

Example 3.3. *Assuming the same situation described in Example 3.2, consider what happens after three applications of* $\mathbf{T}_{E'}$*. For the initial application of* $\mathbf{T}_{E'}$ *I have:*

$$\begin{aligned} \mathbf{T}_{E'}\uparrow_1(C_0) &= \mathbf{T}_{E'}(C_0) && \textit{(item (ii) Definition 3.4)} \\ &= \{\texttt{get_request}, \texttt{ls}, \texttt{cd}, \texttt{uname}, \ldots\} && \textit{(Example 3.2)} \end{aligned}$$

The second time I apply $\mathbf{T}_{E'}$*, the exploit for vulnerability* `2009-1185` *satisfies conditions in Item (ii) of Definition 3.4, hence:*

$$\begin{aligned}\mathbf{T}_{E'}\uparrow_2(C_0) &= \mathbf{T}_{E'}(\mathbf{T}_{E'}\uparrow_0(C_0)) \\ &= \{\texttt{get_request}, \texttt{ls}, \texttt{cd}, \texttt{uname}, \texttt{esc_priv}, \ldots\}\end{aligned}$$

Finally, with the attack-surface "exploit" satisfying conditions in Item (i) of Definition 3.4, I have:

$$\begin{aligned}\mathbf{T}_{E'}\uparrow_3(C_0) &= \mathbf{T}_{E'}(\mathbf{T}_{E'}\uparrow_1(C_0)) \\ &= \{\texttt{get_request}, \texttt{ls}, \texttt{cd}, \texttt{uname}, \texttt{esc_priv}, \texttt{shutdown}, \texttt{reboot}, \ldots\}\end{aligned}$$

This demonstrates that after the third application of $\mathbf{T}_{E'}$*, the attacker can gain access to all capabilities on the system. Notice that the attacker gains no additional benefit from subsequent invocations, showing that the fixed-point was reached in the third application of the operator. Hence in this example,* $\mathbf{T}^*_{E'}(C_0) = \mathbf{T}_{E'}\uparrow_3(C_0)$*.*

In the above definition, Item (i) describes a single application of the operator, which augments the attacker's initial capabilities using C^g from only those exploits where $C^r \subseteq C_0$. Item (ii) models the iterative behavior of the attacker; on each subsequent application of the operator, it uses the augmented capability-set from the previous application, potentially allowing the attacker to exploit additional vulnerabilities. Item (iii) defines the end of this iterative process by defining the fixed-point of the operator, implying that the iterative process must converge, or in real-world terms, the attacker stops when they cannot gain any more capabilities. I can easily prove that this fixed-point exists using lattice theory by first making the following observation about 2^C:

Observation 3.1. $\langle \subseteq, 2^C \rangle$ is a partial ordering and 2^C specifies a complete lattice.

This is straightforward as $\subseteq$ is clearly reflexive, transitive, and antisymmetric. Further, the set 2^C has a clear top element (set C) and a bottom element (the empty set). As C is simply a set of elements, the powerset as under the ordering relationship specified by $\subseteq$ is the classic example of a complete lattice. I can now prove that the fixed-point exists:

Theorem 3.1. $\mathbf{T}_{E'}$ has a least fixed point.

Proof.

Claim 1. $C_0 \subseteq \mathbf{T}_{E'}(C_0)$.

Let C_0 be the attacker's existing set of capabilities. From Item (i) in Definition 3.4, I can say that $\mathbf{T}_{E'}(C_0)$ is the union of C_0 and another set. Hence $\mathbf{T}_{E'}$ is order increasing.

Claim 2. If $C_0 \subseteq C_0'$ then $\mathbf{T}_{E'}(C_0) \subseteq \mathbf{T}_{E'}(C_0')$.

Assume that I have a C_0, C_0' where $C_0 \subseteq C_0'$, and for brevity:

$$X' = \bigcup\{C^g \mid (C^r, C^g, v) \in E' \wedge C^r \subseteq C_0\}$$
$$X'' = \bigcup\{C^g \mid (C^r, C^g, v) \in E' \wedge C^r \subseteq C_0'\}$$

I then have $\mathbf{T}_{E'}(C_0) = C_0 \cup X'$ and $\mathbf{T}_{E'}(C_0') = C_0' \cup X''$. Note that $C_0 \subseteq C_0'$, which implies that $X' \subseteq X''$. Hence it is immediately obvious that $\mathbf{T}_{E'}(C_0) \subseteq \mathbf{T}_{E'}(C_0')$, which proves monotonicity.

Proof of theorem: By Observation 3.1, and claims 1 and 2, I can apply Knaster-Tarski [62] which means that $\mathbf{T}_{E'}$ has a least fixed-point.

□

The set of capabilities at the least fixed-point represents the complete set of capabilities that the attacker can gain for a given E' and C_0. This is useful because this set corresponds to the **obtained set of capabilities** (Definition 3.3):

Theorem 3.2. Given an $E' \subseteq E$ and $C_0 \in 2^C$ with obtained set of capabilities C^*_{E',C_0}, $\mathbf{T}^*_{E'}(C_0) \subseteq C^*_{E',C_0}$ and $\mathbf{T}^*_{E'}(C_0) \supseteq C^*_{E',C_0}$.

Proof.

Claim 1. $\mathbf{T}^*_{E'}(C_0) \subseteq C^*_{E',C_0}$

By way of contradiction, assume that $\mathbf{T}^*_{E'}(C_0) \nsubseteq C^*_{E',C_0}$. This means that there exists a c such that $c \in \mathbf{T}^*_{E'}(C_0)$ and $c \notin C^*_{E',C_0}$. Note that if $c \in \mathbf{T}^*_{E'}(C_0)$, then it must mean that there is some j^{th} application such that $c \in \mathbf{T}_j{\uparrow}_{E'}(C_0)$. Observe that I can easily construct a valid exploit-chain that leads to c, using the applicable exploits identified at this application. This means that if $c \in \mathbf{T}^*_{E'}(C_0)$, then there must necessarily be an exploit chain leading to c. But based on Definition 3.3, because there is an exploit chain leading to c, then $c \in C^*_{E',C_0}$. Hence, a contradiction.

Claim 2. $\mathbf{T}^*_{E'}(C_0) \supseteq C^*_{E',C_0}$

By way of contradiction, assume that $\mathbf{T}^*_{E'}(C_0) \nsupseteq C^*_{E',C_0}$. This implies that there exists a c such that $c \notin \mathbf{T}^*_{E'}(C_0)$ and $c \in C^*_{E',C_0}$. Note that if $c \notin \mathbf{T}^*_{E'}(C_0)$, then it must mean that there is no j^{th} application such that $c \in \mathbf{T}_j{\uparrow}_{E'}(C_0)$. This can happen if either $c \notin C_0$, $c \notin \bigcup\{C^g \mid (C^r, C^g, v) \in E'\}$ or for every exploit $(C^r, C^g, v) \in E'$ where $c \in C^g$, there is no j where $C^r \subseteq \mathbf{T}_{E'}{\uparrow}_{j-1}(C_0)$. This implies that it is impossible to construct a valid exploit-chain, meaning that if $c \notin \mathbf{T}^*_{E'}(C_0)$, then there is no exploit chain leading to c. But based on Definition 3.3, because there is no exploit chain leading to c, then $c \notin C^*_{E',C_0}$. Hence, a contradiction.

□

A useful property of the operator is that it also identifies mutually disjoint exploit-sets associated with each application:

Definition 3.5. Given a $E' \subseteq E$ and $C_0 \in 2^C$ with an $i \in \mathbb{N}$ such that $\mathbf{T}^*_{E'}(C_0) = \mathbf{T}_{E'}\uparrow_i(C_0)$, I can identify the set of exploits used at any $j \in \mathbb{N}$ where $j \leq i$ as:

$$E'_j = \{(C^r, C^g, v) \in E' \setminus \bigcup_{k=1}^{j-1} E'_k \mid C^r \subseteq \mathbf{T}_{E'}\uparrow_{j-1}(C_0)\}$$

where:

$$E'_1 = \{(C^r, C^g, v) \in E' \mid C^r \subseteq C_0\}$$

I can use this to show that the operator converges in a polynomial number of steps; more specifically, the number of steps cannot exceed the total number of exploits available to the attacker

Proposition 3.1. Given $E' \subseteq E$ and $C_0 \in 2^C$ with $i \in \mathbb{N}$ such that $\mathbf{T}^*_{E'}(C_0) = \mathbf{T}_{E'}\uparrow_i(C_0)$, $i \leq |E'|$.

Proof. From Definition 3.5, note that at every $1 \leq j \leq i$ I must have used at least one exploit. This is because otherwise, there would be no difference in the set of capabilities gained at application j compared to $j-1$, which would imply that the fixed point was reached at $j-1$; I know that this definitely cannot be the case. In addition, I also know that I have an upper bound of $|E'|$ on the number of exploits that can be used overall. Hence for every $1 \leq j \leq i$, I can say that $1 \leq \left|E'_j\right| \leq |E'|$ and additionally, $\sum_{j=1}^{i} \left|E'_j\right| \leq |E'|$. Note that even by using one exploit at every application, the total number of applications can never exceed the total number of exploits used by the attacker. That is, even in the worst case, when $\left|E'_j\right| = 1$ for $1 \leq j \leq i$, $\sum_{j=1}^{i} \left|E'_j\right| = |E'|$. Hence $i \leq |E'|$.

□

Exploit sets identified in Definition 3.5 have some interesting properties. Observe that the operator effectively "sorts" E' into an ordered sequence of subsets using dependency information; an exploit in E'_j will have required capabilities provided by at least one exploit from E'_{j-1}. This gives me information about the structure of possible exploit-chains as I now have a general idea about the order in which I expect attackers to use certain exploits.

3.1.2 Fixed-Point Operator Complexity and Memoization

I now investigate the complexity of calculating the fixed point. Observe that for a given C_0 and E', the complexity of calculating $\mathbf{T}_{E'}(C_0)$ is $\Theta(|E'|)$ as I iterate over the entire set. Given a set $E'' \supseteq E'$, observe that I can calculate $\mathbf{T}_{E''}(C_0)$ using $\mathbf{T}_{E'}(C_0)$ via the expression $\mathbf{T}_{E'}(C_0) \cup \mathbf{T}_{E''\setminus E'}(C_0)$. Hence, I can state the following proposition:

Proposition 3.2. Given $E'' \subseteq E$, $E' \subseteq E$ and $C_0 \in 2^C$, if $E' \subseteq E''$ then $\mathbf{T}_{E'}(C_0) \subseteq \mathbf{T}_{E''}(C_0)$

Proof. By way of contradiction, assume that I have $E' \subseteq E''$ such that $\mathbf{T}_{E'}(C_0) \nsubseteq \mathbf{T}_{E''}(C_0)$. Let $E'' = E' \cup \{(C^{r'}, C^{g'}, v')\}$ such that $C^{r'} \subseteq C_0$. Using Item (i) from Definition 3.4 I have:

$$\begin{aligned}\mathbf{T}_{E''}(C_0) &= \bigcup\{C^g \mid (C^r, C^g, v) \in E'' \wedge C^r \subseteq C_0\} \\ &= \bigcup\{C^g \mid (C^r, C^g, v) \in E' \cup \{(C^{r'}, C^{g'}, v')\} \wedge C^r \subseteq C_0\}\end{aligned}$$

As $C^{r'} \subseteq C_0$,I can rewrite the above expression as:

$$\begin{aligned}\mathbf{T}_{E''}(C_0) &= \bigcup\{C^g \mid (C^r, C^g, v) \in E' \wedge C^r \subseteq C_0\} \cup C^{g'} \\ &= \mathbf{T}_{E'}(C_0) \cup C^{g'}\end{aligned}$$

Since the union operation only adds elements to a set, I can conclude that $\mathbf{T}_{E'}(C_0) \subseteq \mathbf{T}_{E''}(C_0)$, which contradicts my assumption. □

Example 3.4. *Assume I have the following contrived exploits:*

- $(\{a\}, \{e, f\}, v_1)$
- $(\{b\}, \{g, h\}, v_2)$
- $(\{c\}, \{i, j\}, v_3)$
- $(\{d\}, \{k, l\}, v_4)$

If I set $C_0 = \{a, b, c, d\}$ *and* $E' = \{(\{a\}, \{e, f\}, v_1), (\{b\}, \{g, h\}, v_2)\}$*, I can see that* $\mathbf{T}_{E'}(C_0)$ *gives me* $\{a, b, c, d, e, f, g, h\}$*. Now let* $E'' = \{(\{a\}, \{e, f\}, v_1), (\{b\}, \{g, h\}, v_2), (\{c\}, \{i, j\}, v_3), (\{d\}, \{k, l\}, v_4)\}$*. If I memoized the result of* $\mathbf{T}_{E'}(C_0)$*, I can express* $\mathbf{T}_{E''}(C_0)$ *as* $\mathbf{T}_{E'}(C_0) \cup \mathbf{T}_{E''\setminus E'}(C_0)$*. This gives me the result* $\{a, b, c, d, e, f, g, h\} \cup \{a, b, c, d, i, j, k, l\}$*, or* $\{a, b, c, d, e, f, g, h, i, j, k, l\}$*. Note that this is equivalent to the result of* $\mathbf{T}_{E''}(C_0)$ *without memoization.*

In the worst case, calculating the fixed point takes $|E'|$ steps, which makes the upper bound on the complexity $O(|E'|^2)$. The lower bound is $\Omega(|E'|)$ as it is conceivable for all exploits to apply on the very first application.

I also extend this memoization to the calculation of the fixed-point. Note that the complexity here is $\Theta(k\,|E'|)$, where k is the number of applications taken to reach the fixed-point ($1 \leq k \leq |E'|$). As before, given a set $E'' \supseteq E'$, I can intuitively see that it is possible to use the result of $\mathbf{T}^*_{E'}(C_0)$ when calculating $\mathbf{T}^*_{E''}(C_0)$. Hence:

Proposition 3.3. Given $E'' \subseteq E$, $E' \subseteq E$ and $C_0 \in 2^C$, if $E' \subseteq E''$ then $\mathbf{T}^*_{E'}(C_0) \subseteq \mathbf{T}^*_{E''}(C_0)$

Proof. Based on Item (ii) from Definition 3.4 I know that there is a $i, j \in \mathbb{N}$ such that $\mathbf{T}_{E'}\uparrow_i(C_0) = \mathbf{T}^*_{E'}(C'_0)$ and $\mathbf{T}_{E''}\uparrow_j(C_0) = \mathbf{T}^*_{E''}(C'_0)$. I will prove by induction that for all $n \geq 1$, $\mathbf{T}_{E'}\uparrow_n(C_0) \subseteq \mathbf{T}_{E''}\uparrow_n(C_0)$, which implies that $\mathbf{T}^*_{E'}(C'_0) \subseteq \mathbf{T}^*_{E''}(C'_0)$.

Base case: When $n = 1$ I have $\mathbf{T}_{E'}\uparrow_1(C_0) \subseteq \mathbf{T}_{E''}\uparrow_1(C_0)$, which I know to be true based on Item (ii) from Definition 3.4 and Proposition 3.2.

Induction step: Let $k \in \mathbb{N}$. Assume that $\mathbf{T}_{E'}\uparrow_k(C_0) \subseteq \mathbf{T}_{E''}\uparrow_k(C_0)$, and furthermore, let $X' = \mathbf{T}_{E'}\uparrow_k(C_0)$ and $X'' = \mathbf{T}_{E''}\uparrow_k(C_0)$. Then:

$$\begin{aligned}\mathbf{T}_{E'}\uparrow_{k+1}(C_0) &= \mathbf{T}_{E'}(\mathbf{T}_{E'}\uparrow_k(C_0)) \\ &= \mathbf{T}_{E'}(X')\end{aligned}$$

and:

$$\begin{aligned}\mathbf{T}_{E''}\uparrow_{k+1}(C_0) &= \mathbf{T}_{E''}(\mathbf{T}_{E''}\uparrow_k(C_0)) \\ &= \mathbf{T}_{E''}(X'')\end{aligned}$$

Based on my assumption, $\mathbf{T}_{E'}\uparrow_k(C_0) \subseteq \mathbf{T}_{E''}\uparrow_k(C_0)$, which implies that $X' \subseteq X''$. Hence, based on claim 2 in theorem 3.1, $\mathbf{T}_{E'}(X') \subseteq \mathbf{T}_{E''}(X'')$ and therefore $\mathbf{T}_{E'}\uparrow_{k+1}(C_0) \subseteq \mathbf{T}_{E''}\uparrow_{k+1}(C_0)$.

Conclusion: By induction, $\mathbf{T}_{E'}\uparrow_n(C_0) \subseteq \mathbf{T}_{E''}\uparrow_n(C_0)$ for all $n \geq 1$, which implies that $\mathbf{T}^*_{E'}(C'_0) \subseteq \mathbf{T}^*_{E''}(C'_0)$ $\square$

I will now try to express $\mathbf{T}^*_{E''}(C_0)$ in terms of $\mathbf{T}^*_{E'}(C_0)$. Since I do not necessarily use every exploit in E' when calculating $\mathbf{T}^*_{E'}(C_0)$, I cannot exclude E' outright as with the memoization of $\mathbf{T}_{E''}(C_0)$. This is because exploits in $E'' \setminus E'$ may provide capabilities that can make previously unused exploits from E' applicable. Hence I must exclude only those exploits that were used to calculate $\mathbf{T}^*_{E'}(C_0)$. I define these exploits as follows:

Definition 3.6. Given a $E' \subseteq E$ and $C_0 \in 2^C$, with an $i \in \mathbb{N}$ such that $\mathbf{T}^*_{E'}(C_0) = \mathbf{T}_{E'}\uparrow_i(C_0)$, the complete set of applicable exploits E_{use} is defined as:

$$E_{use} = \bigcup_{1 \leq j \leq i} E'_j \qquad \text{(Definition 3.5)}$$

I can now express $\mathbf{T}^*_{E''}(C_0)$ in terms of $\mathbf{T}^*_{E'}(C_0)$ as $\mathbf{T}^*_{E''}(C_0) = \mathbf{T}^*_{E''\setminus E_{use}}(\mathbf{T}^*_{E'}(C_0))$. Notice that the runtime is now $\Theta(k' \, |E'' \setminus E_{use}|)$ instead of $\Theta(k \, |E''|)$ (where $k' \leq k$).

Example 3.5. *Assume I have the following, contrived exploits:*

- $(\varnothing, \{a\}, v_1)$
- $(\{a\}, \{b\}, v_2)$
- $(\{b\}, \{c\}, v_3)$
- $(\{c\}, \{d\}, v_4)$
- $(\{d\}, \{e\}, v_5)$

If I set $C_0 = \varnothing$ and $E' = \{(\varnothing, \{a\}, v_1), (\{a\}, \{b\}, v_2), (\{b\}, \{c\}, v_3), (\{c\}, \{d\}, v_4)\}$, I can see that $\mathbf{T}^*_{E'}(C_0)$ gives me $\{a, b, c, d\}$. Notice also, that $E_{use} = E'$ as well, as I have used every single exploit. Now let me set $E'' = \{(\varnothing, \{a\}, v_1)$, $(\{a\}, \{b\}, v_2)$, $(\{b\}, \{c\}, v_3)$, $(\{c\}, \{d\}, v_4)$, $(\{d\}, \{e\}, v_5)\}$. From Definition 3.6, I can see that $\mathbf{T}^*_{E''}(C_0) = \mathbf{T}^*_{E''\setminus E_{use}}(\mathbf{T}^*_{E'}(C_0))$, which means that the memoized fixed-point operator only needs to operate over the single, additional exploit $(\{d\}, \{e\}, v_5)$, with initial

capabilities set to $\mathbf{T}^*_{E'}(C_0)$*, or* $\{a, b, c, d\}$*. This gives me the result* $\{a, b, c, d, e\}$*, which is exactly the same as the result of* $\mathbf{T}^*_{E''}(C_0)$*. Hence, instead of taking five applications to calculate the fixed-point, the memoized operator only takes one.*

3.1.3 Preferred Attack Strategy

I now address the problem of identifying the attacker's strategy. Attackers decide what exploits to use by profiling a system, or by having knowledge about the kinds of systems used in a network. With this information, they can use public vulnerability databases to look up associated vulnerabilities providing specific capabilities. However, the attacker may not have the expertise necessary to exploit such vulnerabilities and could consider purchasing exploit kits from the darkweb. Note that by using the darkweb as a resource, it is possible for the attacker to purchase exploit-kits that target undisclosed (zero-day) vulnerabilities as well. Both expertise and financial cost can be expressed using a cost function:

Definition 3.7. Given a set of exploits E, I define a cost function $cost : E \to \mathbb{R}^+$ that associates a real-valued cost with each exploit.

For simplicity, I use a single cost-function throughout this , but all of the results can be extended for separate ones. Cost functions can also take into account other factors, such as the probability that a vulnerability has been patched, as attackers may not have perfect knowledge about a system. This information can be used by the attacker to "weight" the costs of associated exploits in order to factor in this probability. In ongoing work, I am currently looking at data-driven cost-functions [10], [11] as well.

Since attackers do not have unlimited resources, it makes sense to define a budget. Attackers would prefer to use a set of exploits that let them achieve their goal without exceeding this budget. This is the attacker's preferred strategy and is defined as follows:

Definition 3.8. Given the attacker's budget $b \in \mathbb{R}^+$, set of desired capabilities C_d, and initial set of capabilities C_0, the **preferred attack-strategy** $\mathrm{PAS}(C_0, C_d)$ is the set of exploits E' satisfying the following conditions:

- Coverage: $\mathbf{T}^*_{E'}(C_0) \supseteq C_d$
- Cost: $\sum_{e \in E'} cost(e) \leq b$

Note that the result of Theorem 3.1 is important here since it shows that it takes a finite number of steps to calculate the fixed point and verify the solution. In the optimization variant of the preferred attack-strategy problem, the quantity $\sum_{e \in E'} cost(e)$ is minimized, and the associated strategy is called an ***optimal strategy***.

Identifying the attacker's preferred and optimal strategies gives me a number of advantages. Defenders can easily assess how vulnerable their systems are, and how easy it would be for attackers to compromise them. It is also possible for defenders to vary the attacker's initial capabilities, allowing them to observe how the system responds to different attacker types. For instance, a system that is secure from remote attackers may be particularly vulnerable to a malicious local-user. Defenders can also vary the budget to account for the attacker's expertise or financial resources (for example, to model a lone hacker, a hacker group, or state actor). The optimal attack-strategy is also useful to the defender as it can establish a lower bound on the attacker's expertise and financial resources for the worst-case outcome. Finally, the

strategies themselves provide hints as to which vulnerabilities should be prioritized for patching.

Finding a preferred attack-strategy is NP-complete, and is easily shown by a reduction from set cover.

Theorem 3.3. Finding a preferred attack-strategy is NP-complete.

Proof.

Claim 1. Finding a preferred attack-strategy is in-NP.

Finding the preferred attack-strategy is clearly in NP. Given the attacker's set of desired capabilities C_d, initial set of capabilities C_0, set of exploits E', and budget b, I can calculate the total exploit-cost and $\mathbf{T}^*_{E'}(C_0)$ in polynomial time to verify that E' is the preferred attack-strategy (theorem 3.1 and proposition 3.1).

Claim 2. Finding a preferred attack-strategy is NP-hard.

I can show that finding the preferred attack-strategy is NP-hard by constructing an instance of the problem from an instance of the Set Cover problem. The Set Cover problem is stated as follows: given a set of sets $U = \{S_1, S_2, \ldots, S_n\}$ and a number k, find a $S' \subseteq U$ such that $|S'| \leq k$ and $\bigcup_{S_i \in S'} S_i = \bigcup_{S_i \in U} S_i$. I will construct an instance of the preferred attack-strategy problem from an arbitrary instance of the Set Cover problem using the algorithm SetCover2PAS, which is defined as follows:

- Build the set of vulnerabilities $V = \{v_i \mid S_i \in U\}$.
- Build the global set of capabilities and the attacker's desired set of capabilities $C = C_d = \bigcup_{S_i \in U} S_i$.
- Build the set of available exploits $E = \{(\varnothing, S_i, v_i) \mid S_i \in U\}$.
- Set the attacker's initial set of capabilities $C_0 = \varnothing$, and the attacker's budget $b = k$.

- I define the cost-function for any exploit $e \in E$ as $cost(e) = 1$.

Claim 2.1. SetCover2PAS runs in polynomial time.

The algorithm SetCover2PAS clearly runs in polynomial time (specifically $O(|U|)$), since the construction of each of the sets described in the algorithm only requires me to iterate over elements in U.

Claim 2.2. If there is a solution to the Set Cover problem, then there exists a preferred attack-strategy.

Suppose by way of contradiction, there is a set cover S' of U such that $|S'| \leq k$, which does not have a corresponding preferred-attack strategy. By my construction, S' corresponds to a set of exploits $E' = \{(\varnothing, S_i, v_i) \mid S_i \in S'\}$. Since I set $b = k$, E' clearly contains at most b exploits, and therefore $\sum_{e \in E'} cost(e) \leq b$. I also know that $\mathbf{T}_{E'}(C_0) = \mathbf{T}_{E'}(\varnothing) = \varnothing \cup \bigcup\{S_i \mid (\varnothing, S_i, v_i) \in E'\}$, since every exploit satisfies the conditions in Item (i) of Definition 3.4. Hence one iteration of $\mathbf{T}_{E'}$ is the fixed point and:

$$\begin{aligned}
\mathbf{T}^*_{E'}(\varnothing) &= \mathbf{T}_{E'}(\varnothing) \\
&= \varnothing \cup \bigcup\{S_i \mid (\varnothing, S_i, v_i) \in E'\} \\
&= \bigcup_{S_i \in S'} S_i
\end{aligned}$$

Based on my construction, I also know that $C = C_0 = \bigcup_{S_i \in U} S_i$. Since S' is a cover of U, it follows that $\bigcup_{S_i \in S'} S_i = \bigcup_{S_i \in U} S_i$. But I have shown that $\mathbf{T}^*_{E'}(C_0) = \bigcup_{S_i \in S'} S_i$, which means that $\mathbf{T}^*_{E'}(C_0) = C = C_d$. Therefore E', which corresponds to a solution S' of the Set Cover problem, is the *preferred attack-strategy*. Hence, a contradiction.

Claim 2.3. If there is no solution to the Set Cover problem, then there does not exist a preferred attack-strategy.

Suppose by way of contradiction, there is a preferred attack-strategy E' without a corresponding solution to the Set Cover problem. For the constructed problem, the fixed-point operator always converges after a single iteration, giving me $\mathbf{T}^*_{E'}(C_0) = \bigcup\{S_i \mid (\varnothing, S_i, v_i) \in E'\}$. Given that the desired set of capabilities is defined in the constructed problem as $C_d = \bigcup_{S_i \in S'} S_i$, this means that $\mathbf{T}^*_{E'}(C_0) \supseteq C_d$, which also implies that it is a cover. Likewise, I also know that $|E'| \leq k$. Therefore $\mathbf{T}^*_{E'}(C_0)$ satisfies the criteria to be a cover, which implies that a solution S' to the Set Cover problem must exist. Hence, a contradiction.

Proof of theorem: Follows directly from claims 1 and 2.

□

3.2 Algorithms

In this section I examine algorithms to solve the preferred attack-strategy problem. Our baseline approach used depth-first search across the strategy-space. A major shortcoming with this approach is that it has exponential-time complexity and an optimal solution is not guaranteed. It is also obvious that the algorithm explores paths that are unproductive or invalid; for example, paths containing exploits whose preconditions have not been met, or exploits that provide capabilities the attacker has already obtained. To address these issues I enable more-efficient searches for attacker strategies through the following improvements that maintain correctness:

1. I correctly prune the available exploits at each step and employ the use of an admissible heuristic-function by adopting A* search.
2. I further improve A* search by using the results of the fixed-point operator.

3.2.1 Pruning Exploits

The depth-first search approach considers exploits that need not or cannot be part of the solution. To address this problem I prune the search-tree by discarding such exploits. An exploit is included and not pruned only if the following conditions hold:

1. The exploit does not cause the attacker to exceed their budget.
2. The attacker currently has the required set of capabilities to apply the exploit.
3. The exploit offers at least one capability that the attacker does not already have.

The correctness of this pruning technique follows directly from my original model – hence I state the following proposition:

Proposition 3.4. Given the attacker's initial capabilities C_0, current set of exploits E', and budget b, PAS-DFS-PRUNED ignores all exploits $e = (C^r, C^g, v)$ if $cost(e) + \sum_{e' \in E'} cost(e') > b$, $C^r \not\subseteq \mathbf{T}^*_{E'}(C_0)$, or $C^g \subseteq \mathbf{T}^*_{E'}(C_0)$, and returns a solution if one exists.

Proof. I prove the statement of the theorem by first stating the precondition, loop invariant, and postcondition, and showing that these hold during all iterations.

Precondition: *nodes* is a Last In First Out (LIFO) queue of nodes that contains a root node of a search-tree that may or may not contain correct solutions.

Loop invariant: Aside from the root node, all nodes in *nodes* correspond to applicable exploits (which can include goal-state nodes).

Postcondition: If the search-tree does not contain any correct solutions, the algorithm reports a failure; otherwise the algorithm reports the corresponding solution.

I will now show that these conditions hold during all iterations:

Initialization: At the first iteration, *nodes* initially contains only the root node. Hence, the invariant holds.

Maintenance: At iteration n, I assume that *nodes* contains nodes corresponding to applicable exploits, which can also include goal-state nodes. Note that at iteration $n+1$, I can pick a node *node* out of *nodes* if it is not empty. While the algorithm terminates if *node* is a goal-state node or if *nodes* was empty, the invariant still holds because *nodes* is not modified in either case. If *node* is not a goal-state node, I make a call to EXPAND where I create and return corresponding-nodes for only those exploits where the call to PRUNE returned *false*. I can see that PRUNE returns *true* if any of the following conditions hold, for a given exploit e:

1. $parent.\text{PATHCOST} + cost(e) > b$
2. $e.C^r \not\subseteq \mathbf{T}^*_{E'}(C_0)$
3. $e.C^g \subseteq \mathbf{T}^*_{E'}(C_0)$

This means that each corresponding-node that I add to *nodes* must either represent an applicable exploit or a goal-state node. Hence the invariant holds at iteration $n+1$ as well.

Termination: Note that the algorithm terminates if *nodes* is empty or if the head node satisfies the goal test. In the former case, it implies that the search tree did not contain any correct solutions, because otherwise *nodes* would have had a goal-state node at the head during some iteration. In the latter case it implies that the search-tree has at least one correct-solution, because the head-node was a goal-state node; in either case both the loop invariant and the postcondition hold.

Since the precondition, loop invariant, and postcondition hold during all iterations, the algorithm PAS-DFS-PRUNED not only prunes inapplicable exploits correctly, but will return a correct solution if one exists. □

An additional advantage of PAS-DFS-PRUNED over PAS-DFS is that the solution it returns is a valid exploit-chain, due to the fact that pruning leaves me with only those exploits that can be applied, given the attacker's current set of capabilities.

3.2.2 PAS-A*

While pruning helps me address time-complexity and can provide better solutions than DFS, it still does not guarantee an optimal solution. To address this issue I use A* search (Algorithm Algorithm 3.1). With an admissible heuristic, the tree-search variant of A* is both complete and optimal. A* evaluates nodes by combining $g(e)$, the cost to reach the node, and $h(e)$, the lower bound of the cost to get from the node to the goal.

Algorithm 3.1 Implementation of PAS-A*

```
procedure PAS-A*(E, C_0, C_d, b):
    function PATH(node):
        path as SET
        while node.PARENT ≠ ∅ do:
            ADD(path, node.EXPLOIT)
            node ← node.PARENT
        return path
    function SOLUTION(node):
        return PATH(node)
    function ROOT:
        root as NODE
        root.PARENT ← ∅
        root.EXPLOIT ← ∅
        root.PATHCOST ← 0
        return root
    function MAKENODE(parent, e):
        node as NODE
        node.PARENT ← parent
        node.EXPLOIT ← e
        node.PATHCOST ← parent.PATHCOST + cost(e)
        return node
    function PRUNE(parent, e):
        if parent.PATHCOST + cost(e) > b :
            return true
        else if e.C^r ⊈ T*_{E'}(C_0) :
            return true
        else if e.C^g ⊆ T*_{E'}(C_0) :
            return true
        return false
```

```
function Expand(node):
    children as Set
    for each e in E \ Path(node):
        if not Prune(node, e) :
            Add(children, MakeNode(node, e))
    return children
function RemainingCapabilities(node):
    E' ← Path(node.Parent)
    return C_d \ T*_{E'}(C_0) \ C^g
function RemainingExploits(node, C_rem):
    E_rem as Set
    for each e in E \ Path(node):
        if e.C^g ∩ C_rem ≠ ∅ :
            Add(E_rem, e)
    return E_rem
function EstimatedCost(node):
    C_rem ← RemainingCapabilities(node)
    E_rem ← RemainingExploits(node, C_rem)
    return node.PathCost + h(C_rem, E_rem)
nodes as Priority-Queue ordered by EstimatedCost
Enqueue(nodes, Root)
loop do:
    if Empty(nodes) :
        return failure
    node ← Dequeue(nodes)
    E' ← Solution(node)
    if T*_{E'}(C_0) ⊇ C_d :
        return E'
    for each child in Expand(node):
        if not Exists(nodes, child) :
            Enqueue(nodes, child)
        else:
            existing ← Find(nodes, child)
            f_existing ← EstimatedCost(existing)
            f_child ← EstimatedCost(child)
            if f_child < f_existing :
                Replace(nodes, existing, child)
```

To estimate the cost to the goal, I first look at how many desired capabilities remain to be gained:

Definition 3.9. Given the attacker's current set of exploits E', initial set of capabilities C_0, desired set of capabilities C_d, and the applicable exploit under consideration (C^r, C^g, v), I define the remaining capabilities as $C_{rem} = C_d \setminus \mathbf{T}^*_{E'}(C_0) \setminus C^g$.

These capabilities must come from the set of remaining applicable exploits, which I define as follows:

Definition 3.10. Given the complete set of exploits E, the attacker's current set of

exploits E', and the applicable exploit under consideration e, I define the remaining set of exploits as $E_{rem} = \{(C^r, C^g, v) \in E \setminus E' \setminus \{e\} \mid C^g \cap C_{rem} \neq \varnothing\}$.

Note that if $E_{rem} = \varnothing$ while $C_{rem} \neq \varnothing$, it implies that there is no solution since there are no remaining exploits that provide any of the remaining desired-capabilities.

I now estimate the cost to the goal by identifying the lowest-possible cost to gain each remaining capability, and then summing those costs:

Definition 3.11. Given the node e that represents an applicable exploit under consideration, the remaining set of capabilities C_{rem}, and the remaining set of exploits E_{rem}, the estimated cost to the goal $h : E \to \mathbb{R}^+$ is defined as:

$$h(e) = \sum_{c \in C_{rem}} \min_{\{e' \in E_{rem} | c \in C^g\}} \frac{cost(e)}{|C^g \cap C_{rem}|}$$

$$\text{where } e' = (C^r, C^g, v)$$

Theorem 3.4. $h(e)$ is admissible.

Proof. I prove the statement of the theorem by showing that the cost of an optimal path from node e to the goal can never exceed $h_2(e)$.

By way of contradiction, consider an optimal path to the goal from node e with true cost $h^*(e)$ such that $h_2(e) > h^*(e)$. Let E^* be the corresponding set of exploits. Then:

$$\sum_{c \in C_{rem}} \min_{\{e' \in E_{rem} | c \in C^g\}} cost_r(e') > \sum_{e' \in E^*} cost(e')$$

Note that since E^* represents the optimal path, the following is also true:

$$\sum_{c \in C_{rem}} \min_{\{e' \in E_{rem} | c \in C^g\}} cost_r(e') > \sum_{c \in C_{rem}} \min_{e' \in E^*} cost_r(e')$$

Algorithm 3.2 Implementation of heuristic h

```
function h(C_rem, E_rem):
    h as REAL
    h ← 0
    for each c in C_rem:
        h_min ← ∞
        for each e in E_rem:
            if c ∈ e.C^g :
                h_e ← cost(e) ÷ |e.C^g ∩ C_rem|
                if h_e < h_min :
                    h_min ← h_e
        h ← h + h_min
    return h
```

This implies that there exists a c such that:

$$\min_{\{e' \in E_{rem} | c \in C^g\}} cost_r(e') > \min_{\{e' \in E^* | c \in C^g\}} cost_r(e')$$

Let $e_{min,c}$ and $e^*_{min,c}$ be the corresponding exploits with lowest cost per remaining-capability. Then:

$$cost_r(e_{min,c}) > cost_r(e^*_{min,c})$$

However, for $e_{min,c}$ to have been used in calculating $h_2(e)$ instead of $e^*_{min,c}$, the following must hold:

$$cost_r(e_{min,c}) < cost_r(e^*_{min,c})$$

This is a contradiction, which means that $h^*(e)$ must not be the true cost to the goal. Hence, $h_2(e)$ is admissible. □

Algorithm 3.2 is an implementation of $h(e)$.

3.2.3 Enhancing PAS-A* Using Fixed-Point Operator Results

While PAS-A* identifies optimal solutions, I can improve its runtime performance by making use of the fixed-point operator's results. Observe that Definition 3.6 lets

Algorithm 3.3 Enhanced A* (common functions omitted)

```
1:  procedure PAS-ENHANCED-A*(E, C_0, C_d, b):
2:      function PRUNE(parent, e):
3:          if e.i < parent.EXPLOIT.i or e.i − parent.EXPLOIT.i > 1 :
4:              return true
5:          else if parent.PATHCOST + cost(e) > b :
6:              return true
7:          else if e.C^r ⊈ T*_{E'}(C_0) :
8:              return true
9:          else if e.C^g ⊆ T*_{E'}(C_0) :
10:             return true
11:         return false
12:     function REMAININGEXPLOITS(node, C_rem):
13:         E_rem as SET
14:         for each e in E \ PATH(node):
15:             if e.i ≥ node.EXPLOIT.i and e.C^g ∩ C_rem ≠ ∅ :
16:                 ADD(E_rem, e)
17:         return E_rem
18:     ⟨E_use, C⟩ ← T*_E(C_0)          ▷ Assume that fixed-point operator returns used exploits.
                                          Implementation is trivial and has been left out for brevity.
19:     E ← E_use                        ▷ Redefine E to include only used exploits.
20:     ▷ Remaining code identical to PAS-A*
```

me identify the complete set of exploits that an attacker can possibly use, given their initial set of capabilities. Hence the preferred strategy can never contain an exploit that exists in $E' \setminus E_{use}$. By having PAS-A* consider only those exploits in E_{use}, I can reduce the search space. Additional improvements can be gained using Definition 3.5, which identifies the set of exploits used at each application. By labeling each exploit from E_{use} with the application in which it was used, I can prune the search tree even further. Assume that I am expanding a node associated with exploit e^i, which was used in the ith application. I can then prune any exploit e^j where $j < i$ or $j > i + 1$, because a subsequent exploit must have either been used in the same application as the preceding one, or in the very next application. I also use this information to improve heuristic performance by excluding all remaining exploits that were used in applications earlier than the exploit under consideration. Algorithm 3.3 is an implementation of PAS-ENHANCED-A*.

Hence, I can state the following proposition:

Proposition 3.5. Given the attacker's initial capabilities C_0, desired capabilities C_d,

current set of exploits E', and budget b, PAS-A* returns an optimal solution if one exists.

Proof. I prove the statement of the theorem by first stating the precondition, loop invariant, and postcondition, and showing that these hold during all iterations.

Precondition: *nodes* is a priority-queue of nodes, ordered by EstimatedCost in ascending order, that contains the root node of a search-tree that may or may not contain correct solutions.

Loop invariant: Aside from the root node, all nodes in *nodes* correspond to applicable exploits (which can include goal-state nodes). *nodes* also contains at most one node corresponding to a particular exploit, representing the best-possible path to the goal through that node, identified thus far.

Postcondition: If the search-tree does not contain any correct solutions, the algorithm reports a failure; otherwise the algorithm reports the corresponding solution, which is the optimal one.

I will now show that these conditions hold during all iterations:

Initialization: At the first iteration, *nodes* initially contains only the root node. Hence, the invariant holds.

Maintenance: At iteration n, I assume that *nodes* contains nodes corresponding to applicable exploits, which can also include goal-state nodes. I also assume that *nodes* also contains at most one node corresponding to a particular exploit, representing the best-possible path to the goal through that node, identified thus far.

Note that at iteration $n + 1$, I can pick a node *node* out of *nodes* if it is not empty. While the algorithm terminates if *node* is a goal-state node or if *nodes* was empty, the invariant still holds because *nodes* is not modified in either case.

If *node* is not a goal-state node, I make a call to EXPAND where I create and return corresponding nodes for applicable exploits only (Proposition 3.4). Note that I operate on the priority queue only in the following cases:

1. The node does not already exist in *nodes*, implying that this is the first time I am seeing a path that goes through this node. In this case, I will add the node to the priority queue.
2. A node corresponding to this exploit already exists in *nodes*, but with a higher estimated-cost for the path running through this node to the goal. In this case, I replace the existing node with my new node that has a lower estimated-cost for the path running through this node to the goal.

This means that each corresponding node that I add to *nodes* either represents an exploit I have not seen before, or replaces an existing node corresponding to the same exploit, that has a higher estimated-cost for the path running through that node to the goal. Hence the invariant holds at iteration $n + 1$ as well.

Termination: Note that the algorithm terminates if *nodes* is empty or if the head node satisfies the goal test. In the former case, it implies that the search tree did not contain any correct solutions, because otherwise *nodes* would have had a goal-state node at the head during some iteration. Hence, both the loop invariant and the postcondition hold. In the latter case it implies that I must have found the optimal solution. Notice that the invariant holds in this case because I do not modify the priority queue any further once I find a solution. Observe that the postcondition also holds; the goal-state node that was dequeued must necessarily have had the lowest estimated-cost to the goal (the path cost in this case, as $h(e) = 0$ when $C_{rem} = \varnothing$). This is because the priority queue is ordered by estimated-cost, meaning that best-

possible path to the goal identified thus far necessarily goes through the head node of the priority queue.

Since the precondition, loop invariant, and postcondition hold during all iterations, the algorithm PAS-A* will return an optimal solution if one exists. □

PAS-A* (see Algorithm 3.1) is an implementation of A* that will use either h_1 or h_2, based on the one supplied to the algorithm. Given that I have proved both heuristics to be admissible, and have also proved the correctness, PAS-A* is guaranteed to produce an optimal solution.

3.2.4 Improving Fixed-Point Operator Performance

I implement fixed-point operator memoization by means of a trie-based [63] cache. I first associate a unique integer with each exploit, allowing me to order a set of them consistently. I can then cache the results in a trie (see Figure 1 for an example), giving me lookup time that is linear in the size of the largest subset of E' cached so far. Note that in this approach, I assume that C_0 is fixed.

While the upper bound on the operator's complexity given some arbitrary E' is quadratic even with caching, it is effectively linear when used with algorithms PAS-A* and PAS-Enhanced-A*. As the path to any node in the search tree only consists of applicable exploits, every exploit is used when calculating the fixed-point ($E_{use} = E'$). Furthermore I grow the path a single exploit at a time, making the complexity of calculating $\mathbf{T}^*_{E''}(C_0)$ (where $E'' = E' \cup \{e\}$) just $O(|E'| + 1)$ or $O(|E''|)$. For example, given $E' = \{e_2, e_3, e_4, e_5\}$ the look-up time is $O(|\{e_2, e_3, e_4\}|)$ since that is the greatest subset of E'.

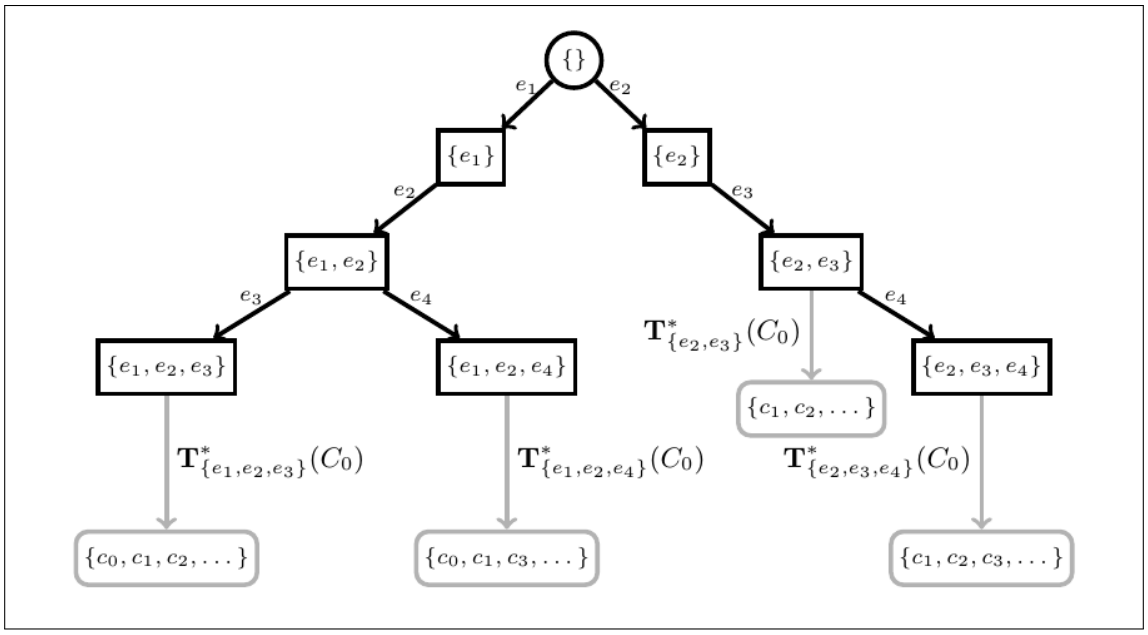

Figure 1. Example of a trie used to cache results of the fixed-point operator.

3.3 Experiments and Results

My experiments were designed to evaluate performance and to examine the viability of my model and its algorithms in real-world scenarios. All experiments except one were implemented in Java. The remaining experiment, which analyzed the performance of my memoization approach, was implemented in JavaScript on NodeJS. All experiments were executed on a machine with an Intel Core i7-4770K processor and 24GB RAM running Linux Mint 18.3.

My first experiment assesses the impact of memoization on $\mathbf{T}^*_{E'}$. I generated a set of exploits $E' = \{e_0, e_1, \ldots, e_n\}$ of the form $\{(\varnothing, \{c_0\}, v_0), (\{c_0\}, \{c_1\}, v_1), \ldots, (\{c_{n-1}\}, \{c_n\}, v_n)\}$. I then measured the runtime of memoized and non-memoized versions of $\mathbf{T}^*_{E'}$ against the following sequence of subsets of E': $\langle\{e_0\}, \{e_0, e_1\}, \ldots, \{e_0, e_1, \ldots, e_i\}, \ldots, \{e_0, e_1, \ldots, e_n\}\rangle$. From the results in Figure 2, it is evident

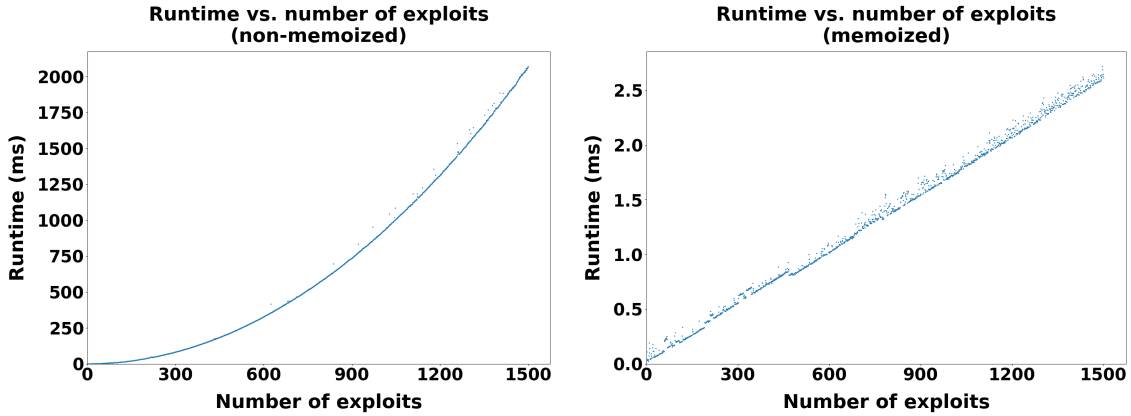

Figure 2. Runtime vs. number of exploits for non-memoized and memoized $\mathbf{T}^*_{E'}$.

that runtime for the non-memoized version is quadratic, compared to linear for the memoized version. This confirms that memoization significantly improves the performance of $\mathbf{T}^*_{E'}$.

The next set of experiments use real-world vulnerability data gathered from the National Institute of Standards and Technology's National Vulnerability Database (NIST NVD). I defined 22 exploits based on 10 Critical Vulnerability Exploits (CVEs) associated with the Windows 10 operating system. I chose CVEs that involve a diverse spectrum of malicious capabilities and also feature a good distribution of attacker types and complexity levels. The cost of each exploit was set to the sum of the attack and access complexities of the associated CVE; these metrics are extracted from the Common Vulnerability Scoring System (CVSS) base-score vector reported for each CVE in the NVD. Both types of complexities are graded as High, Medium, or Low, which I associated with integer values 3, 2, and 1. I chose these specific metrics as they describe how easy or difficult it is to exploit a particular vulnerability. Exploit capability-sets were defined manually, based on CVE descriptions and CVSS vector attributes. I also defined four types of attackers: a **remote attacker** with remote access to the machine, an **authenticated remote-attacker** with remote access and

valid credentials for the machine, a **physically-proximate attacker** with physical and network access to the machine, and a **local attacker** with physical access and valid credentials for local use of the machine. C_d for all attackers was defined to contain the following capabilities: **remote code-execution**, **privilege escalation**, **installing a crafted boot-manager**, and **denial of service**. C_0 for each attacker was defined to reflect their level of access to the machine, with the remote attacker having the least number of initial capabilities and the local attacker having the most.

Attacker Type	$\lvert\mathbf{T}_E^*(C_0)\rvert$	$\lvert E'\rvert$	**Cost**
Remote	13	10	20
Remote authenticated	13	8	16
Physically proximate	18	9	15
Local	22	7	11

Table 1. Number of obtained capabilities, and size and cost of optimal solution based on attacker type.

My hypothesis for this experiment was that attackers with greater initial-access to the system would be able to cause more damage than those with lower access, given identical sets of exploits. The results (Table 1) clearly validate this hypothesis, as attackers with a greater number, or with more significant initial-capabilities, obtain a larger set of capabilities. This also shows that the results of the fixed-point operator are realistic and match real-world expectations.

The next experiment investigates how the preferred strategy (as identified by PAS-A*) varies across attacker types. Each attacker's budget was set to be unlimited. The results (Table 1) show that in general, solution costs are inversely proportional to the number and significance of initial capabilities. This is to be expected, as attackers with more initial access to a system would be able to compromise it using fewer resources. These results provide additional confirmation that the model is able to

produce results that are not only accurate, but also make sense intuitively and reflect real-world attacker behavior.

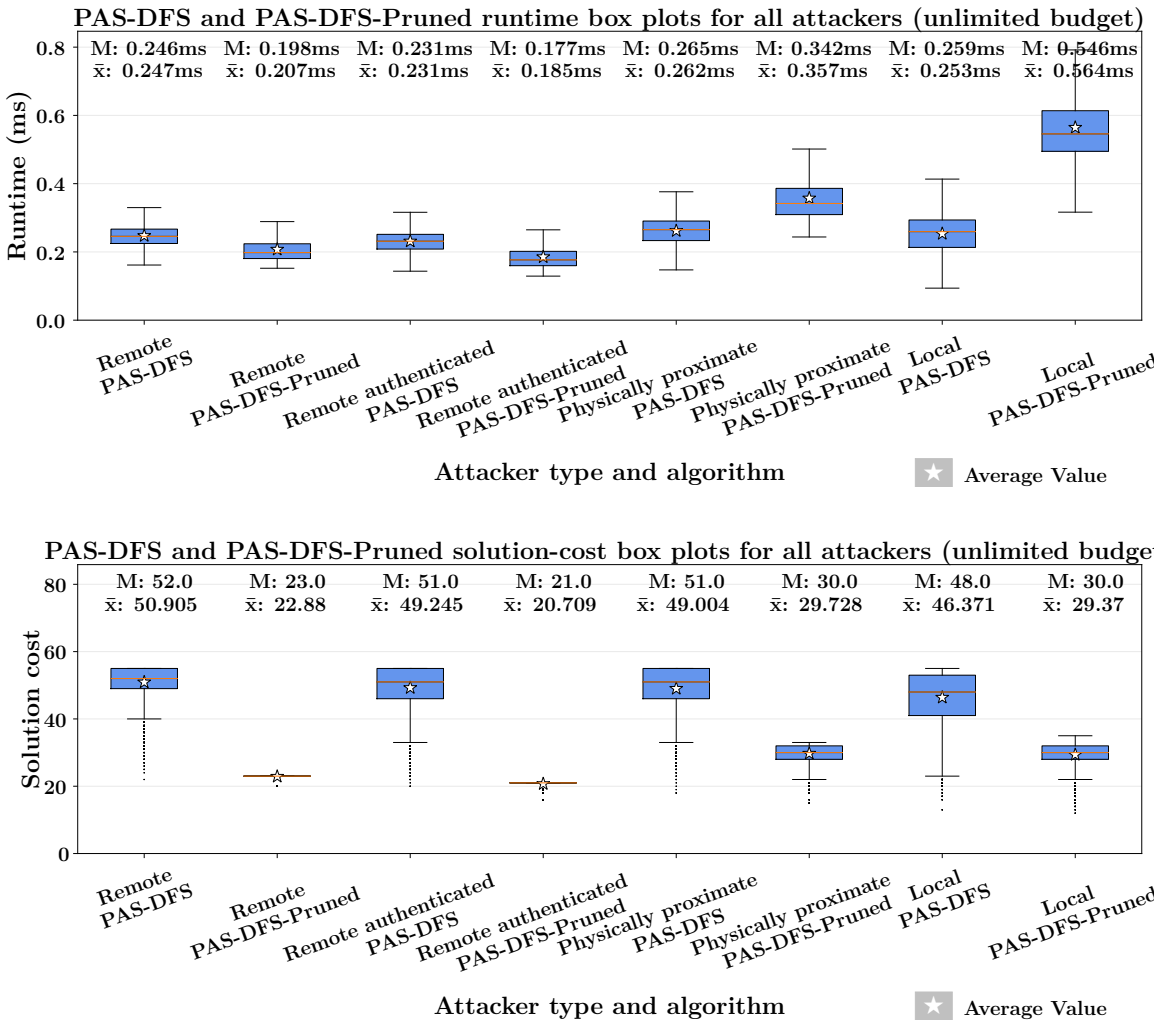

Figure 3. Box plot of runtimes and solution costs for PAS-DFS and PAS-DFS-PRUNED, against different attacker types (1000 samples each) with unlimited budgets. Some outliers excluded for clarity.

The aim of my next experiment was to examine the effectiveness of my pruning approach by running PAS-DFS and PAS-DFS-PRUNED against each attacker type,

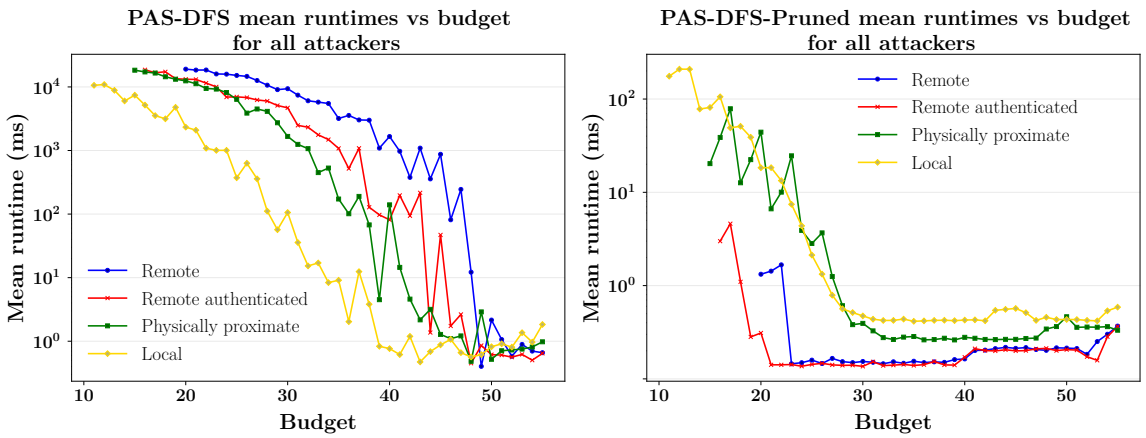

Figure 4. Mean runtime vs. budget for PAS-DFS and PAS-DFS-PRUNED against different attacker types with restricted budgets (100 and 1000 samples respectively).

initially with unlimited budgets. The results (Figure 3) show that while the average runtime of PAS-DFS-PRUNED is generally not lower than PAS-DFS, the solutions identified are cheaper and qualitatively better. The low runtimes of PAS-DFS are easily explained by the fact that with an unlimited budget, a degenerate solution (one that contains all exploits) is likely to be quickly identified. In particular, note that the average cost of solutions identified by PAS-DFS is quite close to the total exploit-cost of 55, showing that a large proportion of solutions identified include most or all exploits. This is also evidence of the effectiveness of pruning, as PAS-DFS-PRUNED discards paths that contain such solutions which, though they provide the attacker with their desired capabilities, are typically more expensive as they include redundant or invalid exploits (see items 2 and 3 in 3.2.1). The runtime advantage of pruning is more evident if budgets are restricted to be lower than the total cost of all available exploits. I looked at the mean runtime as a function of the budget for each algorithm, against each attacker type, by varying the budget from 55 (the total cost of all exploits) down to the cost of the optimal solution for each attacker. I observed that in many cases, especially as the budgets approached the cost of the optimal solution,

PAS-DFS took more than an hour to complete. Hence I capped the execution time to 20 seconds, which is an order of magnitude higher than the highest-observed runtime for PAS-DFS-Pruned (2 seconds). The results (Figure 4) clearly show that PAS-DFS-Pruned is much faster on average. With restricted budgets that are lower than the total cost of all available exploits, PAS-DFS cannot identify degenerate solutions and must therefore explore more, usually unproductive paths, that PAS-DFS-Pruned ignores. As the budget approaches the cost of the optimal solution, the runtime advantage of PAS-DFS-Pruned is especially evident. These results show that my pruning approach significantly lowers the size of the search space, and helps identify solutions that are much cheaper and of higher quality than those identified by PAS-DFS.

In my next experiment, I investigate how memoization and attacker type affect performance. Attacker budgets were once again set to be unlimited. As expected, the results (Figure 5) show that memoization improves runtime in all cases; on average I see a 27% decrease in runtime over non-memoized implementations. Generally, runtimes also appear to be directly proportional to the number of initial capabilities. This is because more initial capabilities allow more exploits to be applicable, resulting in a larger search-space due to an increased branching-factor. Finally, I see that the memoized implementation of PAS-Enhanced-A* performs the best, with a runtime that is on average 82% lower than standard A* (PAS-A* without memoization). When comparing to the baseline approach (Figures 3 and 4), it can be seen that with unrestricted budgets, runtimes for all PAS-A* variants are higher; this is to be expected as there are far more sub-optimal solutions than optimal ones, and thus it is more likely that one would be found quickly by PAS-DFS or PAS-DFS-Pruned. However, I can clearly see that when budgets approach the cost of the optimal solution,

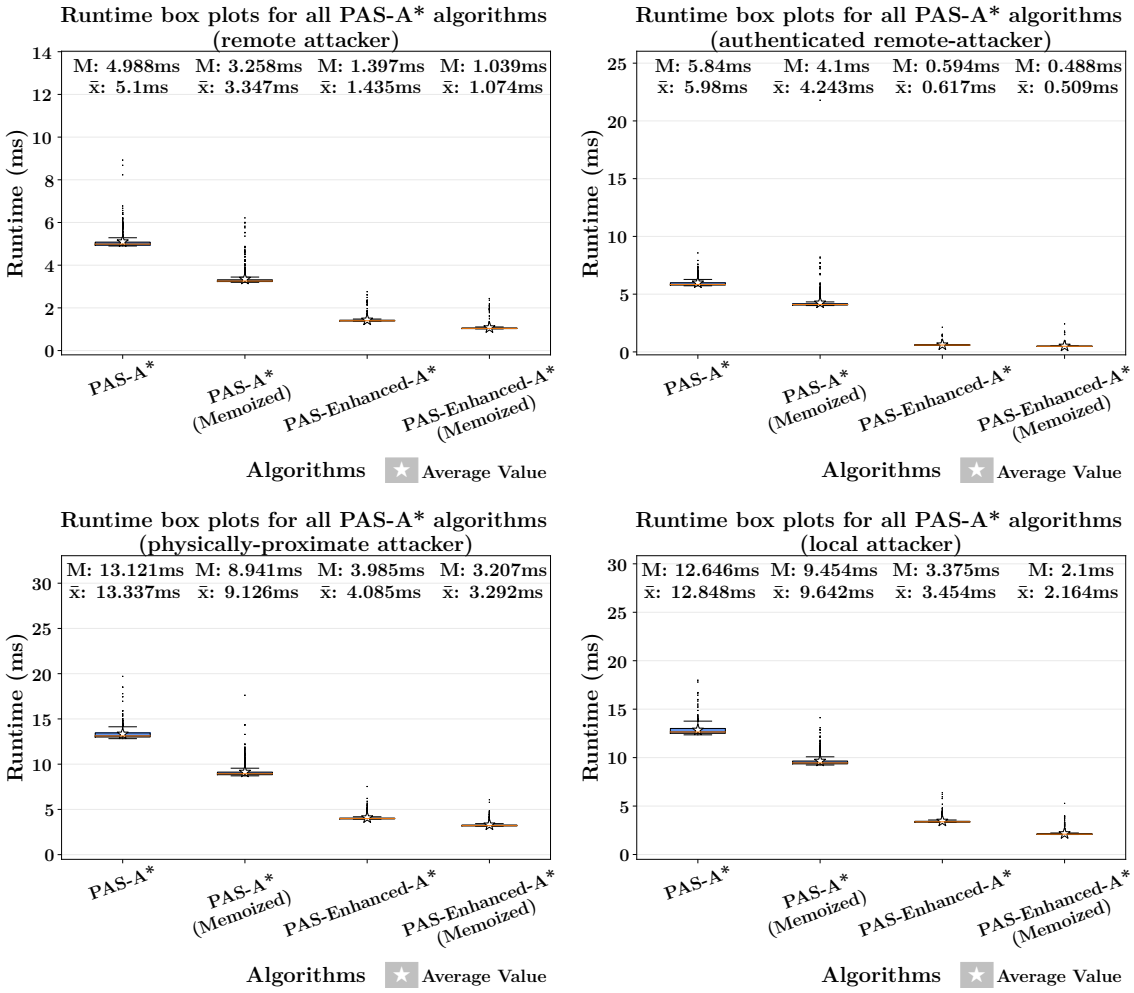

Figure 5. Box plot of runtimes for all PAS-A* algorithms (memoized and non-memoized) against different attacker types (1000 samples each).

PAS-DFS is in many cases unable to find a solution in a reasonable time, whereas PAS-DFS-Pruned is able to find a solution, but takes far longer than any of the PAS-A* algorithms. These results clearly show that PAS-A* and its variants are far superior to the baseline approach.

The final experiment investigates how the attacker's skill-level affects the solution. The initial budget for all attackers is set to the size of their optimal solution as determined from previous experiments. To model attackers with different skill-levels,

I multiply their initial budget by an associated factor: 1 for low skill, 2 for medium skill, and 3 for highly skilled. The results (Table 2) suggest that attackers need to have a medium skill-level at the very least, in order to compromise the system. This also makes sense intuitively, as I expect attackers with greater expertise to be able to exploit more-complex vulnerabilities.

Attacker Type	**Low skill**	**Medium skill**	**High skill**
Remote	None	20 ($\|E'\| = 10$)	20 ($\|E'\| = 10$)
Remote authenticated	None	16 ($\|E'\| = 8$)	16 ($\|E'\| = 8$)
Physically proximate	None	15 ($\|E'\| = 9$)	15 ($\|E'\| = 9$)
Local	None	11 ($\|E'\| = 7$)	11 ($\|E'\| = 7$)

Table 2. Solution costs and sizes for attackers of different skill-levels.

3.4 Related Work

There has been previous work that studied the problem of sequential attack-sequences using exploit preconditions and postconditions. However, the approach in this chapter offers several advantages. First, my approach differs from [14], [52] in that it returns a constructive result by providing actual attack-sequences. Second, I have shown that queries can be answered efficiently, which is a favorable advantage over more computationally-intensive approaches [14], [50]–[53]. Third, my approach does not require generating explicit attack-graphs or the enumeration of all-possible attack scenarios, which again makes it far less computationally-intensive compared to approaches that do [50]–[53]. While there are approaches [59], [64] that use scalable attack-graph generation techniques [61], my approach still offers advantages. In [64] impact-assessments are done on a per-scenario basis, and a comprehensive

assessment would require enumerating over all-possible attack scenarios. Whereas in [59], identifying possible attack-sequences requires a breadth-first traversal of the attack-graph, which in the case of large, or poorly-secured networks, can be memory intensive; in comparison, I use A^* search with an efficient heuristic, and also leverage fixed-point operator results to prune the search tree, making my approach less resource-intensive. Finally, in contrast to domain-specific approaches that focus on specific situations, such as attacks on smart grids [14] or web applications [53], my approach is general and can be applied to multiple situations, including networked systems.

3.5 Conclusion

In this chapter, I developed a framework for modeling sequential-cyberattacks based on vulnerability dependencies. I showed that the attacker's obtained set of capabilities corresponds precisely with the result of a fixed-point computation, and that an actual attack-strategy can be computed efficiently via an A*-based approach, using heuristics that I have developed. Results presented in this chapter demonstrate that my model and its algorithms are viable in practice, and that solutions can be easily interpreted according to an intuitive, real-world understanding of attacker behavior. Possible avenues of future research include augmenting the selection of vulnerabilities by considering external sources of information, such as the darkweb, and investigating methods to automatically generate capability-sets from CVE descriptions and other sources of information, such as Metasploit modules.

Chapter 4

DEFENDING AGAINST SEQUENTIAL CYBERATTACKS

In the previous chapter, I introduced a formalism that allows modeling adversarial-actions against a system with possibly-interdependent vulnerabilities using associated exploits. I also addressed the problem of determining the optimal/most-dangerous strategy for a cyber-adversary with respect to this model. In this chapter I address the problem of determining the optimal/most-effective strategy that the defender can use to block the attacker from gaining certain capabilities, and find it to be an NP-Complete problem. To address this complexity, I once again utilize a provably-correct A*-based approach and develop an admissible heuristic. I provide an implementation and show through a suite of experiments using actual vulnerability data that this method performs well in practice for identifying defensive courses of action in this domain.

4.1 Defender Problem

The defender problem is fundamentally different from the attacker problem due to one significant observation: while the attacker is satisfied if they can gain a certain capability in at least one way, the defender must be able to identify and block all-possible ways in which that particular capability can be gained. That is, the attacker can gain a capability if they are able to exploit at least one vulnerability providing that capability. However, the defender must block all vulnerabilities whose exploits either directly or indirectly (i.e., through additionally exposed vulnerabilities) provide

access to that capability. Hence the defender must be able to identify the number of ways in which a particular capability can be gained. To model this, I define a function that, given a set of exploits used by the attacker and a set of vulnerabilities patched by the defender, returns a multiset of capabilities gained by the attacker. I define the function $patch : 2^E \times 2^V \to \langle 2^C, m \rangle$ as follows:

Definition 4.1. Given $E' \subseteq E$ and $V' \subseteq V$ I define the function $patch : 2^E \times 2^V \to \langle 2^C, m \rangle$ (where $m : C \to \mathbb{N}$) as:

$$patch(E', V') = \biguplus \{C^g \mid (C^r, C^g, v) \in E' \setminus E'_{V'} \wedge C^r \subseteq C^*_{E' \setminus E'_{V'}, C_0}\}$$

where:

$$E'_{V'} = \{(C^r, C^g, v) \in E' \mid v \in V'\}$$

For ease of notation, I also provide the following definition to represent the multiset of capabilities gained by the attacker, after the defender has patched a certain set of vulnerabilities:

Definition 4.2. Given $E' \subseteq E$ and $V' \subseteq V$ of patched vulnerabilities, the multiset of capabilities gained by the attacker after patching is represented by $C^p_{E',V'}$ where $C^p_{E',V'} \equiv patch(E', V')$.

Note that the number of occurrences of a particular capability in $patch(E', V')$ represents the number of remaining, applicable exploits that provide the attacker with that capability.

The defender's goal is to patch some set of vulnerabilities in order to prevent the attacker from gaining a certain set of capabilities. Assume that the defender has patched a set of vulnerabilities $V' \subseteq V$, blocking all capabilities in the set $C_b \subseteq C$. This implies that $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} = \varnothing$. Intuitively, I can also see that the corresponding

multiset of capabilities gained by the attacker will not contain any capabilities from C_b either. Hence I can state the following proposition:

Proposition 4.1. Iff $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} = \varnothing$ then $C_b \cap C^p_{E',V'} = \varnothing$.

Proof.

Claim 1. $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} = \varnothing$ implies $C_b \cap C^p_{E',V'} = \varnothing$.

Assume by way of contradiction that there exists a C_b such that $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} = \varnothing$ and $C_b \cap C^p_{E',V'} \neq \varnothing$. By definition 4.1, this implies that there exists some $(C^r, C^g, v) \in E' \setminus E'_{V'}$ where $C^r \subseteq C^*_{E' \setminus E'_{V'}, C_0}$ and $C^g \cap C_b \neq \varnothing$. This further implies that there must be some $j \in \mathbb{N}$ such that $C^r \subseteq \mathbf{T}_{E' \setminus E'_{V'}} \uparrow_j (C_0)$. Then by items (i) and (ii) from definition 3.4, $C^g \subseteq \mathbf{T}_{E' \setminus E'_{V'}} \uparrow_{j+1} (C_0)$ and so $C^g \subseteq C^*_{E' \setminus E'_{V'}, C_0}$ necessarily. But $C^g \cap C_b \neq \varnothing$ which implies that $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} \neq \varnothing$. Hence, a contradiction.

Claim 2. $C_b \cap C^p_{E',V'} = \varnothing$ implies $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} = \varnothing$.

Assume by way of contradiction that there exists a C_b such that $C_b \cap C^p_{E',V'} = \varnothing$ and $C_b \cap C^*_{E' \setminus E'_{V'}, C_0} \neq \varnothing$. This implies that there exists some $(C^r, C^g, v) \in E' \setminus E'_{V'}$ with $C^g \cap C_b \neq \varnothing$. Additionally, there must be some $j \in \mathbb{N}$ such that $C^r \subseteq \mathbf{T}_{E' \setminus E'_{V'}} \uparrow_{j-1} (C_0)$ as that is the only way $C^g \subseteq C^*_{E' \setminus E'_{V'}, C_0}$ (by items (i) and (ii) from definition 3.4). This further implies that $C^r \subseteq C^*_{E' \setminus E'_{V'}, C_0}$ and hence $C^g \subseteq C^p_{E',V'}$ (definition 4.1). But $C^g \cap C_b \neq \varnothing$ which implies that $C_b \cap C^p_{E',V'} \neq \varnothing$. Hence, a contradiction.

□

4.1.1 Preferred Defense Strategy

The defender aims to patch a set of corresponding vulnerabilities on the system to deny the attacker a certain set of capabilities, and must do so within a budget. Hence, I need a way to define the cost associated with patching a vulnerability:

Definition 4.3. Given the set of vulnerabilities V, I define a cost function $cost : V \rightarrow \mathbb{R}^+$ that associates a real-valued cost with each vulnerability.

For simplicity, I will use a single cost-function throughout this chapter. However, all of the results can be extended for separate cost-functions for the defender, as long as the cost is computable in polynomial time. Also, throughout this chapter, I will use a unit cost-function, where for each $v \in V$, $cost(v) = 1$ (unless otherwise specified). Having defined the *cost* function, I can now formally define the defender's preferred strategy:

Definition 4.4. Given the defender's budget $b \in \mathbb{R}^+$, set of exploits $E' \subseteq E$ for vulnerabilities on the system, and the set of capabilities $C_b \subseteq C$ that the defender wants to deny the attacker, the **preferred defense-strategy** is to patch the set of vulnerabilities $V' \subseteq V$ subject to the following conditions:

- $C_b \cap C^p_{E',V'} = \varnothing$
- $\sum_{v \in V'} cost(v) \leq c$

In the optimization variant of the preferred defense-strategy problem, the quantity $\sum_{v \in V'} cost(v)$ is minimized, and the associated strategy is called the ***optimal strategy***.

I examine the computational complexity of solving the preferred defense-strategy problem. I will show that the finding such a strategy is NP-complete.

Theorem 4.1. Finding a preferred defense-strategy is NP-complete with a cost function that is computable in polynomial time.

Proof.

Claim 1. Finding a preferred defense-strategy is in-NP.

Finding the preferred defense-strategy is clearly in NP. Given the set of capabilities C_b that the defender wishes to block, budget c, set of exploits on the system E, set of patched vulnerabilities V' and the corresponding set of exploits E_v, and given that both *cost* and *expose* are computable in polynomial time, I can verify that $|V'| \leq c$ and check that $\mathbf{T}^*_{E \setminus E_v}(\varnothing) \cap C_b = \varnothing$ in polynomial time to verify that V' is the preferred defense-strategy.

Claim 2. Finding a preferred defense-strategy is NP-hard.

I can show that finding the preferred defense-strategy is NP-hard by constructing an instance of the problem from an instance of the Set Cover problem. The Set Cover problem is stated as follows: given a set of sets $U = \{U_1, U_2, \ldots, U_n\}$ and a number k, find a $U' \subseteq U$ such that $|U'| \leq k$ and $\bigcup_{U_i \in U'} U_i = \bigcup_{U_i \in U} U_i$.

Construction: I will construct an instance of the preferred defense-strategy problem from an arbitrary instance of the Set Cover problem. My construction creates an instance of the problem where an individual capability is gained by exploiting a single set of vulnerabilities. The construction is performed using the algorithm SetCover2PDS, which is defined as follows:

- Build the global set of capabilities $C = \{c_i \mid U_i \in U\} \cup \bigcup_{U_i \in U} U_i$.
- Build the set of capabilities to be blocked $C_b = \bigcup_{U_i \in U} U_i$.
- Build the set of vulnerabilities $V = \{v_i \mid U_i \in U\} \cup \{v'_j \mid j \in \bigcup_{U_i \in U} U_i\}$.

- Build the set of exploits $E = \{\langle \varnothing, \{c_i\}, v_i \rangle \mid U_i \in U\} \cup \{\langle C_{1j}, \{j\}, v'_j \rangle \mid j \in \bigcup_{U_i \in U} U_i\}$ where $C_{1j} = \{c_k \mid U_k \in U \text{ where } j \in U_k\}$.
- Set the attacker's initial set of capabilities $C_0 = \varnothing$.
- I define the cost-function for patching any vulnerability $v \in V$ as $cost(v) = 1$.
- I define *expose* for any set of capabilities $X \subseteq C$ as $expose(X) = V$.
- Set the defender's budget $c = k$.

Claim 2.1. SetCover2PDS runs in polynomial time.

The algorithm SetCover2PDS clearly runs in polynomial time. Building C, C_b, and V only requires me to iterate over all sets in U and elements $\bigcup_{U_i \in U} U_i$. Building E requires a runtime of $O(|\bigcup_{U_i \in U} U_i| \times |U|)$ as I iterate over every set in U for every element in $\bigcup_{U_i \in U} U_i$.

Claim 2.2. If there is a solution to the Set Cover problem, then there exists a preferred defense-strategy.

Suppose by way of contradiction, there is a set cover U' of U such that $|U'| \leq k$, which does not have a corresponding preferred defense-strategy. By my construction, U' corresponds to the set of vulnerabilities $V' = \{v_i \mid U_i \in U'\}$. Since I set $c = k$, $|V'| = |U'|$ and therefore $\sum_{v \in V'} cost(v) \leq c$. An attacker can gain a capability i by targetting vulnerability v'_i using exploit $e_i = \langle C_{1i}, \{i\}, v'_i \rangle$. However in order to do so, they need to have gained every $c_j \in C_{1i}$ by having successfully exploited each corresponding vulnerability v_j. Note that from my construction, $C_{1i} = \{c_j \mid U_j \in U$ where $i \in U_j\}$. Consequently, by patching a vulnerability v_j, the defender effectively prevents the attacker from gaining access to every capability $i \in U_j$, as they are unable to use the corresponding exploit e_i. I know that $E_{V'} = \{\langle \varnothing, \{c_j\}, v_j \rangle \mid v_j \in V'\}$ corresponds to the set of patched vulnerabilities V'. As V' derives from U', each exploit in $E_{V'}$ provides a capability corresponding to a set in U'; let $C_{V'}$ be the set of

all such capabilities from $E_{V'}$ and note that it is essentially equivalent to $\{c_j \mid U_j \in U'\}$. Observe that the complete set of capabilities denied by patching V' can be expressed as $\bigcup_{c_j \in C_{V'}} \{i \mid i \in U_j\}$. As $C_{V'}$ is analogous to the cover U', I have:

$$\begin{aligned}
\bigcup_{c_j \in C_{V'}} \{i \mid i \in U_j\} &= \bigcup_{U_j \in U'} \{i \mid i \in U_j\} \\
&= \bigcup_{U_j \in U'} U_j \\
&= \bigcup_{U_j \in U} U_j \\
&= C_b
\end{aligned}$$

Thus $C_b \cap \mathbf{T}^*_{E \setminus E_{V'}}(\varnothing) = \varnothing$ and by proposition 4.1, $C_b \cap C^p_{E,V'} = \varnothing$ as well. Hence V', which corresponds to a solution U' of the Set Cover problem, is the *preferred defense-strategy*. Hence, a contradiction.

Claim 2.3. If there is no solution to the Set Cover problem, then there does not exist a preferred defense-strategy

Suppose by way of contradiction, there is a preferred defense-strategy V' without a corresponding solution to the Set Cover problem. This means that there is a corresponding set of exploits $E_{V'} = \{\langle \varnothing, \{c_j\}, v_j \rangle \mid v_j \in V'\}$. From my construction, I know that each capability c_j corresponds to a $U_j \in U$; let $C_{V'}$ be all such capabilities from $E_{V'}$. I know that an attacker can gain a capability i by targetting vulnerability v'_i using exploit $e_i = \langle C_{1i}, \{i\}, v'_i \rangle$, only if they have gained every $c_j \in C_{1i}$ by having successfully exploited each corresponding vulnerability v_j. By my construction, $C_{1i} = \{c_j \mid U_j \in U \text{ where } i \in U_j\}$. Hence, patching vulnerability v_j prevents the attacker from gaining every capability $i \in U_j$ by using the corresponding exploit e_i. As V' is the preferred defense-strategy, I know that $C_b \cap C^p_{E,V'} = \varnothing$, and by proposition

4.1, $C_b \cap \mathbf{T}^*_{E \setminus E_v}(\varnothing) = \varnothing$ as well. This means that by patching vulnerabilities in V', the defender denies access to all capabilities in C_b. Hence, I can state that $\bigcup_{c_j \in C_{V'}} \{i \mid i \in U_j\} = C_b$. But based on my construction, $C_b = \bigcup_{U_j \in U} U_j$ and so $\bigcup_{c_j \in C_{V'}} \{i \mid i \in U_j\} = \bigcup_{U_j \in U} U_j$, which implies that it is a cover. Likewise, I also know that $|V'| \leq k$. Therefore $C_{V'}$, which is derived from V' via $E_{V'}$ satisfies the criteria to be a cover, which implies that a solution U' to the Set Cover problem must exist. Hence, a contradiction.

Proof of theorem: Follows directly from claims 1 and 2. □

4.2 Algorithms

In this section I examine several algorithms to solve the preferred defense-strategy problem. My first, baseline approach is PDS-DFS (see Algorithm 4.1) that performs a depth-first search across the strategy-space. This algorithm obviously has shortcomings; the first being that it is not guaranteed to find an optimal solution. Additionally, the time-complexity is factorial. Since the maximum number of vulnerabilities is $|V|$, and the branching-factor at each level is one less than the previous, time complexity is $O(|V|!)$. Despite these shortcomings, PDS-DFS can be used as an anytime algorithm and is guaranteed to return a solution, though not necessarily optimal. However, as the algorithm continues to search, it can return better solutions. To address issues with depth-first search, breadth-first is an alternative. However the extremely-large space complexity, $O(|V|!)$ in this case, makes this infeasible. I provide two improvements over standard DFS that help me search for defender strategies in a more-efficient manner:

Algorithm 4.1 DFS algorithm to find preferred defense-strategy

```
procedure PDS-DFS(E, V, C_b, b):
    function PATH(node):
        path as SET
        while node.PARENT ≠ ∅ do:
            ADD(path, node.VULNERABILITY)
            node ← node.PARENT
        return path
    function SOLUTION(node):
        V' ← PATH(node)
        return ⟨V', C_b ∩ C^p_{E,V'}⟩
    function ROOT:
        root as NODE
        root.PARENT ← ∅
        root.VULNERABILITY ← ∅
        root.PATHCOST ← 0
        return root
    function MAKENODE(parent, v):
        node as NODE
        node.PARENT ← parent
        node.VULNERABILITY ← v
        node.PATHCOST ← parent.PATHCOST + cost(v)
        return node
    nodes as LIFO-QUEUE
    ENQUEUE(nodes, ROOT)
    loop do:
        if EMPTY(nodes) then:
            return failure
        node ← DEQUEUE(nodes)
        ⟨V', C'⟩ ← SOLUTION(node)
        if node.PATHCOST ≤ b and C' = ∅ then:
            return V'
        for each v in V \ V':
            ENQUEUE(nodes, MAKENODE(node, v))
```

1. I perform an initial pruning of the list of vulnerabilities to include only those that can lead to capabilities I want to block.
2. I correctly prune the available vulnerabilities at each step.
3. I employ the use of an admissible heuristic-function by adopting A* search.

All of these improvements allow me to maintain the correctness of the approach.

4.2.1 Pruning Vulnerabilities

PDS-DFS has factorial-time complexity due to the fact that it considers vulnerabilities that need not or cannot be part of the solution. To address this problem,

Algorithm 4.2 DFS with pruning (common functions omitted)

```
procedure PDS-DFS-Pruned(E, V, C_b, b):
    function PrePrune:
        E_p ← ∅
        C' ← C_b
        do
            E' ← {e ∈ E \ E_p | e.C^g ∩ C' ≠ ∅}
            E_p ← E_p ∪ E'
            C' ← C' ∪ ⋃{e.C^r | e ∈ E'}
        while E' ≠ ∅
        return (E_p, {e.v | e ∈ E_p})
    function Prune(parent, v):
        V' ← Path(parent)
        if parent.PathCost + cost(v) > c then:
            return true
        else if {e ∈ E_{v} | e.C^r ⊆ C^p_{E,V'}} = ∅ then:
            return true
        return false
    function Expand(node):
        children as Set
        for each v in V \ Path(node):
            if not Prune(node, v) then:
                Add(children, MakeNode(node, v))
        return children
    E, V ← PrePrune()
    nodes as LIFO-Queue
    Enqueue(nodes, Root)
    loop do:
        if Empty(nodes) then:
            return failure
        node ← Dequeue(nodes)
        ⟨V', C'⟩ ← Solution(node)
        if node.PathCost ≤ b and C' = ∅ then:
            return V'
        for each child in Expand(node):
            Enqueue(nodes, child)
```

I can use PDS-DFS-Pruned (see Algorithm 4.2) which prunes the search-tree by discarding such vulnerabilities. I perform an initial "pre-pruning" step by working backwards from C_b to identify vulnerabilities that can lead to those capabilities. This prunes those vulnerabilities that do not provide any capabilities in C_b, or any transitive capabilities that can lead to capabilities in C_b.

When searching, each node in the search tree (other than the root node) represents a particular vulnerability, and a path to a node represents a set of vulnerabilities. When expanding a node in PDS-DFS to find the set of available vulnerabilities, I considered every vulnerability other than those in the path to the expanded node. In

PDS-DFS-Pruned, I additionally determine the viability of an available vulnerability by ensuring that the following conditions hold:

1. The cost of patching this vulnerability does not cause the defender to exceed their budget.
2. This vulnerability has at least one exploit whose required capabilities have not been blocked by previously-patched vulnerabilities in V'.

Only if these conditions hold, will PDS-DFS-Pruned include the vulnerability; otherwise it will ignore it. Note that I do not prune a vulnerability if patching it did not block capabilities in C_b. This is because such a vulnerability provides one or more transitive capabilities that eventually lead to one or more capabilities in C_b, as the pre-pruning step has already filtered out all vulnerabilities that do not (directly or indirectly) provide capabilities in C_b.

The correctness of this pruning technique follows directly from my original model – hence I state the following proposition:

Proposition 4.2. PDS-DFS-Pruned correctly prunes non-viable vulnerabilities and finds a correct solution if one exists.

Proof. I prove the statement of the theorem by first stating the precondition, loop invariant, and postcondition, and showing that these hold during all iterations.

Precondition: *nodes* is a LIFO queue of nodes that contains a root node of a search-tree that may or may not contain correct solutions.

Loop invariant: Aside from the root node, all nodes in *nodes* are either roots of viable subtrees or goal-state nodes.

Postcondition: If search-tree does not contain any correct solutions, the algorithm reports a failure; otherwise it reports the corresponding solution.

I will now show that these conditions hold during all iterations:

Initialization: At the first iteration, *nodes* initially contains only the root node. Hence, the invariant holds.

Maintenance: At iteration n, I assume that *nodes* contains roots of viable subtrees or goal-state nodes. Note that at iteration $n+1$, I can pick a node *node* out of *nodes* if it is not empty. While the algorithm terminates if *node* is a goal-state node or if *nodes* was empty, the invariant still holds because *nodes* is not modified in either case. If *node* is not a goal-state node, I make a call to EXPAND where I create and return corresponding-nodes for only those vulnerabilities where the call to PRUNE returned *false*. I know that PRUNE disregards those vulnerabilities that cannot be part of a solution, because it not only excludes vulnerabilities that would cause me to exceed the budget, but also excludes those vulnerabilities where patching provides no additional benefit; specifically PRUNE will also return *true* in the for any vulnerability whose exploits cannot be used due to requiring capabilities that are already blocked:

Suppose by way of contradiction, there is a vulnerability v with a single exploit requiring one or more currently-blocked capabilities, which was not pruned. I know that v will not be pruned if there exists at least one associated-exploit that does not require any of the currently blocked capabilities; that is, there must be at least one exploit such that $C^r \cap (C_b \setminus C^p_{E,V'}) = \varnothing$. However, based on my assumption, v has a single associated exploit where $C^r \cap (C_b \setminus C^p_{E,V'}) \neq \varnothing$. Hence, a contradiction.

Therefore, each corresponding-node that I add to *nodes* must either represent a viable subtree or a goal-state node. Hence the invariant holds at iteration $n+1$ as well.

Termination: Note that the algorithm terminates if *nodes* is empty or if the head node satisfies the goal test. In the former case, it implies that the search tree did not contain any correct solutions, because otherwise *nodes* would have had a goal-state

node at the head during some iteration. In the latter case it implies that the search-tree has at least one correct-solution, because the head-node was a goal-state node; in either case both the loop invariant and the postcondition hold.

Since the precondition, loop invariant, and postcondition hold during all iterations, the algorithm PDS-DFS-PRUNED not only prunes non-viable subtrees correctly, but will return a correct solution if one exists. □

4.2.2 A* Based Approach

PDS-DFS-PRUNED helps address time-complexity by pruning non-viable vulnerabilities. However, it still does not guarantee an optimal solution. One way to address this issue is by using an A* search across the strategy space. With an admissible heuristic-function, the tree-search variant of A* is both complete and optimal.

The defender wants to find the most-inexpensive set of vulnerabilities to patch that block all of the desired capabilities. Therefore, a sensible heuristic would be to patch a vulnerability that is both inexpensive, and blocks the capabilities I want blocked. A* evaluates nodes by combining $g(v)$, the cost to reach the node, and $h(v)$, the lower bound of the cost to get from the node to the goal. Deriving the estimated cost to the goal first requires defining some items related to the current state of the defender:

Definition 4.5. Given a set of vulnerabilities V' and complete set of exploits E, I define the corresponding set of blocked exploits $E_{V'}$ as:

$$E_{V'} = \{(C^r, C^g, v') \mid (C^r, C^g, v') \in E \text{ and } v' \in V\}$$

Definition 4.6. Given the complete set of vulnerabilities V, the defender's current set of vulnerabilities V', and the vulnerability under consideration v, I define the

remaining set of vulnerabilities V_{rem} as:

$$V_{rem} = \{v' \mid v' \in V \setminus V' \setminus \{v\} \wedge \{(C^r, C^g, v') \in E_{\{v'\}} \mid C^r \subseteq C^p_{E,V'}\} \neq \varnothing\}$$

Definition 4.7. Given the set of capabilities to block C_b, the complete set of exploits E, the defender's current set of vulnerabilities V' and the vulnerability under consideration v, I define the remaining set of un-blocked capabilities C_{rem} as:

$$C_{rem} = C_b \cap C^p_{E,V' \cup \{v\}}$$

As I do not want to overestimate the cost to the goal, an obvious approach is to use the cheapest vulnerability. While this heuristic is certainly admissible, it vastly underestimates the cost to the goal, as it does not take into account the effect of patching a vulnerability. Hence a more useful estimate would be to use the cost of what is *effectively* the cheapest vulnerability:

Definition 4.8. Given the vulnerability under consideration v, the remaining set of vulnerabilities V_{rem}, and the remaining set of capabilities to be blocked C_{rem} I define the cost per remaining-capability of a $v' \in V_{rem}$, $cost_r : V \to \mathbb{R}^+$ as:

$$cost_r(v') = \frac{cost(v')}{\sum_{\{c' \in C_{v'} \mid c' \in C_{rem}\}} m(c')}$$

where:

$$C_{v'} = C^p_{E,V' \cup \{v\}} \setminus C^p_{E,V' \cup \{v,v'\}}$$

This cost is the ratio between the cost of the vulnerability and the number of remaining capabilities blocked by patching this vulnerability. Note that this ratio also takes into account the number of ways a particular blocked-capability could have been gained. For example, suppose I have two vulnerabilities v_1 and v_2 with cost 1 and 2 respectively. Assume that patching v_1 blocks a single exploit that provides capability

c, whereas patching v_2 blocks four exploits that provide c. This means that v_1 has an effective cost of 1, whereas v_2 has an effective cost of 0.5. I will now define a heuristic that estimates the cost to the goal, given a node representing a vulnerability:

Definition 4.9. Given the node v that represents an applicable vulnerability under consideration, the remaining set of vulnerabilities V_{rem} and the remaining set of capabilities to be blocked C_{rem} I define the estimated cost to the goal $h : V \to \mathbb{R}^+$ as:

$$h(v) = \sum_{c \in C_{rem}} \min_{\{v' \in V_{rem} | c \in C_{v'}\}} cost_r(v')$$

where:

$$C_{v'} = C^p_{E,V' \cup \{v\}} \setminus C^p_{E,V' \cup \{v,v'\}}$$

When estimating cost to the goal from v, I find, for each remaining capability, the lowest effective-cost out of all vulnerabilities that can be exploited to provide that capability, and then summing those costs.

Theorem 4.2. $h(v)$ is admissible.

Proof. I prove the statement of the theorem by showing that $h(v)$ is a lower bound of the true cost from node v to the goal.

By way of contradiction, consider an optimal path to the goal from node v with true cost $h^*(v)$ such that $h(v) > h^*(v)$. Let V^* be the corresponding set of vulnerabilities. Then:

$$\sum_{c \in C_{rem}} \min_{\{v' \in V_{rem} | c \in C_{v'}\}} cost_r(v') > \sum_{v' \in V^*} cost(v')$$

Since V^* represents the optimal path, the following is also true:

$$\sum_{c \in C_{rem}} \min_{\{v' \in V_{rem} | c \in C_{v'}\}} cost_r(v') > \sum_{c \in C_{rem}} \min_{v' \in V^*} cost_r(v')$$

Algorithm 4.3 A* algorithm that solves the preferred-defense strategy problem.

```
procedure PDS-A*(E, V', C_b, b):
    function RemainingCapabilities(node):
        V' ← Path(node.Parent)
        v ← node.Vulnerability
        return C_b ∩ C^p_{E,V'∪{v}}
    function RemainingVulnerabilities(node):
        V' ← Path(node.Parent)
        V_rem as Set
        for each v in V \ V' \ {node.Vulnerability}:
            if not Prune(node, v) then:
                Add(V_rem, v)
        return V_rem
    function h(node):
        C_rem ← RemainingCapabilities(node)
        V_rem ← RemainingVulnerabilities(node)
        V' ← Path(node.Parent)
        v ← node.Vulnerability
        h ← 0
        for each c' in C_rem:
            h_min ← ∞
            for each v' in V_rem:
                C_v' ← C^p_{E,V'∪{v}} \ C^p_{E,V'∪{v,v'}}
                count_v' ← Σ_{c∈(C_rem∩C_v')} Count(C_v', c)
                if c' ∈ C_v' then:
                    h_v' ← cost(v') ÷ count_v'
                    if h_v' < h_min then:
                        h_min ← h_v'
            h ← h + h_min
        return h
    function EstimatedCost(node):
        return node.PathCost + h(node)
    nodes as Priority-Queue ordered by EstimatedCost
    Enqueue(nodes, Root)
    loop do:
        if Empty(nodes) then:
            return failure
        node ← Dequeue(nodes)
        ⟨V', C'⟩ ← Solution(node)
        if C' = ∅ then:
            return V'
        for each child in Expand(node):
            if not Exists(nodes, child) :
                Enqueue(nodes, child)
            else:
                existing ← Find(nodes, child)
                f_existing ← EstimatedCost(existing)
                f_child ← EstimatedCost(child)
                if f_child < f_existing then:
                    Replace(nodes, existing, child)
```

This implies that there exists a c such that:

$$\min_{\{v' \in V_{rem} | c \in C_{v'}\}} cost_r(v') > \min_{\{v' \in V^* | c \in C_{v'}\}} cost_r(v')$$

Let $v_{min,c}$ and $v^*_{min,c}$ be the corresponding vulnerabilities with the lowest effective-

cost. This means that $cost_r(v_{min,c}) > cost_r(v^*_{min,c})$. However, for $v_{min,c}$ to have been used in calculating $h(e)$ instead of $v^*_{min,c}$, it must mean that $cost_r(v_{min,c}) < cost_r(v^*_{min,c})$. This is a contradiction, which means that $h^*(v)$ must not be the true cost to the goal. Hence $h(v)$ is admissible.

□

PDS-A* (see Algorithm 4.3) is an implementation of A* to solve the preferred-defense strategy problem. Given that the heuristic is admissible, PDS-A* is guaranteed to produce an optimal solution.

4.3 Experiments and Results

In this section I examine the results of experiments that I conducted to show the viability of My model and previously-described algorithms. My experimental suite involves two kinds of experiments: those that evaluate performance and those that illustrate how the model and its algorithms behave in real-world scenarios; in particular, the results show that I not only able to solve the preferred defense-strategy problem in a timely manner, but that the solutions conform to real-world expectations of both attacker and defender behavior. I also show that the identified solutions are useful in practice, and can be used for risk assessment and to provide guidance for security policies. The algorithms themselves, as well as the experiments, were implemented in Java 8 and executed on a machine with an Intel Core i7-4770K processor and 24GB RAM running Linux Mint 18.2.

All my experiments use real-world vulnerability data gathered from the National Institute of Standards and Technology's National Vulnerability Database (NIST NVD). For the first two experiments, I defined 33 exploits based on 16 vulnerabilities

associated with the Windows 10 operating system. The vulnerabilities used involve a diverse spectrum of malicious capabilities, such as denial of service, privilege escalation, remote-code execution, as well as those that take advantage of physical proximity. In addition, these exploits also feature a good distribution of attacker types.

In order to see how the attacker's initial capabilities affect the solution, I also defined four types of attackers. These are, in order from least to most-threatening:

1. A **remote attacker** with remote access to the machine.
2. An **authenticated remote-attacker** with remote access and valid credentials for the machine.
3. A **physically-proximate attacker**; i.e., one that has physical and network access to the machine.
4. A **local attacker**; i.e., one with physical access and valid credentials for local use of the machine.

The corresponding sets of initial-capabilities for these attackers were set as follows:

1. $C_0 = \{$`remote`$\}$
2. $C_0 = \{$`remote`, `auth`$\}$
3. $C_0 = \{$`remote`, `phys-access`, `proximate`$\}$
4. $C_0 = \{$`remote`, `local`, `auth`, `proximate`, `phys-access`$\}$

Finally, I defined the set of capabilities that the defender wishes to block as $C_b = \{$`code-exec`, `escalate-privilege`, `denial-of-service`, `install-crafted-boot-manager`$\}$.

My first set of experiments were designed to evaluate the performance of my algorithms. For each algorithm, I also investigated the effects of enabling and disabling memoization; I did this for both a unit cost-function and a complexity-based cost-

function, which is based on attack and access-complexity costs. Here, I set the cost of patching a vulnerability to the sum of the attack and access complexity of the associated CVE; these metrics are extracted from the Common Vulnerability Scoring System (CVSS) base-score vector reported for each CVE in the NVD. Both types of complexities are graded as High, Medium, or Low, which I associated with the integer values 3, 2, and 1. Hence, a CVE with a high attack-complexity and a low access-complexity will have a cost of 4. For CVEs that do not have an access-complexity metric, I only use the attack complexity.

To see how PDS-DFS and PDS-DFS-Pruned compare against PDS-A*, I ran the latter algorithm with an unlimited defender-budget to identify the cost of the optimal solution. This cost was then set as the defender budget when running PDS-DFS and PDS-DFS-Pruned. I then ran these algorithms (with and without memoization) for each attacker type; the results can be seen in figures 6 and 7. It is evident that the performance of PDS-A* is much better than either PDS-DFS or PDS-DFS-Pruned. This, in spite of the fact that PDS-DFS-Pruned appears to have a better runtime in the case of both remote-attacker types. Note that results for PDS-DFS and PDS-DFS-Pruned are missing in the case of physically-proximate and local attackers; I could not provide these results due to the fact that the algorithms took far too long to finish. This shows that beyond a certain solution-depth, the runtime advantage of PDS-DFS-Pruned disappears because pruning, while helpful, does not diminish the strategy space enough to impact performance in an appreciable way. In addition, note that PDS-DFS-Pruned in general is not able to provide an optimal solution, which clearly makes PDS-A* the better option. The results also confirm that memoization has a significant, positive effect on performance. In all cases, memoization drastically improves the runtime.

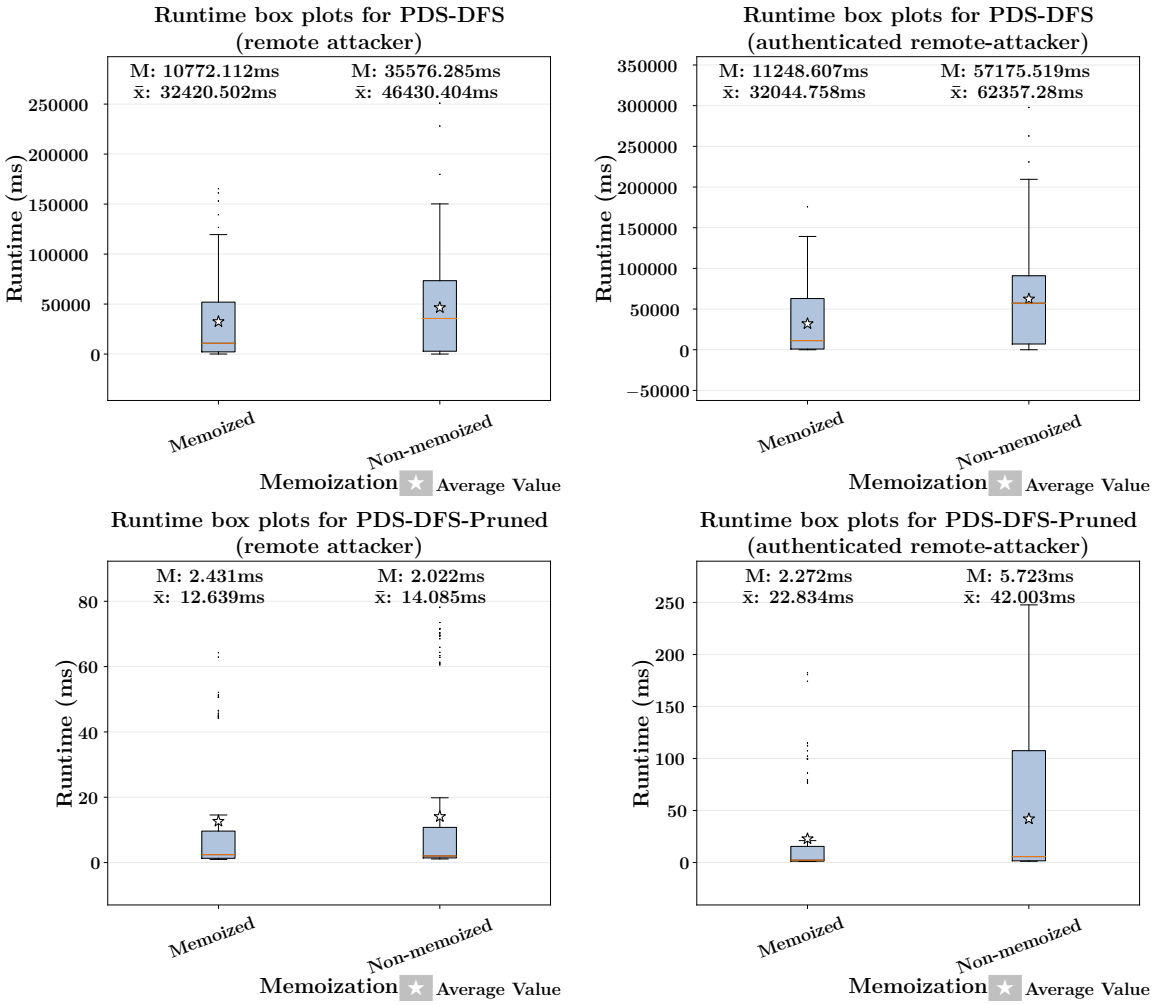

Figure 6. Runtime box plots for PDS-DFS and PDS-DFS-PRUNED against both authenticated and unauthenticated remote attackers, with and without memoization. Note that the large variance is due to non-deterministic vulnerability order when expanding nodes.

My second experiment was designed to see how different attacker-types affect the solution to the defender problem. My hypothesis is that the number of patched vulnerabilities is directly proportional to the threat level that the attacker represents; results of my experiment, seen in table 3 confirm this hypothesis. To block both types of remote-attackers, the defender need only patch 6 vulnerabilities with a total cost of 16, whereas in the case of the local attacker, the defender must patch a total of

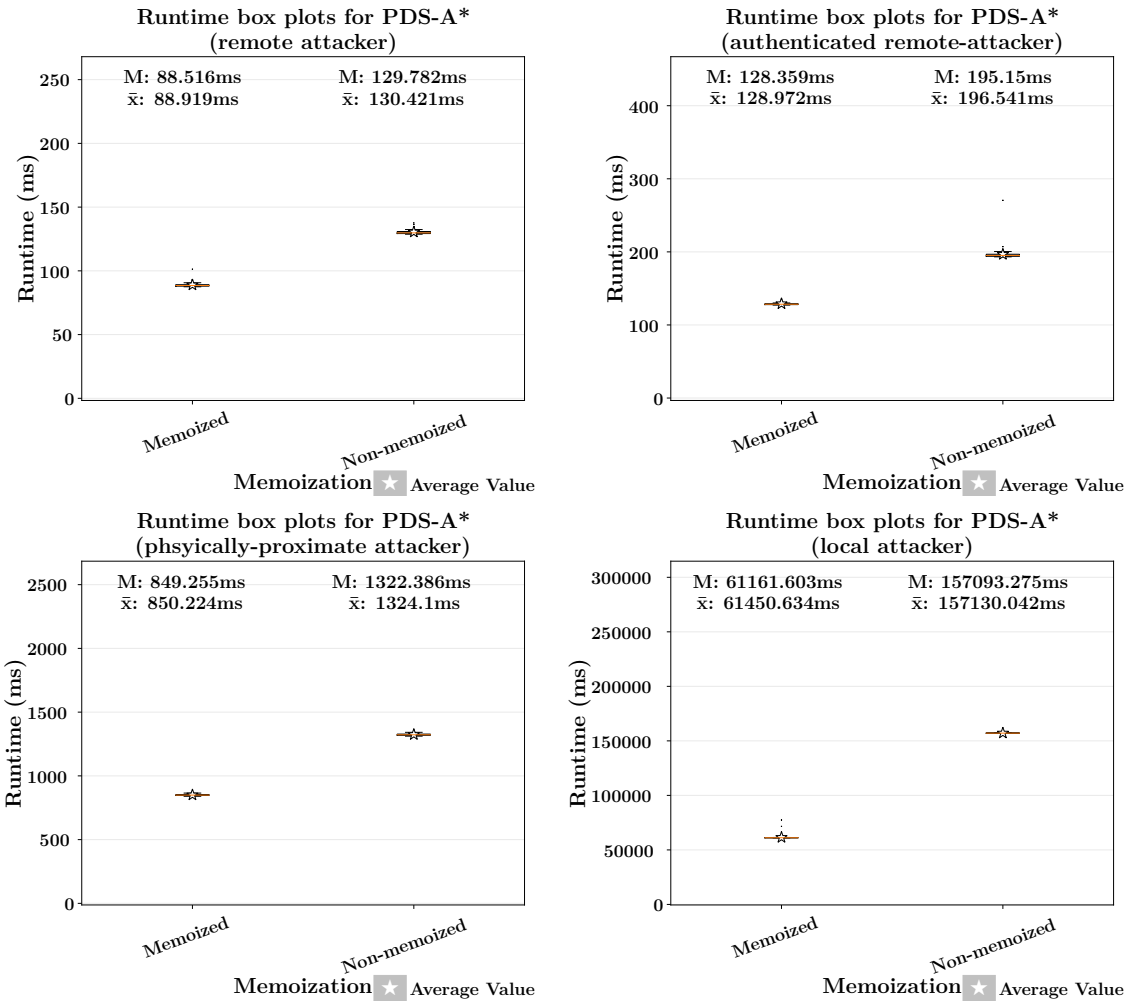

Figure 7. Box plot of runtimes for all PDS-A* algorithms (memoized and non-memoized) against different attacker types (1000 samples each).

13 vulnerabilities with a total cost of 36. In order to block the physically-proximate attacker, the defender must patch 9 vulnerabilities with a total cost of 26. This shows that in order to defend against more-threatening attackers, who possess a larger or more-significant set of initial capabilities, the defender should expect to patch a larger set of vulnerabilities. The result is significant, because it allows security professionals to tailor their responses based on the kinds of attacks that are expected. Note that current patching strategies involve an "all or nothing" approach due to

Attacker	Solution Cost	Solution Size
Remote	16	6
Remote authenticated	16	6
Physically proximate	26	9
Local	36	13

Table 3. Patched vulnerabilities, solution costs, and sizes for different attacker types (CVEs omitted for brevity).

the fact that vulnerability dependencies are not taken into account. By factoring in vulnerability dependencies and anticipated attacker-types, it would be possible to generate tailor-made patch-sets that protect defenders from specific kinds of attackers. This can also decrease the amount of time it takes for patch-sets to make it out to consumers, as they can be released on a staggered schedule, progressively targeting more and more threatening types of attackers.

My final experiment tries to see how the framework could be used to model attacks involving a network of machines. I modeled a simple network consisting of one Windows machine with public, remote access, and fire-walled OS X machine running an Oracle server, that is only accessible with proper credentials via the Windows machine. For this experiment, I defined a total of 10 exploits based on 4 vulnerabilities. The Windows machine contains a credentials-disclosure vulnerability that is exploitable by remote users (CVE-2016-3312), and the Meltdown and Spectre vulnerabilities (CVE-2017-5715, CVE-2017-5753, CVE-2017-5754), which require local access. The OS X machine running the Oracle server contains the Meltdown and Spectre vulnerabilities as well, and an Oracle denial-of-service and data-disclosure vulnerability exploitable by anyone with network access and/or credentials (CVE-2018-2590). I defined two types of attackers: a **remote attacker** with initial capabilities {`remote`} and a **local attacker** with initial capabilities {`win-creds`, `mac-creds`}. The set of capabilities

Attacker	Vulnerabilities Patched	
	Windows	OS X
Remote	CVE-2016-3312	None
Local	CVE-2017-5715 CVE-2017-5753 CVE-2017-5754	CVE-2018-2590 CVE-2017-5715 CVE-2017-5753 CVE-2017-5754

Table 4. Patched vulnerabilities for different attacker types.

that the defender wishes to block was defined as $C_b = \{\texttt{oracle-dos}, \texttt{oracle-data}\}$, which represents the defender's desire to block denial-of-service attacks against the Oracle server, and to also block unauthorized access to data.

The results of this experiment can be seen in table 4. A remote attacker can be prevented from compromising the Oracle server by patching a single vulnerability (CVE-2016-3312); this makes sense as that is the only way that the attacker can gain local credentials in order to exploit other vulnerabilities. However in order to stop a local attacker, the defender needs to not only patch the Meltdown and Spectre vulnerabilities on *both* machines, but needs to patch the Oracle vulnerability (CVE-2018-2590) as well. This result shows that the model can be applied to a network of machines, and provides solutions that are just as intuitive and useful as the single-machine scenario. This is especially seen when comparing the number of vulnerabilities and systems that need to be patched based on the attacker type; whereas the remote attacker can be stopped by patching a single vulnerability on the Windows machine, a local attacker can be stopped only by patching multiple vulnerabilities on both machines. The difference makes sense intuitively as one expects a local user with valid credentials to have greater access to system and network capabilities, compared to a remote user.

4.4 Related Work

In recent years, many contributions have been made in the field of cybersecurity, using attacker-defender models to inform defender-actions. In the field of network security, work has been done to address problems related to optimal network-security hardening [65] and resource-allocation for malicious packet-detection [66]. Work has also been done on modeling adversary-initiated cascading-failures in power-grid [24] and critical-infrastructure systems [25], to identify defender strategies to mitigate these failures. In these papers, the systems are modeled as a graph of nodes, with the adversary aiming to disrupt the network by initiation actions against specific nodes. When compared to my work, there are some differences. The first is that my approach can be used to model both a single system and a network of systems, and the second is that I model the actual mechanics of the adversary's attack, instead of merely treating it as a binary action (i.e., a node is either attacked or it is not). This allows us to identify a minimal set of vulnerabilities to patch (subject to cost constraints), to prevent the attacker from gaining certain capabilities.

The work presented in both [67] and [68] are close to mine in that their models take into account the sequence of actions required to exploit a system. However, model dynamics and approaches are significantly different. The work in [69] bears some similarity to mine, in that the attacker's aim is to exploit a specific vulnerability. However, the paper models attacks and vulnerabilities at a high level, and does not take into account specific capabilities associated with vulnerabilities; it can also only handle an attack against a single type of vulnerability at a time.

The application of game theory to security situations was first popularized by [26], where it was applied to the problem of airport security. Since then, it has

been applied to many other scenarios as well, including the placement of security checkpoints [27], assignment of air marshals to flights [28], and more pertinently, to cybersecurity [29]–[31]. While my work also identifies the defender's optimal strategy, I do not model attacker-defender interactions as a Bayesian game [26]–[28] and the details of my model are inherently different. My model is deterministic and focuses on the mechanics of the attacks themselves, using an iterative approach to identify the overall set of capabilities gained by the attacker, by taking into account their initial capabilities, the exploits used, and dependencies between vulnerabilities on the system. The work in [31] is slightly similar to mine, given that their model also considers how capabilities gained by an attacker after exploiting a vulnerability, can be used to further increase their chances of compromising the system. However, this model does not distinguish between individual capabilities, and thus cannot take the attacker's capabilities into account when determining their ability to compromise a system.

Approaches involving the use of exploit preconditions and postconditions to model sequential-attacks were previously introduced in [13], [14], where Petri nets [32] are used to model cyberattacks. While the approach can determine the overall set of capabilities gained by the attacker through the use of sequential exploits, it is necessary to construct a coverability graph [70], [71] to do so. In comparison, my model provides a simpler solution to the same problem through the use of a simple fixed-point operator that converges in polynomial time. I further note that my approach has another key advantage: it is able to identify the minimal set of vulnerabilities (subject to cost constraints) to be patched in order to prevent the attacker from gaining certain capabilities.

Regarding effective vulnerability-patching strategies, there are examples of similar work from [15]–[17]. However, the key difference is that the aforementioned models

are stochastic, and largely based on metrics such as vulnerability discovery-rate, attack rates, and aspects of the vulnerability life-cycle itself. Hence, these approaches are far more general and when identifying effective patching-strategies, do not take into account the specific capabilities that attackers wish to gain.

To the best of my knowledge, my work presents the first approach that models both the attacker's intent and the mechanics of an attack, and uses that information to identify an optimal defensive-strategy aimed at preventing access to specific capabilities. In my work, I model specific-actions that an attacker can take against a given system and the degree to which they can compromise that system, by taking into account the attacker's capabilities, the exploits used, and the dependencies between vulnerabilities on the system. The modeling of exploit application in an iterative manner also matches real-world attacker-behavior. The attacker's activities are also driven by unconventional sources of information (specifically darknets in this case) and do not necessarily depend on information directly related to defender systems. Furthermore, the recent rise of darknet markets specializing in zero-day exploits allows us to integrate information unavailable to previous work.

4.5 Conclusion

The results not only demonstrate that the model and associated algorithms are viable in practice, but that they can be easily interpreted according to an intuitive, real-world understanding of both attacker and defender behaviors. The results provide useful information for defenders of these systems, that can be used to assess overall system-vulnerability, its susceptibility to various types of attackers, and the maximum amount of damage that can be caused by those attackers.

Additionally, the results show that the model is able to identify effective and optimal patching-strategies that are also intuitive and make sense in a real-world context. The information provided by this model can be used for the development of intelligent agents that can learn defense and contingency strategies based on exploit and vulnerability information. It also allows for the development of agents that can perform risk analyses using this same information; this makes it easy for security professionals and administrators to immediately assess the vulnerability of systems in a quantifiable way, allowing them to make informed decisions regarding security policies, patching strategies, and other contingency actions.

Avenues for future research involve methods to automatically identify the capabilities that comprise vulnerability preconditions and postconditions, from the vulnerability's description or from security advisories. It would also prove useful to include additional data, such as exploit-costs gathered from exploit kits sold on darknet markets [72].

Chapter 5

AUTOMATIC IDENTIFICATION OF STATE-REPRESENTATIVE VARIABLES TO PROVIDE DEEP-STATE FEEDBACK

As vulnerability after vulnerability rocks the software bedrock underpinning our modern society, researchers developed ways to automatically identify software flaws so that they can be fixed before causing havoc. In recent years, fuzz testing, a dynamic-analysis technique that attempts to trigger bugs with pseudo-random input, demonstrated its effectiveness by uncovering a large number of vulnerabilities in diverse software. Hence, fuzzing received significant interest in both academia [23], [33]–[38] and industry [21], [39]–[41].

Modern fuzzing research has predominantly focused on maximizing *code coverage* — such as basic block coverage and branch/edge coverage — based on the notion that covering more code triggers more vulnerabilities. This is usually done by satisfying previously-failing program branch conditions, whether by solving them using symbolic techniques [37], [38], identifying specific parts of program input that should be stochastically mutated to increase the chance of satisfying conditions [33], [34], or modifying the conditions themselves [35]. As a result of these interventions, code coverage is increased and additional bugs are found.

Interestingly, increased code coverage is not always correlated with more discovered vulnerabilities: Shoshitaishvili et al. showed that fuzzers could use human-assisted input mutation to find more vulnerabilities—*while triggering strictly less code*—than standard (automated) code-coverage guided fuzzers [22]. An explanation is that not all code coverage is equal [73], and that coverage alone does not imply good exploration of

a program's *state space*. IJON expanded on this concept by using manual source-code annotations to extend traditional code coverage feedback with *state-representative variables* [23]. The resulting *state-coverage guided fuzzer* showed promise in—with human help—triggering program states that standard code-coverage guided fuzzers cannot reach.

A natural next step in exploring the impact of state coverage on analysis outcomes is, of course, *automatic* reasoning about state-representative variables. Recent work in this area has made progress toward detecting state-representative variables [47], [48], but either uses such variables purely to increase code coverage (rather than *state* coverage) [47] or is only applicable to specific types of state-representative variables [48].

In this chapter, I propose SANDPUPPY, the first technique to automate the process of identifying and using state-representative variables in fuzzing to increase state coverage, and thus detecting *state-dependent* bugs.

In a reductive sense, every variable used in a program represents some sort of state. However, this does not necessarily imply that variables are equally representative to explore program state-space. Annotating too many variables overwhelms the fuzzer, as it explores unproductive execution paths due to the generation of useless inputs [23]. Therefore, a core challenge for SANDPUPPY then, is to select a minimal subset of state-representing variables from the set of all variables used by a program.

To tackle this challenge, my solution is inspired by recent observations that a significant number of program states are encoded in a subset of *integer* variables in a program [23], [47], [48]. Building on this observation, SANDPUPPY identifies candidate integer-variables in a target program and then uses simple, yet novel heuristics, on dynamic runtime value-traces aggregated across multiple executions, to gain a coarse

understanding of their semantics. In particular, I am interested in variables encoding the following kinds of state:

- State identifying program-input elements, whose sequential processing drives complex, internal state changes.
- State that influences the number of iterations in sequential-processing or iterative logic.
- State related to program input-size or sizes of input elements within structured input.
- Combined state encoded by variables used in similar contexts.

For such variables, SandPuppy uses one of several annotation methods (depending on the identified variable semantics) to add these variables to a fuzzer's coverage feedback.

SandPuppy enables state-coverage guided fuzzing without human effort and without overwhelming the fuzzer with spurious feedback. My evaluation showcases improved state-coverage on both synthetic and real-world examples. For example, SandPuppy automatically identified 165 state-representative integer variables in libtpms (out of a total of 14,436), including the one manually annotated by the authors of IJON, and continues to identify valid Trusted Platform Module commands long after other fuzzers "stall out" (§5.8.1). Similarly, SandPuppy solved two levels of the game Super Mario Bros that were not solvable by IJON, despite human annotations in the latter system. Finally, in fuzzing real-world programs, I show that SandPuppy's state coverage leads to superior code coverage compared to state-of-the-art fuzzers on most of my dataset, allowing my technique to find more vulnerabilities, including vulnerabilities in code that can be triggered, but not triggered *(in)correctly*,

by state-unaware fuzzers. As a result, SandPuppy managed to identify 12 unknown vulnerabilities from 2 well-fuzzed open-source projects.

Contributions. In summary, my contributions are as follows:

- I design an approach for detecting state-representative variables in a program and for exposing associated state to guide fuzzing towards deep, complex states.
- I implement this approach into a research prototype called SandPuppy.
- I evaluate SandPuppy against custom targets representing specific fuzzing-challenges as well as real-world programs. I find that SandPuppy not only identifies interesting state-representative variables, but that the exposed state allows SandPuppy to explore a deeper and more diverse set of program states and solve certain challenges that purely coverage-based fuzzing techniques cannot solve. I also demonstrate that SandPuppy can identify existing and novel vulnerabilities and improve program coverage, when compared against existing tools such as AFL [21], AFL++ [74], LAFIntel [40], RedQueen [33], SGFuzz [48], and Ferry [47].

Ethics. I reported all identified vulnerabilities (11 in PcapPlusPlus and 1 in dmg2img). The PcapPlusPlus developers agreed with me and have fixed all vulnerabilities. The dmg2img developers did not respond to my disclosure, and I am continuing to contact them.

5.1 Background

Fuzzers aim to generate inputs exploring novel program states. Intuitively, the more states explored, the greater the chance that vulnerabilities are found. However,

lacking a way to directly measure *state*, most fuzzers tend use *code coverage* as a proxy. I describe here the coverage-feedback mechanism of AFL (the first ubiquitous coverage-guided fuzzer), how IJON augments AFL's feedback mechanism to better approximate program state, and the lack of automation in IJON that SANDPUPPY solves.

AFL Coverage Feedback Mechanism. AFL and its descendants (e.g., AFL++ [74]) instrument programs to expose coverage data. The fuzzer places instrumentation at the edges of the program's control flow graph (CFG) in order to track edge-to-edge transitions. The instrumentation identifies transitions using tuples of identifiers (id_s, id_d), where id_s is the source-block identifier and id_d is the destination-block identifier. These unique identifiers are generated during compile-time instrumentation.

AFL approximates the execution state of a program with a *bitmap*. Roughly speaking, when the program executes a transition, AFL hashes the tuple (id_s, id_d) to an index in the bitmap and increments the counter at that index.

AFL also uses a *global bitmap* to store the history of all bitmaps for the current fuzzing session. At the end of a test run, AFL updates the global bitmap with any new coverage information provided by the shared bitmap. Monitoring for changes in the global bitmaps allows AFL to determine whether the input used for the current test is "interesting" (i.e., if it triggered previously-unseen program transitions and resulted in new coverage).

Evolving from Code Coverage to State Coverage. Directly measuring program state-space coverage during fuzzing is difficult, and modern coverage-guided fuzzers use code coverage as an approximation of program state-space coverage. However purely coverage-guided fuzzers cannot effectively explore the state space of a program [23], as *code coverage does not always correlate to program-state coverage.*

Coverage-based fuzzers encounter three problems when exploring state spaces [23]:

Known Relevant State Values: Coverage-based fuzzing is effectively blind to program semantics and so cannot distinguish truly interesting states, which are often represented by a small set of known values.

Known State Changes: Sometimes it is unclear what values represent program states or there may be too many such values.

Missing Intermediate State: In certain cases, there are no known values or state changes that can be exposed to the fuzzer. As a result, it is necessary to create artificial intermediate states and expose them to the fuzzer.

Ijon addresses these issues by augmenting the coverage bitmap with values of human-annotated state-representative variables to *increase the correlation between bitmap coverage and state space coverage*: For example, consider a maze program where each valid input character permits a move direction, as shown in Figure 8. Although the user has many possible positions and paths (thus a large state space), a coverage-guided fuzzer that only considers code or branch coverage will struggle to explore them. Ijon exposes x and y to the fuzzer, enabling the fuzzer to recognize inputs leading to new locations in the maze as interesting, essentially creating a *player-location-based coverage bitmap* (the bottom of Figure 8). This results in a better exploration of the state space and a solution to the maze.

Ijon requires human annotation of state-representative variables, which does not scale. In this chapter, I explore techniques to *automatically* infer and use state-representative variables to improve fuzzing efficacy.

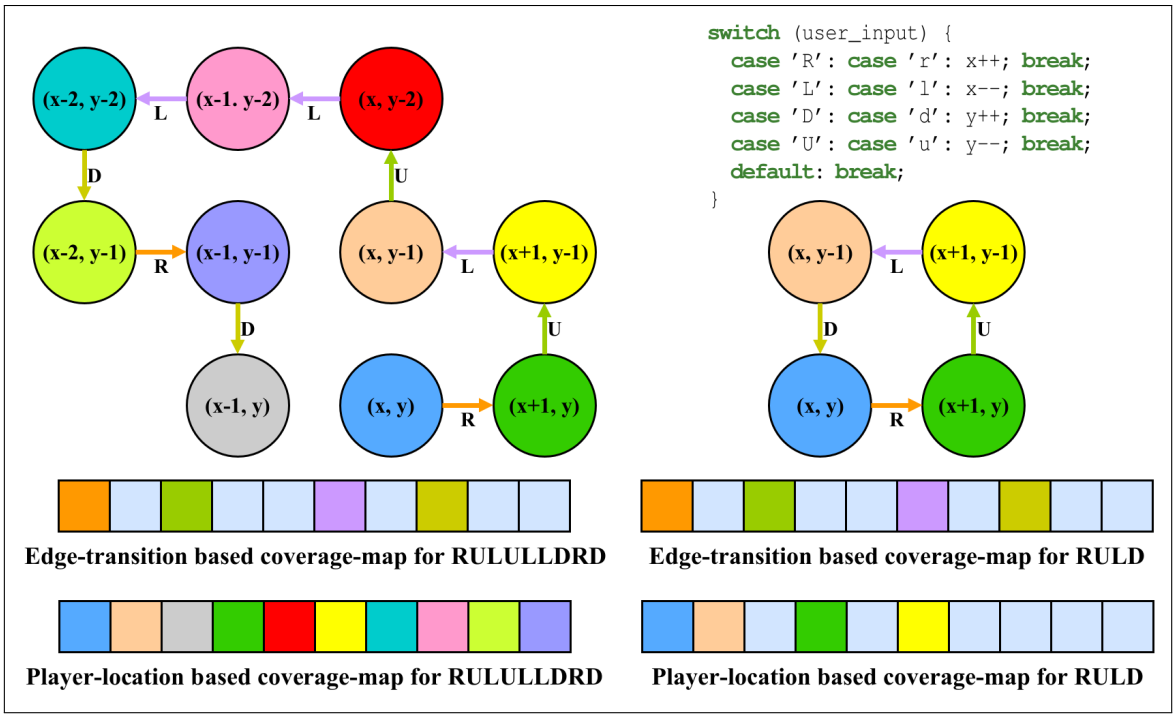

Figure 8. Comparison of edge-transition based and player-location based coverage-maps for a maze program. For ease of comparison, I disregard the edge-transition count used in edge-transition based coverage. Edge-transition cannot identify the distinct states resulting from different player locations whereas player-location can.

5.2 Overview

I propose SandPuppy, a technique that automatically identifies and instruments state-representative variables, thus exposing program state to the fuzzer.

Figure 9 provides an overview of SandPuppy's fuzzing pipeline. Given buildable program source, SandPuppy carries out the following steps iteratively, identifying and selecting new state-representative variables in each iteration:

Collect variable-value traces: SandPuppy instruments the target program to output variable-value traces of integer variables. It then fuzzes this instrumented binary using a standard fuzzer for a fixed amount of time, using its accumulated

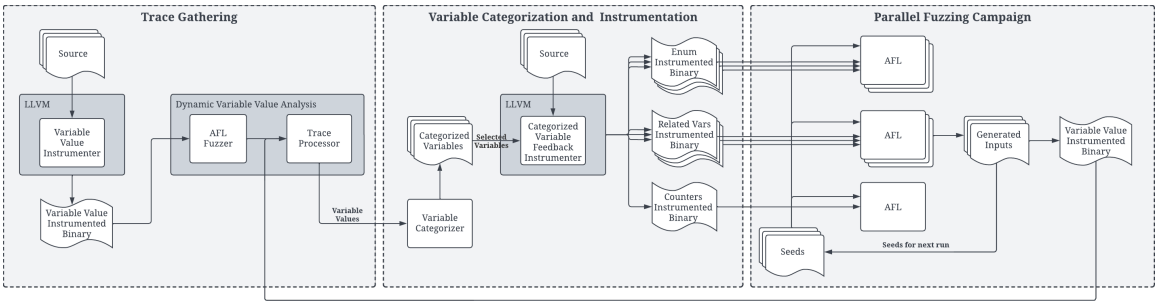

Figure 9. SandPuppy's architecture is comprised of three phases: a trace-gathering phase that gathers variable value traces from an instrumented binary, a categorization and instrumentation phase that categorizes variables based on value traces and performs feedback instrumentation using selected variables, and a parallel-fuzzing phase were the instrumented binaries are fuzzed together. Inputs generated by the parallel-fuzzing campaign are used to generate additional traces and as seeds for subsequent runs.

set of test-cases (seeds). SandPuppy uses the trace results to select a tractable subset of state-representative variables, as described in Section 5.3.

Identify state-representative variables: SandPuppy analyzes the collected traces to identify potential state-representative variables and their semantic categorization (Section 5.4) using a set of heuristics (defined in Section 5.5 and applied in Section 5.6). Traces from inputs that result in crashes or errors are excluded to minimize noise by ignoring invalid states.

Fuzz: SandPuppy generates instrumented binaries that collect feedback from corresponding state-representative variables, as detailed in Section 5.7. It then runs a parallel-fuzzing session using this set of binaries. Note that generated inputs are periodically synchronized between all fuzzing instances.

Repeat: The inputs generated by the parallel-fuzzing session are used as seeds for the next run's variable-value trace collection and fuzzing.

I implemented SandPuppy atop AFL [21]. In my evaluation (Section 5.8), Sand-

PUPPY outperforms traditional coverage-based fuzzers and even slightly outperforms IJON, despite not benefiting from manual analysis.

5.3 Trace Collection

SANDPUPPY classifies variables by observing their use by the program in different states. It uses a trace collecting step to instrument the program and record the complete history of values that each integer variable receives throughout the program's execution.

SANDPUPPY needs a variety of variable traces. Consider a variable counting up to a number provided by input: if the input is always zero, I will only have a single trace of the variable, which is indistinguishable from a trace of a constant variable set to zero and never modified. It is also unlikely that all code paths would have been exercised by an input, meaning that the trace will not contain information from variables not modified in those paths.

For the first iteration of SANDPUPPY, I leverage an initial fuzzing phase to generate traces on multiple inputs, mitigating the influence of invalid inputs on later variable classification by discarding crashing and erroring traces. I also exclude redundant data by only collecting traces from inputs that improve coverage, or increase existing block-transition counts. On subsequent iterations, I directly use inputs identified previously to generate traces. In this manner I can detect, categorize, and instrument any newly-detected variables. I also filter out the traces of any variable whose category has not changed since the last run, assuming that resources would be better spent on tracing and classifying still-changing variables.

For the variable v, I define the *value trace* T as a sequence of tuples of the form

```
void process_input(const char* input, state* state) {
  parsed_messages pmsgs = parse_input(input);
  for (int i = 0; i < pmsgs.num_messages; i++) {
    msg_type mtype = process_msg(pmsgs.messages[i], state);
    char* mtype_str = get_message_type(msg_str);
    if (strcmp(mtype_str, "type_1") == 0) {
        mtype = TYPE_1;
        process_type_1_msg(msg_str, state);
    } else if (strcmp(mtype_str, "type_2") == 0) {
        mtype = TYPE_2;
        process_type_2_msg(msg_str, state);
        ...
```

Listing 5.1. Enum variables exposing known state changes.

(m, ν) where m is the source line-number where v was modified and ν is the current value of v. I also define $\mathbf{T}$ to be a sample of all traces I have collected for that variable from the fuzzing session; i.e., assuming the program was run against n inputs $\mathbf{T} = (T_1, T_2, \ldots, T_n)$.

5.4 Semantic Categories

SandPuppy instruments *state-representative* variables to approximate program states at runtime. Different variables contribute differently to this approximation. Here, I describe categories of state-representative variables and my intuition of how they represent the application's state space.

Enums. *Enumerated types* represent a fixed set of named values. While the internal representation is arbitrary, they typically compile to integer values. Programmers use enums to enhance readability and leverage the type system to prevent variables from being set to invalid values. Enums can also be correlated with input, especially input involving structured formats. In such cases, the program uses enums to identify the different sections or components of the input.

Format processing code often has states associated with user input, and variables encoding such states are attractive targets for instrumentation. Listing 5.1 shows a message-processing function, where different combinations of messages can affect the program state in unique ways. Coverage-based fuzzers identify values of `mtype`, but may not recognize combinations of messages as interesting states as they trigger already-seen control-flow edges. Conversely, fuzzers that can reason about the value of `mtype` directly may be able to trigger additional program states.

Similar code can be found in a wide variety of real-world software, such as FFMpeg [75]–[78], jsoncpp [79], libjpeg [80], and libpcap [81].

Counters. Counters are variables that count up or down to some value. I assume that counters are monotonically increasing or decreasing, and that they may do so by arbitrary (and varying) amounts. Counters can either be *static*: counting to a fixed value in every program execution, or *dynamic*: counting to different values in every program execution. Counters may also correlate with input as programs use them to count input elements or to sum input-element sizes. By guiding fuzzers towards maximum counter values, states revealing boundary and edge-case conditions can be explored.

Listing 5.2 shows an example of counter usage. By exposing the counter values `i` and `processed_messages` to the fuzzer and favoring executions that maximize their values, the fuzzer will explore program states involving both a larger number of messages and a larger number of elements per message. Examples of such variables can once again be found in real-world software such as FFMpeg [82]–[84].

Related Variables. Programs often feature *related variables* whose *combined* values represents a unique state. For example, variables x and y used to represent a player's location in a maze are related variables. As the example in Section 5.1 demonstrates,

```
unsigned int processed_messages = 0;
void process_message(message* msg) {
  for (int i = 0; i < get_num_elements(msg); i++) {
    process_element(msg, i);
  }
  processed_messages++;
}
```

Listing 5.2. Counter variables exposing memory-related state.

with standard coverage-based approaches this state information is not visible to fuzzers. By identifying related variables and exposing their combined state, fuzzers gain insight into a program's state space.

5.5 Semantic Category Heuristics

I use several heuristics on traces collected in Section 5.3 to classify variables as state-representative and sort them into categories that are described in Section 5.4.

5.5.1 Non-Semantic Categories

I start by defining heuristics to identify constants and booleans; two categories of variables that are not state-representative. Constants do not expose any useful state, as they do not change values. While booleans can expose useful state, coverage-based fuzzing is usually sufficient to explore these states as booleans often dictate control flow.

First, I define the observed set of values of the variable v:

$$\mathcal{V} = \bigcup_{T \in \mathbf{T}} \left\{ \nu \mid (m, \nu) \in T \right\}$$

Using this I will also define the *number of unique values* $h_{nuniq} = |\mathcal{V}|$. If $h_{nuniq} = 1$,

then it is highly probable that the variable is a constant. However, if $h_{nuniq} = 2$ and $\mathcal{V} = \{0, 1\}$, then it is likely a boolean. Note that there is one surprising case of constants: a variable initialized to zero and then set to a value later. To handle this case, SandPuppy categorizes variables with $\mathcal{V} = \{0, n\}$ where $n \neq 1$ as representing a constant.

5.5.2 Semantic Categories

Enums. I define a heuristic for identifying enums that take on values that are either directly present in the input or are mapped to certain input values. Additionally, I also assume that an arbitrary number of such values exist in the input. This means that, for enums, the number of times it is modified may correlate with the input size. I define the function $isz : 2^T \rightarrow \mathbb{Z}_{\geq 0}$ that returns the size of the input used to generate the trace T. I then have:

$$\mathbf{S} = (isz(T_i) \mid T_i \in \mathbf{T})$$

$$\mathbf{M} = (|T_i| \mid T_i \in \mathbf{T})$$

I can now calculate the Pearson correlation coefficient for my sample, called the *times-modified to input-size correlation*, as follows:

$$h_{tmisc} = \frac{\sum_{i=1}^{|\mathbf{T}|}(s_i - \bar{s})(m_i - \bar{m})}{\sqrt{\sum_{i=1}^{|\mathbf{T}|}(s_i - \bar{s})^2}\sqrt{\sum_{i=1}^{|\mathbf{T}|}(m_i - \bar{m})^2}}$$

The closer h_{tmisc} is to 1, the more likely that the variable is set based on values from the input. Note that this measure has some caveats, particularly if the discrete input elements are arbitrarily sized. Consider the example in Listing 5.1. With arbitrarily-sized messages, the program can process 3 messages of size n each, or a single message of size $3n$. In the first case the enum variable is modified three times,

whereas in the second case it is modified only once. However, in both cases the input size is $3n$.

Consider another example demonstrated in Listing 5.1: The enum variable `mtype` is set based on the value of the string `mtype_str`. Also, `mtype` is modified on three lines; one for each message type. This means that if the number of lines where the variable is modified equals the number of unique values it can take, it is probably part of a construct such as the one in Listing 5.1. Therefore, I define the *number of modified lines* as:

$$h_{nmod} = \left| \bigcup_{T \in \mathbf{T}} \left\{ m \mid (m, \nu) \in T \right\} \right|$$

I also consider the case where variable values are assigned without being part of a structure similar to Listing 5.1. Here, the variable is only modified on one line.

Counters. I created several heuristics to identify if a variable is a counter. My first intuition is to classify variables correlated with input size, as counters. This allows me to detect variables being used as such without detecting if they actually "count" (which other heuristics will do). If a strong correlation exists between a variable's maximum value in a trace and the input size for that trace, it is categorized as a counter:

$$\mathbf{S} = (isz(T_i) \mid T_i \in \mathbf{T})$$

$$\boldsymbol{\mathcal{V}}^{max} = (\nu_i^{max} \mid T_i \in \mathbf{T} \wedge \nu_i^{max} = \max(\{\nu_i \mid (m_i, \nu_i) \in T_i\}))$$

Using this I can calculate the Pearson correlation coefficient for my sample, which I call the *maximum-value to input-size correlation*, as follows:

$$h_{mvisc} = \frac{\sum_{i=1}^{|\mathbf{T}|}(s_i - \bar{s})(\nu_i^{max} - \bar{\nu}^{max})}{\sqrt{\sum_{i=1}^{|\mathbf{T}|}(s_i - \bar{s})^2}\sqrt{\sum_{i=1}^{|\mathbf{T}|}(\nu_i^{max} - \bar{\nu}^{max})^2}}$$

Using h_{mvisc} as a measure of input-size correlation—the closer the value is to 1, the more likely it is that the variable holds values correlated with input size.

Next, I develop heuristics to detect if a variable is counting and to distinguish between static and dynamic counters. I first define *maximum-values variance*:

$$h_{mvv} = \frac{\sum_{i=1}^{|\mathbf{T}|}(\nu_i^{max} - \bar{\nu}^{max})^2}{|\mathbf{T}|}$$

This heuristic calculates the variance of the observed maximum values of the variable. As static counters count over the same set of values every time, the corresponding value of h_{mvv} will be 0 whereas dynamic counters will have a non-zero value.

The second heuristic, the *average value-set-cardinality ratio*, is defined as:

$$h_{avscr} = \frac{\sum_{T \in \mathbf{T}} \frac{|\{\nu|(m,\nu) \in T\}|}{|\mathcal{V}|}}{|\mathbf{T}|}$$

This heuristic is an average of the ratio of the number of unique values in a trace, and the number of unique values across *all* traces for a particular variable. If h_{avscr} is 1, then the variable takes on the exact same set of values in every trace. If this variable is a counter, then it must be static, and I can ignore it as it does not expose useful state.

The next set of heuristics for identifying counters requires pre-processing of variable traces. The intent is to identify segments of traces that could be *counter segments*, meaning that the segment reflects a variable counting to some value. This is important because counters are frequently used multiple times in a single run, and so a trace can contain multiple such segments. The algorithm (Algorithm 5.1) takes as input a single trace and returns one or more counter segments. It also ensures that each segment contains at least two values and that there are no repeating values in a segment. Runs can occur if a piece of code with a counter is called repeatedly, where the counter loops only a single time. But it can also happen in the case where a variable is being

Algorithm 5.1 Implementation of *counter_segments* function

```
1:  function COUNTER_SEGMENTS(T):
2:      function FILTER(segment):
3:          filtered as LIST
4:          previous_value as INTEGER
5:          filtered ← []
6:          previous_value ← NONE
7:          for each current_value in segment:
8:              if current_value ≠ previous_value :
9:                  filtered.ADD(current_value)
10:                 previous_value ← current_value
11:         return filtered
12:     diff_sum as INTEGER
13:     segment as LIST
14:     filtered as LIST
15:     segments as LIST
16:     diff_sum ← 0
17:     segment ← []
18:     for each i in [1, |T| − 1]:
19:         diff ← t_{i+1}.ν − t_i.ν
20:         if diff ≠ 0 and diff_sum ≠ 0 and diff/|diff| ≠ diff_sum/|diff_sum| :
21:             segment.ADD(t_i.ν)
22:             filtered ← filter(segment)
23:             if LENGTH(filtered) > 1 :
24:                 segments.ADD(filtered)
25:             diff_sum ← 0
26:             segment ← []
27:         else:
28:             diff_sum ← diff_sum + diff
29:             segment.ADD(t_i.ν)
30:     segment.ADD(t_{|T|}.ν)
31:     filtered ← filter(segment)
32:     if LENGTH(filtered) > 1 :
33:         segments.ADD(filtered)
34:     return segments
```

repeatedly set to the same value. Due to this ambiguity, I collapse such runs into a single value.

The first heuristic using counter segments is called *directional consistency*:

$$h_c = \frac{\left|\sum_{T \in \mathbf{T}} \sum_{S \in counter_segments(T)} dir(S)\right|}{\sum_{T \in \mathbf{T}} |counter_segments(T)|}$$

where:

$$dir(S) = \begin{cases} 1 & \text{if } s_2 > s_1 \\ -1 & \text{otherwise} \end{cases}$$

The purpose of the directional consistency heuristic is to identify counter variables that are incremented or decremented consistently. For counters, h_c will be close to 1.

The final heuristic using counter segments is called *loop-sequence proportion*:

$$h_{lsp} = \frac{\sum_{T \in \mathbf{T}} \sum_{S \in counter_segments(T) \wedge |S| > 2} |S|}{\sum_{T \in \mathbf{T}} |T|}$$

This heuristic measures what proportion of a variable's traces are part of a loop sequence. For counter variables, this value should be close to 1. I deliberately exclude counter segments of length 2 or less, as two-value segments of the form $[a, b]$ where $a > b$ or $a < b$ are common, meaning that even non-counter variables have a h_{lsp} close 1. However, it is far less likely for such variables to have segments of size 3 or higher where they are consistently incremented or decremented.

5.6 Identification of Semantic Categories

Once a set of variable-value traces are discovered, SandPuppy uses the heuristics defined in Section 5.4 to categorize each of the variables into a semantic category. Note that the heuristics have thresholds associated with them, which I define here, and Section 5.8 describes the thresholds set during the evaluation. Variables deemed state-representative are instrumented to provide feedback during fuzzing (described in Section 5.7).

Enums and Counters. To determine if a variable is an enum, SandPuppy checks if h_{tmisc} is greater than or equal to a threshold τ_{tmisc}. If the variable is modified on multiple lines, SandPuppy also checks whether the number of lines is the same as the number of values it receives.

SandPuppy determines whether a variable is correlated with input size by checking if the variable's h_{mvisc} feature is greater than or equal to the threshold τ_{mvisc}.

For counters, I check to see if a variable's h_{lsp} and h_c are greater than or equal to the threshold τ_{lsp} and τ_c. To determine if a counter is static or not, SandPuppy

checks if h_{mvv} is 0 (i.e., it always counts to or from the same value) and h_{avscr} is 1 (i.e., it counts over the same set of values every time).

Related Variables. SandPuppy identifies sets of related variables using Algorithm 5.2. SandPuppy starts with a list of all variables from the specified function and file that are not constants, booleans, or unclassified variables.

SandPuppy then creates a candidate list of related variables for each variable. First, SandPuppy chooses one of the lines where the variable is modified. Second, it scans up to δ_{max} lines before and after to identify other variables are modified that range. SandPuppy does not do this for every line that modifies the variable because the intuition is that related variables are modified in close proximity to each other. Lastly, SandPuppy prunes each variable's candidate list to include only those variables that it considers related, by ensuring that (1) both variables are modified on the same number of lines and (2) their trace-length distributions are identical.

SandPuppy's pruning strategy does exclude certain variable pairs that have identical trace-length distributions. For example, SandPuppy will not identify related variables when one variable is modified on one line, but the second is modified based on mutually-exclusive conditions. In this situation, for any trace generated from the same input, both variables will have been modified the same number of times, even if overall, the second variable is modified on more lines than the first. SandPuppy does this to make feedback instrumentation simpler by avoiding reasoning about the logical context in which such variables are modified.

Algorithm 5.2 Identification of related-variable sets

```
1:  function IDENTIFY_RELATED_VARIABLE_SETS(file, func, δ_line):
2:      related_var_sets as SET
3:      candidates as SET
4:      vars_by_modified_line as DICTIONARY
5:      var_to_related as DICTIONARY
6:      V ← {v ∈ get_vars(file, func) | v.class ∉ {unknown, constant, boolean}}
7:      for each v in V:
8:          for each m in ⋃_{T∈v.T} {m | (m, ν) ∈ T}:
9:              if m ∉ variables_by_modified_line :
10:                 variables_by_modified_line[m] ← ∅
11:             variables_by_modified_line[m].ADD(v)
12:     for each v in V:
13:         m ← (⋃_{T∈v.T} {m | (m, ν) ∈ T} as LIST)[0]
14:         candidates ← ∅
15:         for each l in [m − δ_line, m + δ_line]:
16:             if l ∈ vars_by_modified_line :
17:                 candidates ← candidates ∪ (vars_by_modified_line[l] \ {v})
18:         for each v_c in candidates:
19:             if |v.T| = |v_c.T| ∧ v.h_nmod = v_c.h_nmod ∧
                  max(M) = max(M_c) ∧ min(M) = min(M_c) ∧
                  mean(M) = mean(M_c) ∧ stddev(M) = stddev(M_c) :
20:                 if v ∉ var_to_related :
21:                     var_to_related[v] ← ∅
22:                 var_to_related[v].ADD(v_c)
23:     for each v in var_to_related:
24:         for each v_r in var_to_related[v]:
25:             related_var_sets.ADD([v, v_r] as SET)
26:     return related_var_sets
```

5.7 Feedback Instrumentation

Having mapped variables to semantic categories, SandPuppy instruments state-representative variables to generate several variants of the program that expose different feedback information.

My implementation is based on the feedback mechanisms proposed in Ijon [23] (as my goal is to apply them automatically, where Ijon required manual annotations). Instrumentation uses LLVM passes based on the FuzzFactory framework [42]. In all cases, I use the FuzzFactory max reducer function to aggregate feedback across multiple inputs. The max reducer function marks an input as interesting if in the corresponding bitmap, a new location is set, or if a location has a higher value than is currently known.

```
unsigned int sequence = 0u;
msg_type process_msg(const char* msg_str, state* state) {
  msg_type mtype;
  char* mtype_str = get_message_type(msg_str);
  if (strcmp(mtype_str, "type_1") == 0) {
    mtype = TYPE_1;
    sequence = (sequence << 8u) + mtype;
    dsf_set(dsf_map, sequence, 1);
    process_type_1_msg(msg_str, state);
  } else if (strcmp(mtype_str, "type_2") == 0) {
    mtype = TYPE_2;
    sequence = (sequence << 8u) + mtype;
    dsf_set(dsf_map, sequence, 1);
    process_type_2_msg(msg_str, state);
  }
  ...
}
```

Listing 5.3. Variable value permutation feedback. The sequence-variable maintains the state history of message-types processed, allowing the fuzzer to recognize different permutations of message types.

5.7.1 Permutation Feedback

This instrumentation is similar to IJON's state-change log. To record state changes for enum variables, I shift the value of a "state-log" variable (initialized to zero) by 8 bits, then add the enum's value to it. I use the calculated value to set the corresponding location in the feedback bitmap to 1. Figure 10 demonstrates how this feedback is an improvement over edge-transition-based coverage. In edge-transition-based coverage, different permutations of the same five states result in the same edge coverage. However, by tracking the history of processed messages the fuzzer can recognize new message sequences as interesting. IJON always shifts by 8 bits. In my implementation I also shift by 4 bits if the observed values of an enum variable are in the range 0 to 15, allowing me to record longer state history for it. I perform variable-value permutation feedback (see Listing 5.3) on enum variables to identify inputs resulting in unique permutations of values, potentially allowing exploration of

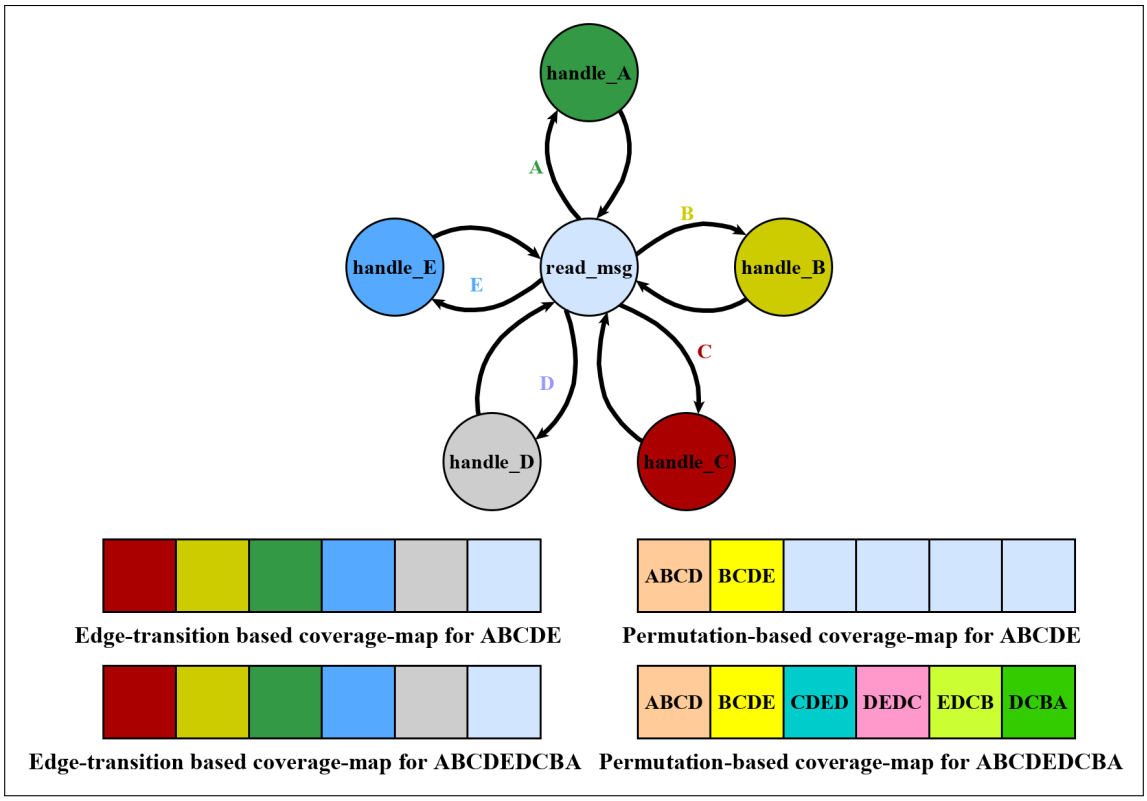

Figure 10. Comparison of edge-transition based and permutation-based coverage-maps for a message-processing program. The former cannot identify the distinct states resulting from different message-sequences whereas the latter can. The permutation-based coverage map does include entries for A, AB, and ABC; these have been excluded for brevity.

associated complex-states. I generate a variant with this feedback for each identified enum variable.

5.7.2 Maximization Feedback

This instrumentation (see Listing 5.4) is similar to the `IJON_MAX(x)` primitive in IJON [23]. For dynamic counters and variables correlated with input size, I employ instrumentation to guide fuzzing towards inputs that maximize their values. By doing

```
// Random static values generated by LLVM pass
unsigned int i_idx = 0x00c0ffeecafef00d;
unsigned int proc_msgs_idx = 0xdeadbeefcafef00d;
unsigned int processed_messages = 0;
void process_message(message* msg) {
  for(int i = 0; i < get_num_elements(msg); i++) {
    dsf_set(dsf_map, i_idx, i);
    process_element(msg, i);
  }
  processed_messages++;
  dsf_set(dsf_map, proc_msgs_idx, processed_messages);
}
```

Listing 5.4. Variable value maximization feedback. This feedback helps the fuzzer recognize states that maximize the value of the instrumented variable.

```
unsigned int slot_size = 1;
void foo(const char* input) {
  int var_1 = bar(input), var_2 = baz(input);
  dsf_set(dsf_map, var_2 / slot_size, var_1);
}
```

Listing 5.5. Related-variable-pair maximization feedback. This feedback maximizes one variable with respect to another, allowing the fuzzer to recognize different maximum values of `var_1` with respect to `var_2`.

so, I hope to uncover boundary or edge-case related bugs and memory vulnerabilities. I generate a single variant with this feedback for all identified counters.

5.7.3 Related-Variable Maximization Feedback (RVMF)

This instrumentation (see Listing 5.5) is performed on every related-variable pair and is similar to the `IJON_MAX(slot, x)` primitive in IJON [23]. It maximizes the value of one variable with respect to another, which is useful in situations where the range of values taken on by a variable is a function of another. I generate two variants with this feedback for each identified pair of related variables, as I do not know which variable is the dependent one.

Consider the message-processing example: It is common for messages to have

```
while(true) {
  curr_row = row;
  curr_col = col;
  unsigned int idx = hash_ints(curr_row, curr_col);
  dsf_set(dsf_map, idx, 1);
  ...
}
```

Listing 5.6. Related-variable-pair combination feedback. This feedback helps the fuzzer recognize unique combinations of values of the two variables as novel states.

payloads, with payload-size varying based on the message type. Assume that a vulnerability exists in the processing code for a message type with the lowest payload-size limit, triggered when the payload size equals the limit. Maximizing payload size by itself is unlikely to trigger the vulnerability as it only helps identify the *overall* maximum-possible payload size. But maximizing the payload size with respect to message type allows the fuzzer to discover the maximum-possible payload size for *every* message type, leading to a thorough state space exploration and triggering the vulnerability. A real-world example of such code can be seen in FFMpeg's DTS-HD demuxer [85].

5.7.4 Related-Variable Combination Feedback (RVCF)

This last instrumentation (see Listing 5.6) is also performed on every related-variable pair, and is similar to the `IJON_SET(hash_ints(x, y))` annotation in Ijon [23].

5.8 Evaluation

I perform several evaluations to understand SandPuppy's ability to automatically add state-representative variable feedback to fuzzing, and the impact of this ability on analysis outcomes.

Reasoning about states. I first develop understanding of SandPuppy's core contribution by evaluating its variable semantic category identification in Section 5.8.1 and exploring the value of this identification in the actual exploration of complex states in software in Section 5.8.2.

Fuzzing for vulnerabilities. I then perform a comparative evaluation of SandPuppy against state-of-the-art fuzzers, both in terms of achieved code coverage in Section 5.8.3 and triggered vulnerabilities in Section 5.8.4.

Understanding variable impact. Finally, for a deeper understanding of individual categories identified by SandPuppy, I perform an ablative evaluation in Section 5.8.5.

I performed initial fuzzing for trace-gathering and classification on a Linux Mint 20.2 machine with an Intel Core i9-8950HK CPU and 32 GB RAM. Parallel fuzzing was performed on a 40-node Kubernetes cluster with each node having two Intel(R) Xeon(R) CPU E5-2670 v2 processors. Each fuzzing instance receives identical resources during the parallel-fuzzing session.

I set initial fuzzing time to one hour, and observed that the dense instrumentation in this phase slows the fuzzer down significantly, making it equivalent to around 5-10 minutes of fuzzing with a regular AFL instrumented binary. The results will demonstrate however, that even with this performance hit, SandPuppy is able to gather enough traces to identify and categorize state-representative variables.

Through manual experimentation and observation on LIBTPMS and a test-bed described in Section 5.8.1, I derived the following threshold values: $\tau_{mvisc} = 0.6$, $\tau_{lsp} = 0.7$, $\tau_c = 0.8$, $\tau_{tmisc} = 0.5$, and $\delta_{max} = 6$. I note that these values may be program dependent and could potentially vary from program to program. However, in my experiments these threshold values appeared to work across multiple programs. Increasing the threshold values makes them "more strict" and could help minimize false positives, but could also exclude legitimate variables, whereas relaxing them does the opposite. The trade-off is between exposing enough state and generating too many useless inputs that can slow down fuzzing.

5.8.1 Identification of Semantic Categories

There is a fundamental trade-off between extraneous instrumented state-representative variables (too many variants slow down fuzzing) and insufficient state-representative variables (missing exposing state to fuzzing). As my goal is not to solve the variable semantic categorization problem in general, but to improve fuzzing efficacy, I first evaluated SANDPUPPY against a custom test-bed program (to establish a baseline) and then against real-world programs to better understand its state reasoning.

My test-bed program features 10 static counters, 10 dynamic counters, and 10 enums. The program reads a structured string from `stdin` line by line. The input causes the program to either (1) trigger a specified static counter, (2) trigger a specified dynamic counter up to a provided limit, or (3) set the value of a specified enum using the provided list of values. I manually analyzed traces to verify that variables and their semantic categories were identified properly. To validate my thresholds, I compared

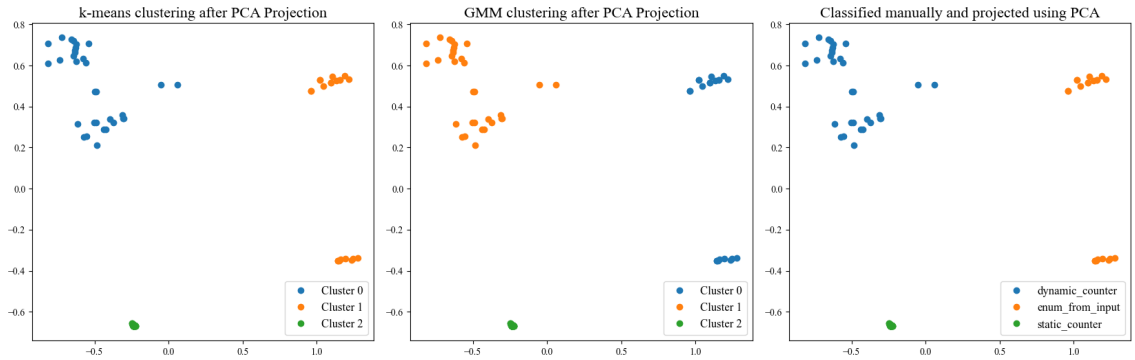

Figure 11. SandPuppy's classification on test-bed program variables versus k-means and GMM. SandPuppy's classification is identical to both approaches.

against k-means and GMM clustering on feature vectors composed of my heuristics, after reduction to two dimensions using PCA. As can be seen in Figure 11, my heuristic approach matches both clustering approaches.

I evaluated SandPuppy's automatic variable categorization on the following real-world programs: libpng (1.6.37), libtpms (0.8.3), jsoncpp (1.9.5), dmg2img (commit `a3e4134`), and Super Mario Bros. In this experiment, I manually verified the semantic category of the variables based on my analysis of the code. My analysis discovered minor discrepancies that were limited to dynamic counters being classified as enums and some enum variables that were not identified. Two factors cause this: (1) lack of a sufficiently-representative trace-distribution for a particular variable and (2) ambiguity inherent in my approach for detecting enums, which depends on correlation between input size and how often the variable is modified. In structured input-formats with variably-sized components, this correlation can be difficult to derive because it is possible, for example, to have an input with two components of size n, or an input with a single component of size $2n$.

However based on my analysis, the categorization of variables was largely accurate. A small sample of the results is in Table 5. SandPuppy accurately identified enums, static and dynamic counters, and related variables.

Table 5. Sample of classified variables with labels from various real-world programs. Variable names include the file and function name. The last column specifies whether the variable was identified as state-representative or not.

Target	Qualified Variable Name	Label	State Representative
libpng	pngrutil.c:png_check_chunk_name:i:3140	static_counter	N
	pngrutil.c:png_check_chunk_name:c:3147	enum_from_input	Y
	pngrutil.c:png_read_filter_row_up:i:3956	dynamic_counter	Y
libtpms	AlgorithmTests.c:AllModesAreDone:alg:265	static_counter	N
	ExecCommand.c:ExecuteCommand:command.index:111	enum_from_input	Y
	Marshal.c:TPML_ALG_Marshal:i:752	dynamic_counter	Y
jsoncpp	json_value.cpp:JsonValue::ValueType:type:347	enum_from_input	Y
	json_value.cpp:JsonValue::setType:v:595	enum_from_input	Y
	json_reader.cpp:OurReader::readArrayToken:index:1507	dynamic_counter	Y
dmg2img	dmg2img.c:percentage:i:77	static_counter	N
	base64.c:decode_base64_char:c:49	enum_from_input	Y
	dmg2img.c:read_kolyblk:k.XMLLength:148	dynamic_counter	Y
smbc	PPU.cpp:PPU::writeDMAEh:i:501	static_counter	N
	Main.cpp:mainLoop:world_pos:309	related_variable	Y
	Main.cpp:mainLoop:pos_y:311	related_variable	Y

I compared the state-representative variables SandPuppy identified to those *manually identified* by Ijon [23] in Table 6[1]. In every case SandPuppy automatically identified the same state-representative variables with the appropriate semantic category. From Table 7 it is also clear that SandPuppy automatically identified a large number of state-representative variables compared to the (understandably) few manually-annotated ones.

An interesting result from dmg2img involves the `kolyblk.XMLLength` variable in the `main` function, that Ijon identified. `XMLLength` is never modified in the `main` function, and SandPuppy cannot recognize it. However, the `kolyblk` struct is passed as argument `k` to the `read_kolyblk` function *which does modify the* `XMLLength` *member*. Therefore, `k.XMLLength` and `kolyblk.XMLLength` are aliases. These results demonstrate that my approach identifies some of the same state-representative variables as human experts and can be applied to real-world situations.

[1]Note that libpng is not present because it is not included in the released data: https://github.com/RUB-SysSec/ijon-data

Table 6. Comparison between state-representative variables manually identified by IJON and automatically identified by SANDPUPPY. SANDPUPPY identified every state-representative variable identified by a human expert, with the appropriate semantic.

Target	Qualified Variable Name	SandPuppy Identified	Label
libtpms	`ExecCommand.c:ExecuteCommand:command.index:111`	Y	`enum_from_input`
dmg2img	`dmg2img.c:read_kolyblk:k.XMLLength:148`	Y	`dynamic_counter`
smbc	`Main.cpp:mainLoop:world_pos:309`	Y	`related_variable`
smbc	`Main.cpp:mainLoop:pos_y:311`	Y	`related_variable`

Table 7. Comparison of number of integers SANDPUPPY identified as state-representative, and the number of IJON annotations. Total integers in each target have also been shown.

Target	Integers	SandPuppy State	Ijon Annotations
libtpms	14,436	65	1
dmg2img	193	18	1
smbc	598	12	1
libpng	2525	54	1

5.8.2 Exploring Complex States

In the following experiments I evaluated SANDPUPPY's ability to explore complex state typically not accessible to purely coverage-based approaches; games and protocol parsers present a significant challenge as certain states can only be reached through a particular sequence of prior ones. In these experiments I limited SANDPUPPY to a single run as initial trace-collection was sufficient to correctly identify and categorize variables that expose state information of interest.

5.8.2.1 Traversing the Maze

A maze is a simple program originally used to demonstrate symbolic test case generation [86]. I used three maze programs: one used by KLEE [86], one presented in IJON [23], and one that I developed. All three programs terminate when the player

runs into a wall or backtracks. Additionally, both the IJON maze and my maze feature dead-ends. In my maze I added loops, which do not exist in the others.

SANDPUPPY accurately identified the pair of related variables representing the player's coordinates and used RVCF instrumentation (in addition to feedback from other potentially-interesting variables). I compared the ability of AFL and SANDPUPPY to find a solution to the maze across 5 runs (24 hours each); both were provided an initial, uninformative seed. The results are shown in Table 8.

AFL only solved the KLEE maze while SANDPUPPY solved all mazes. With the KLEE maze, my approach was able to find a solution faster than plain AFL every time.

SANDPUPPY surprisingly identified variables *other than* player coordinates, which allowed the fuzzer to find a solution—often faster than the instance with player coordinates. My maze program maintains two variables: `last_option` and `moves`. The former ensures that players cannot backtrack and the latter holds the number of moves made. SANDPUPPY identified these as related and instrumented them such that `moves` is maximized with respect to `last_option`. The generated feedback effectively maximizes the number of moves, even if the values are bucketed according to the last option. Since the KLEE and IJON mazes do not have loops, and because the longest path is also the solution, maximizing `moves` ends up helping the fuzzer solve the maze. This shows that SANDPUPPY can identify *useful* state-representative variables that a human might not consider.

Table 8. Median solution times (hh:mm) for all three mazes (5 runs). SandPuppy solved all mazes, while AFL only solved the Klee maze in 2 out of 5 runs.

Fuzzer \ Maze	**Klee Maze**	**Ijon Maze**	**SandPuppy Maze**
AFL	14:52 (2/5)	No Solution	No Solution
SandPuppy	00:28	04:30	04:38

5.8.2.2 Super Mario Bros

In addition to the maze, Ijon [23] also evaluated on Super Mario Bros. I used the version of the game modified by the Ijon authors [23]: Changes include reading input from `stdin` and speedrunning changes. SandPuppy identified a number of related variables, including those describing Mario's position in the world.

I fuzzed Super Mario Bros using AFL and SandPuppy for up to 200 hours and recorded Mario's progress. Again, all fuzzers used identical uninformative seeds. Results are shown in Table 9 and include how often the level was solved, the median time until the best solution was found (hh:mm), and the median percentage of distance covered.

SandPuppy solved all worlds except two. SandPuppy also found the warp zones in worlds 1-2 and 4-2. World 2-1 could not be fully solved as Mario cannot use the spring due to the speedrunning code-changes. In world 4-4, Mario repeatedly gets stuck in a dead end. SandPuppy also solved worlds 6-2 and 8-4 that Ijon could not.

A fair number of solutions were found by targets using RVMF from player coordinates, but many were also found by those using RVCF from the same variables. In a few cases, solutions were found by targets using RVMF from Mario's world and screen x coordinates. As the latter is constant, this effectively maximizes the world x coordinate which normally would have caused Mario to deadend. However syncing

Table 9. SandPuppy, AFL, and Ijon playing Super Mario Bros (3 runs each). I show how often the level was solved and the median percentage of distance traveled. For SandPuppy and AFL, I include the median time (hh:mm) to find the solution. Ijon results presented are the originals (time omitted as it is not comparable).

	SandPuppy			**AFL**			**Ijon**	
World	Solved	Time	% Distance	Solved	Time	% Distance	Solved	% Distance
1-1	✓	27:58	100%	✗	168:04	69%	✓	100%
1-2	✓	30:03	100%	✗	96:15	68%	✓	100%
1-3	✓	41:10	100%	✗	58:04	53%	✓	100%
1-4	✓	04:47	100%	✗	32:47	87%	✓	100%
2-1	✗	46:55	94%	✗	136:42	65%	✗	94%
2-2	✓	04:13	100%	✗	61:37	74%	✓	100%
2-3	✓	05:15	100%	✗	40:25	81%	✓	100%
2-4	✓	05:38	100%	2/3	125:36	100%	✓	100%
3-1	✓	52:19	100%	✗	10:00	29%	✓	100%
3-2	✓	09:42	100%	✗	54:29	66%	✓	100%
3-3	✓	12:02	100%	✗	90:18	94%	✓	100%
3-4	✓	07:09	100%	✗	26:21	74%	✓	100%
4-1	✓	09:51	100%	✗	72:03	95%	✓	100%
4-2	✓	03:54	118%	✗	18:18	57%	✓	118%
4-3	✓	13:44	100%	✗	26:31	56%	✓	100%
4-4	✗	04:13	83%	✗	128:20	82%	✗	82%
5-1	✓	12:05	100%	✗	192:01	45%	✓	100%
5-2	✓	18:01	100%	✗	179:52	82%	✓	100%
5-3	✓	08:42	100%	✗	91:03	63%	✓	100%
5-4	✓	05:16	100%	1/3	78:34	100%	✓	100%
6-1	✓	10:34	100%	✗	02:28	47%	✓	100%
6-2	✓	136:20	100%	✗	11:17	37%	✗	82%
6-3	✓	09:08	100%	✗	32:03	85%	✓	100%
6-4	✓	04:03	100%	1/3	63:22	100%	✓	100%
7-1	✓	22:34	100%	✗	196:51	68%	✓	100%
7-2	✓	10:17	100%	✗	178:09	82%	✓	100%
7-3	✓	12:43	100%	✗	19:40	81%	✓	100%
7-4	✓	08:55	100%	1/3	28:21	100%	✓	100%
8-1	1/3	164:29	100%	✗	21:16	22%	2/3	100%
8-2	1/3	199:26	100%	✗	47:05	60%	2/3	100%
8-3	✓	10:47	100%	✗	19:12	72%	✓	100%
8-4	✓	21:10	100%	✗	09:25	47%	✗	-

with other fuzzing instances appears to have a synergistic effect, allowing one with a simple but incomplete strategy to improve on a partial solution from another instance.

I believe that this allowed SandPuppy to solve levels 6-2 and 8-4 when Ijon could not.

5.8.2.3 State Exploration Evaluation

I evaluated SandPuppy's ability to explore the state-space of stateful parsers on the libtpms library. This is a real-world target also used in Ijon [23]. libtpms is a software emulator for Trusted Platform Module (TPM); TPM defines a protocol which can be used to interact with a cryptoprocessor. The protocol defines various command-types and associated payloads, which affect processor-state in different ways.

To evaluate the systems on generating complex messages over a significant time-frame, I fuzzed libtpms for over 100 hours with SandPuppy, AFL, AFL++, RedQueen, and LAFIntel. I used diverse seeds and equal compute for all fuzzers.

I track the following information to evaluate state-space exploration: total number of unique command-sequences, counts of unique command-sequences of different lengths, and counts of unique command sub-sequences of lengths 1, 2, 3, and 4. I also tracked command parameter-sizes, which helps distinguish between different varieties of the same command. SandPuppy was able to automatically identify and correctly categorize state-variables holding this information.

The results show that SandPuppy outperforms all other fuzzers after just a single run. SandPuppy found more unique sub-sequences overall, and more unique sequences of various lengths as well; this is especially evident when taking parameter size into account (Table 10). Looking at sequences found over time (Figure 12), it is

Table 10. Number of distinct command subsequences and total unique sequences found by all fuzzers in LIBTPMS, without accounting for command parameter-size and with. SANDPUPPY outperformed all other fuzzers.

Sub-seq. Length	AFL	AFL++	LAFIntel	RedQueen	SandPuppy
1	96 / 737	94 / 741	105 / 876	97 / 783	**111 / 115,751**
2	301 / 978	328 / 1,050	392 / 1,259	348 / 1,077	**671 / 116,613**
3	413 / 691	589 / 932	677 / 1,098	671 / 989	**879 / 2,659**
4	461 / 633	865 / 1,101	907 / 1,172	1,075 / 1,324	**1,186 / 1,792**
Unique Sequences	1,324 / 1,991	2,137 / 2,840	1,882 / 2,769	2,706 / 3,379	**2,998 / 119,278**

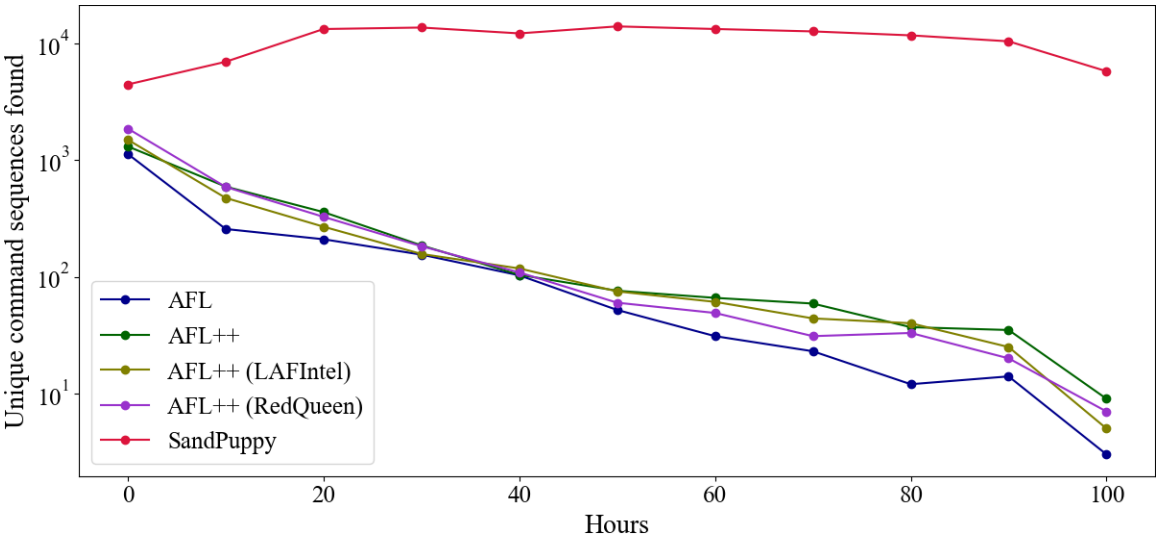

Figure 12. Unique command-sequences with parameter size found over time in LIBTPMS. SANDPUPPY consistently finds more unique-sequences throughout the session compared to other fuzzers, which "stall" as fuzzing progresses. The y-axis is in log scale.

clear that SANDPUPPY finds them at a higher and relatively-constant rate compared to other fuzzers, whose rates of discovery decrease over time.

Further evidence that SANDPUPPY performs better at state exploration is shown in the heatmap Figure 13. For each discovered sequence length, SANDPUPPY discovers a larger variety of command sequences and continues to do so as fuzzing progresses. In contrast, other fuzzers initially identify a variety of states corresponding to smaller sequence lengths, with finds growing increasingly sparse over time. This shallow exploration of the state space also explains why these fuzzers find longer sequences

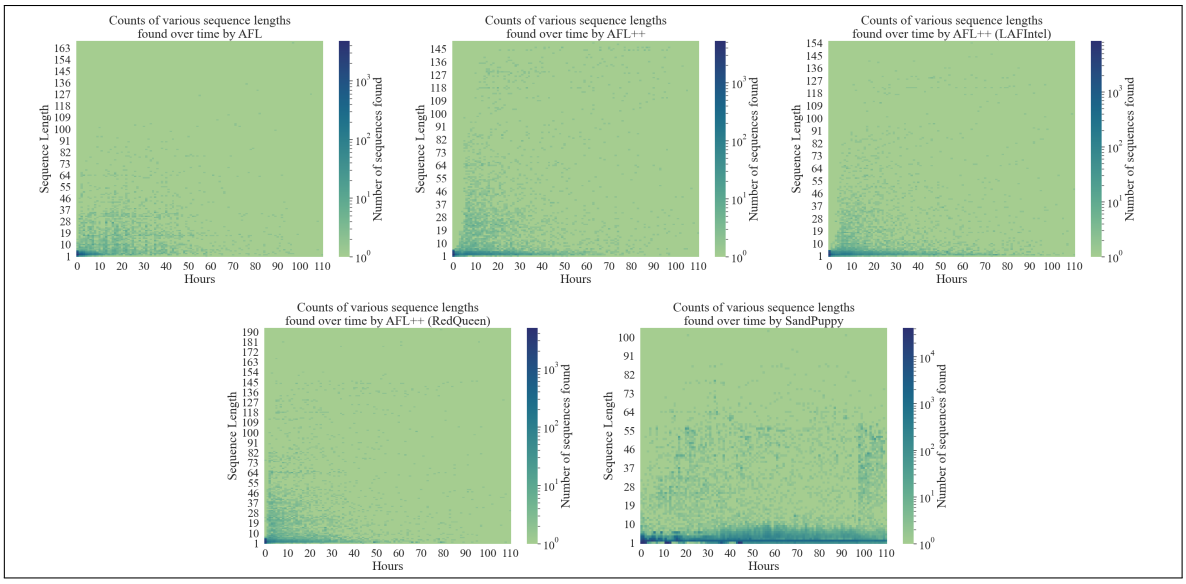

Figure 13. Heatmap of counts of various sequence lengths found over time by all fuzzers in LIBTPMS. SANDPUPPY consistently discovers far more sequences of various lengths throughout the session, whereas findings from other fuzzers grow sparser; even if they find longer sequences.

over the same amount of time compared to SANDPUPPY: They sacrifice diversity for length, whereas SANDPUPPY does the opposite.

I also evaluated my approach on a custom test-bed program called RAREBUG, featuring a state-dependent bug that is hard to trigger (see Listing 5.7). Here as well, SANDPUPPY significantly outperformed all other fuzzers by finding the vulnerability the fastest.

5.8.3 Coverage Evaluation

I used JSONCPP, LIBTPMS, PCAPPLUSPLUS (commit `a817631`), and READELF (BINUTILS version 2.32), and evaluated SANDPUPPY against AFL, AFL++, LAFINTEL, REDQUEEN, SGFUZZ and FERRY. SGFUZZ and FERRY are recent approaches

```
const unsigned int MAGIC_SEQUENCE = (8 << (3u * 8u)) +
                                    (4 << (2u * 8u)) +
                                    (7 << (1u * 8u)) +
                                    (3 << (0u * 8u));
unsigned int curr_seq = 0u;
int process_message(int m_type, const char* msg) {
  curr_seq = (curr_seq << 8u) + m_type;
  if (curr_seq == MAGIC_SEQUENCE) {
    // code to generate buffer overflow
  }
  //..
}
```

Listing 5.7. Section of RAREBUG with bug triggered by message-type sequence 8, 4, 7, 3.

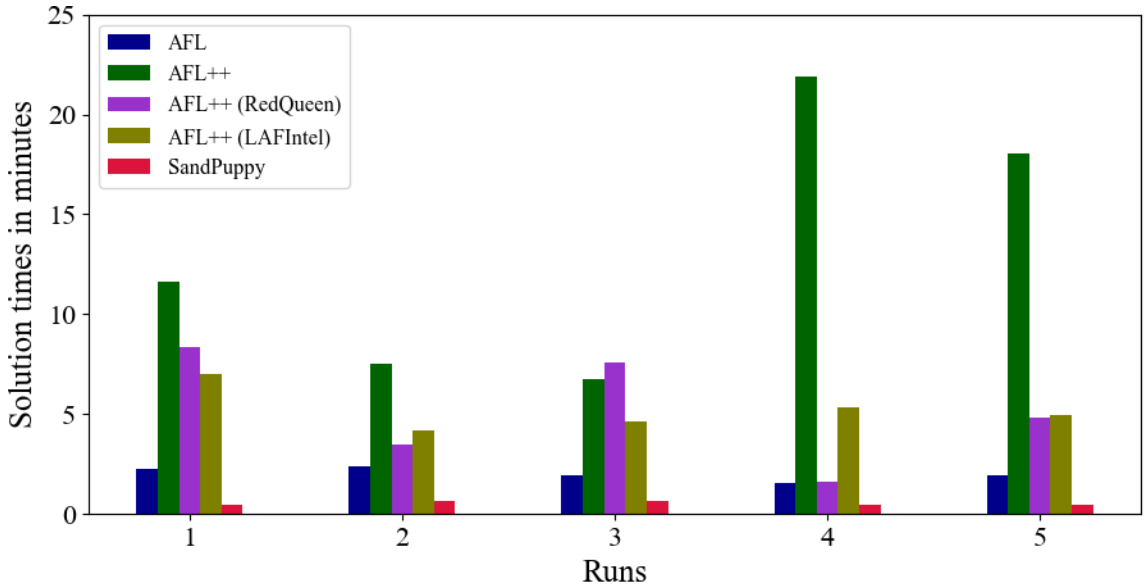

Figure 14. RAREBUG solution times for all fuzzers (average of 5 runs). SANDPUPPY significantly and consistently outperforms all other fuzzers.

that try to leverage state-representative variables. SGFUZZ uses static analysis to identify and instrument enum variables in order to explore complex states in programs like protocol parsers and message processors. FERRY aims to identify state-representative variables that guard branch conditions and tries to solve for values that help satisfy those conditions.

I ran SANDPUPPY for 6 iterations (runs), 4 hours each, for a total fuzzing-time of

Table 11. Number of variables detected after each trace-generation step of each run.

Target \ Run	Variables detected					
	Initial	**1**	**2**	**3**	**4**	**5**
jsoncpp	90	107	107	107	108	108
libtpms	554	872	912	912	912	945
PcapPlusPlus	384	495	499	500	506	506
readelf	314	595	626	636	637	640

24 hours. As the number of SandPuppy targets vary with each run, a one-to-one comparison based on equal compute is difficult. I therefore calculate the rounded-up average of the number of targets fuzzed during each SandPuppy run, and spin up that many parallel-fuzzing sessions for the other fuzzers, which I then fuzz for 24 hours. In the case of both SGFuzz and Ferry, my experimental setup was similar to the ones described by the respective authors, and I ran both for 24 hours as well.

Recall that instrumented variables are recognized only if traces were generated for them, which happens only if the pertinent code path was executed. Table 11 clearly shows that SandPuppy detects an increasing number of variables in the trace-generation phase, after each fuzzing run. This allows it to potentially explore additional complex states and also suggests that coverage increases over each run. To quantify, I calculated the percentage increase over baseline coverage, which is derived from the initial seeds provided to all fuzzers. I also calculated the percentage of uniquely covered blocks with respect to baseline coverage for each fuzzer, to quantify uniqueness of coverage (see Table 12).

The results (Table 12) show that SandPuppy superseded or matched the performance of all other fuzzers with respect to raw coverage improvement on jsoncpp, libtpms, and readelf. For pcapplusplus, SandPuppy's achieved 85.70% coverage improvement compared to RedQueen's 86.23%. It is also clear that SandPuppy

Table 12. Improvement over baseline coverage for all fuzzers. The value in parentheses is the percentage of unique coverage relative to the baseline. For FERRY comparison is done only using basic-block counts as it uses a symbolic approach and does not have initial seeds.

Target \ Fuzzer	Coverage Improvement (%)							Blocks Covered	
	AFL	**AFL++**	**LAFIntel**	**RedQueen**	**SGFuzz**	**SandPuppy**	**Total**	**Ferry**	**SandPuppy**
jsoncpp	45.78 (0.00)	46.09 (0.00)	45.98 (0.00)	45.76 (0.00)	42.63 (0.33)	**46.65** (0.00)	47.10	623	**1314**
libtpms	96.78 (0.00)	111.77 (0.76)	112.26 (0.49)	111.06 (0.49)	0.33 (0.00)	**113.78** (5.67)	120.11	47	**3923**
pcapplusplus	28.37 (0.00)	34.23 (0.00)	35.97 (0.00)	**86.23** (10.67)	69.25 (0.00)	85.70 (10.40)	97.31	251	4910
readelf	235.58 (1.37)	188.52 (1.49)	174.74 (3.8)	178.11 (14.25)	n/a	**261.48** (9.6)	292.32	342	**8473**

provides significant unique coverage, which is evident when looking at LIBTPMS, PCAPPLUSPLUS, and READELF.

A few cases stood out when inspecting unique basic blocks that were covered by SANDPUPPY. In LIBTPMS, SANDPUPPY triggered a path that detects and mitigates dictionary attacks against the TPM processor. The path is only triggered by 5 successive authorization failures. In the case of PCAPPLUSPLUS—a network-packet parsing library—SANDPUPPY triggered parsing of DHCP requests, which rely on a specific combination of source and destination UDP ports. Additionally, SANDPUPPY also triggered far more paths dealing with the parsing of specific HTTP methods and response codes. These findings clearly show that in addition to improving coverage in general, SANDPUPPY is also able to identify and cover multiple deep and complex states.

I did notice changes in variable classification from run to run and verified that re-classifications were accurate based on variable usage in the source code. Misclassifications have the potential to generate useless inputs, but do not appear to significantly degrade performance otherwise. I also observed that some misclassified variables were accurately re-classified as non state-representative, which reduced the generation of useless inputs in subsequent runs.

Compared to SGFUZZ, SANDPUPPY exceeds coverage performance in every case. When analyzing the results, I noted several reasons for these results. First, the

regex-based static analysis performed by SGFuzz is fragile; when attempting to use it against the binutils source to instrument readelf, SGFuzz crashed due to issues parsing enum definitions containing comments with braces. When used against libtpms, SGFuzz was able to identify certain enum definitions. However, it was not able to detect or instrument *any* variables, including enums identified by SandPuppy. This is because the values of these variables are set using integer constants instead of enum constants. In contrast, SandPuppy's approach that looks at variable semantics and is able to identify such variables and expose their state.

SGFuzz also identifies false-positives that are not actually enums. In this case, its source-level instrumentation causes a compilation error due to a type-mismatch. While a blacklist solves this issue SGFuzz does not distinguish between identically-named variables based on usage context. For example, on encountering two identically-named variables of which one is an enum, there is no way to only blacklist the non-enum variable. This was evident with PcapPlusPlus, where a few valid enum variables had to be blacklisted in order to compile the source successfully.

To compare fairly against Ferry, I looked at the individual block counts as Ferry uses a symbolic approach to which I cannot provide initial seeds similar to how I can with AFL. Here, I can see that SandPuppy significantly outperforms Ferry. When running against jsoncpp, PcapPlusPlus, and readelf, Ferry terminated early and quite quickly. In the case of libtpms, Ferry ran out of memory when using 64 GB. Given that it is a symbolic approach, Ferry suffers from the familiar path-explosion problem.

5.8.4 Vulnerability Detection

I evaluated SandPuppy's ability to find vulnerabilities on two real-world targets: dmg2img and PcapPlusPlus. dmg2img was first fuzzed in academic research by WEIZZ [87] and then by Ijon [23]. During the analysis phase, SandPuppy identified multiple state-representative variables, including `kolyblk.XMLLength`, which was manually identified by Ijon authors. SandPuppy discovered 7 bugs; 4 were identified by WEIZZ and Ijon, and 2 are known issues [88], [89]. The last bug is an unreported bug that is a one-byte read heap-buffer overflow in `convert_char4`, which I reported. This bug is triggered when parsing a malformed `BT_TERM` block. Instrumented fuzzing instances that found this bug all provided feedback from variables in the `mishblk` struct.

PcapPlusPlus is a library for capturing, parsing, manipulating, and crafting network packets. Here I evaluated SandPuppy against AFL, AFL++, RedQueen, and LAFIntel. The experimental setup is similar to coverage evaluation with respect to both fuzzing time and compute. The results are shown in Table 13. In total the tools discovered 11 previously unknown bugs which I reported; they include 2 stack-overflows (read, write) and 9 heap-overflows (read). SandPuppy discovered 10 out of 11 bugs, out of which 2 were not found by other fuzzers. LAFIntel, AFL++, RedQueen, and AFL discovered 9, 6, 4, and 3 bugs respectively.

Of the two bugs unique to SandPuppy, Heap Overflow #7 was triggered the "traditional" way of fuzzing-research improvements: SandPuppy's state awareness helped it trigger additional code that contained the bug. The code containing the other bug, Stack Overflow #2, was covered by *all* the fuzzers, but SandPuppy was the only fuzzer to trigger the code with the correct *state*, using its state awareness,

Table 13. Bugs discovered by each fuzzer in PcapPlusPlus. SandPuppy discovered 10 out of 11 bugs (2 uniquely). LAFIntel, AFL++, RedQueen, and AFL discovered 9, 6, 4, and 3 bugs respectively. The last column specifies whether the vulnerable block was covered by other fuzzers.

Bug	AFL	AFL++	LAFIntel	RedQueen	SandPuppy	Covered by others
Heap overflow #1	✗	✗	✓	✗	✓	✓
Heap overflow #2	✓	✓	✓	✓	✓	✓
Heap overflow #3	✗	✓	✓	✓	✗	✓
Heap overflow #4	✓	✓	✓	✓	✓	✓
Heap overflow #5	✗	✓	✓	✓	✓	✓
Heap overflow #6	✗	✓	✓	✗	✓	✓
Heap overflow #7	✗	✗	✗	✗	✓	✗
Heap overflow #8	✓	✓	✓	✗	✓	✓
Heap overflow #9	✗	✗	✓	✗	✓	✓
Stack overflow #1	✗	✗	✓	✗	✓	✓
Stack overflow #2	✗	✗	✗	✗	✓	✓

to induce the vulnerability. To detect the vulnerability, SandPuppy identified and instrumented a computed length variable, maximizing it over the course of the next fuzzing run to overflow the stack buffer.

5.8.5 Feedback Instrumentation Evaluation

The results so far demonstrate that SandPuppy identifies state-representative variables providing feedback sufficient enough to improve coverage, vulnerability identification, and state-exploration. However this does not mean that *all* instrumented variables and associated feedback expose *useful* state enhancing these metrics. There is also a risk of fuzzing slowdown due to generating a large number of useless inputs [23]. Therefore I designed an experiment to measure the effect of each feedback-instrumentation type on coverage and state-exploration. I fuzzed libtpms and libpng for 24 hours: with a single feedback-type enabled/excluded, and with all feedback enabled on a random subset of variables. I evaluated these against a regular SandPuppy run.

Table 14. Number of distinct command subsequences and total unique sequences with parameter size found for each feedback-evaluation run in LIBTPMS. Minus signs preceding feedback types describe runs where the corresponding feedback was excluded. Permutation feedback by itself outperforms all other feedback types and excluding it leads to a dramatic drop in performance.

Sub-seq. Length	RVCF	Perm	Max	RVMF	-RVCF	-Perm	-Max	-RVMF	Random	Regular
1	222	**41,492**	638	260	15,824	623	16,864	18,062	379	12,998
2	277	**41,754**	756	335	16,381	930	17,230	18,552	497	13,547
3	218	700	166	262	**1,237**	454	820	1,064	298	978
4	69	669	38	104	**702**	113	601	654	271	377
Unique Sequences	425	**43,073**	768	559	17,177	1,013	17,852	19,189	1,170	13,964

Table 15. Coverage (percentage of basic blocks hit) for each feedback-evaluation run in LIBPNG. Minus signs preceding feedback types describe runs where the corresponding feedback was excluded. Excluding permutation feedback increases performance compared to all other runs. RVMF performs the best out of all individual feedback-types.

	RVCF	Perm	Max	RVMF	-RVCF	-Perm	-Max	-RVMF	Random	Regular
Coverage	32.10%	31.80%	31.76%	33.60%	32.99%	**34.47%**	31.97%	32.15%	31.87%	32.13%

Results for LIBTPMS are shown in Table 14. Permutation improves state exploration the most; excluding it leads to a dramatic reduction in identified unique sequences and sub-sequences. In contrast, excluding other feedback types (RVCF, maximization, and RVMF) increases state diversity.

This trend holds when evaluating individual feedback: Permutation feedback by itself is almost four times better, while remaining feedback types do not perform better than random feedback. This suggests that—based on this particular diversity-metric—other feedback types do not expose useful state and generate inputs that slow down fuzzing.

Results for LIBPNG are shown in Table 15. Here, *excluding* permutation feedback increases coverage. Variables instrumented include the chunk name, the chunk-header length and the text length in a `tEXT` chunk. While this feedback allows the fuzzer to recognize different combinations of chunk types and associated lengths, it results in

a large number of inputs. Chunk ordering does not affect rendering or parsing of a PNG, so such inputs are of limited use.

In contrast, RVMF appears to have a positive effect on coverage both by itself and when included with maximization feedback and RVCF. In either case, coverage is better than when fuzzing with all feedback types. The is because some variable pairs instrumented by this feedback hold information related to the dimensions and sizes of chunks, and the image itself. The functions these variables reside in parse chunk and image-data, and feature a number of sanity checks, boundary checks, and conditional parsing-logic. Maximizing these variables with respect to each other is likely to exercise these paths, resulting in greater coverage. This is more likely when the fuzzing campaign is not overwhelmed by many uninteresting inputs, which is observed when I disable the permutation feedback.

Both cases demonstrate that while fuzzing with all feedback types generally improves coverage and state exploration, the usefulness and relevance of each one is dependent on the target and associated evaluation metric. Better performance in future fuzzing runs could be achieved by analyzing the contributions of different feedback types, and including only those enhancing coverage productively.

5.9 Discussion, Limitations, and Future Work

I propose an approach that guides fuzzing towards complex or deep states using feedback from automatically-identified state-representative variables. SandPuppy is used in an end-to-end, dynamic-feedback approach where test cases discovered after one full run are used for trace-generation and variable classification, and as initial seeds in a subsequent run. As coverage generally increases during a run, new test

cases have a chance to exercise new code paths, leading to the discovery of additional state-representative variables. SandPuppy thus progressively explores deeper states within a program.

But all instrumented targets are not necessarily useful (Section 5.8.5). An evaluation step could therefore calculate how much each instrumented target improves over the baseline coverage and weed out ones that do not perform well.

In large programs, SandPuppy can identify a large number of state-representative variables, resulting in a correspondingly large number of targets that need to be built and fuzzed. FuzzFactory can support up to four coverage maps at a time; leveraging this feature might allow exposing feedback from multiple variables through a single binary.

During coverage evaluation, I ran SandPuppy for six iterations of 4 hours each to compare against 24 hour runs of other fuzzers. However it is possible to run SandPuppy for any number of iterations of any duration. Furthermore, it is also possible to run SandPuppy for as long as needed, and terminate when either no new coverage has been detected, or no new state-representative variables have been identified.

Currently SandPuppy focuses on integer state-representative variables. Future research could investigate identifying variables of other types. In addition, devising different types of feedback, perhaps for different variable classes might improve performance.

Core fuzzing. Since its release in 2013, AFL has been greatly improved. Seed selection and scheduling enhancements have increased performance in different situations [90]–[95]. Target information can be used to improve seed-mutation strategies, making the generation of interesting inputs more likely [33], [96]–[102]. Taint tracking improved efficiency by focusing mutations only on interesting sections of input [34], [36], [103]–[105]. While these approaches and SandPuppy improve fuzzing, they solve different problems.

Playing with coverage. Angora, CollAFL, kAFL, and InsTrim enhance coverage-tracking and feedback by improving how AFL recognizes novel coverage [34], [106]–[108]. Angora and CollAFL extend coverage tracking by reducing coverage-map collisions, helping AFL distinguish between different program states. kAFL uses hardware-assisted techniques to improve coverage information when fuzzing kernels, while InsTrim uses CFG-aware instrumentation to improve execution-path differentiation. These approaches also improve feedback similar to mine, but focus on edge-based coverage rather than program state.

Input-state correspondence. AFL has difficulty solving magic-value comparisons, and exploring deeply-nested branches and complex program-states. LAFIntel [40] solved magic-value comparisons by splitting compare instructions into multiple smaller ones. Each new matching byte registers new coverage, allowing gradual convergence. Steelix [109] uses lightweight static-analysis and instrumentation to provide information about comparison progress and locations of magic bytes.

Approaches using concolic [37], [38], [110]–[113] and symbolic execution [114]–[117] can help deal with magic bytes or deeply-nested branches. But both struggle to

explore deep or complex states relying on a specific sequence of state updates. With concolic execution, the path is fixed to what has already been observed making it impossible to solve for an input reaching a desired state. Symbolic execution can help solve some of these problems, but scalability is a concern [90] due to state explosion.

RedQueen [33] is an alternative to symbolic/concolic execution and taint tracking. It derives input-state correspondence to identify input regions to mutate. SandPuppy is similar as it also relies on internal program-state. However I do not try to correlate this state with input, and instead directly expose it to influence coverage feedback.

Domain-specific state reasoning. Domain-specific fuzzing [42]–[46] has successfully solved certain deep-state fuzzing problems. PerfFuzz [43] and SlowFuzz guide the fuzzer towards pathological inputs to expose algorithmic-complexity vulnerabilities, while MemFuzz [45] augments coverage information by accounting for memory accesses. However being highly focused, the approaches are limited in their applicability as target programs may not expose necessary state. In contrast, SandPuppy aims to be more generally applicable.

General program state reasoning. Ijon uses manual annotations [23] to address the deep-state fuzzing problem. It provides a library of feedback-instrumentation primitives that can be used to annotate source code to provide state feedback. While successful at overcoming a number of obstacles that affect deep-state fuzzing and surpasses prior fully-automated approaches, Ijon requires a human with enough knowledge and expertise to recognize state-representative variables that can actually improve target coverage.

SandPuppy builds on Ijon by automating detection and instrumentation of state-representative variables. While it does not aim to completely replace human experts,

my results clearly show that SandPuppy can match and even slightly outperform Ijon in certain cases.

InvsCov [118] also explores program state-space to improve coverage and detect vulnerabilities. It uses runtime dynamic-analysis of program variables similar to SandPuppy. However the focus is on identifying invariants and augmenting coverage feedback with this information, to partition program state-space and guide the fuzzer towards states violating these invariants, potentially triggering vulnerabilities.

SGFuzz [48] also attempts to solve a similar problem by using regex-based static analysis to identify enum variables in protocol parsers. My approach focuses on additional variable categories making it applicable to other program types. The advantages of my approach are clearly seen from the evaluation results. Recall that I identify variables based on their semantics rather than syntax, allowing me to detect those that behave similar to enums even if not defined as such; SGFuzz is unable to detect such variables. SGFuzz's regex-based approach is also fragile as it crashes in cases where enum definitions contain problematic characters. Finally, SGFuzz maintains a global variable-blacklist that does not distinguish identically-named variables based on their usage context. As a result, this has the effect of blacklisting valid enum variables simply because they happen to have the same name as a non-enum one.

Ferry [47] also explored identification of state-representative variables, attempting to categorize them based on certain characteristics (with different categories than SandPuppy). However, rather than exploring the triggering of other program *states*, Ferry exclusively uses information from these variables to explore dependent *branches*. In this sense, it is more similar to existing symbolic-execution approaches aiming to trigger previously-untriggered code. Based on my evaluation results, I saw that

SandPuppy significantly outperforms Ferry. Additionally, Ferry also seems to suffer from the path-explosion problem that typically affects symbolic approaches.

5.11 Conclusion

SandPuppy is a fully-automated source-code based fuzzing approach that explores deep-states in programs by identifying and labeling state-representative integer variables using runtime value-traces, and instrumenting them to provide appropriate feedback based on their semantics. Ijon addressed the issue of prior fully-automated approaches struggling with deep-state exploration as coverage alone does not provide enough information to guide fuzzers towards complex states. However, SandPuppy effectively automates Ijon, requires no knowledge or manipulation of program source, and results in performance that either matches, or in certain cases slightly outperforms Ijon. I believe that SandPuppy represents the first step toward fully-automated deep-state fuzzing. With techniques that minimize expertise needed, perhaps fuzzing can finally be used by non-fuzzing-expert developers, so that swaths of bugs can be discovered and quashed.

Chapter 6

CONCLUSION

In the span of just fifteen years, the realm of cybersecurity has witnessed a paradigm shift. From being occasional events, cyberattacks and cybercrime have transformed into persistently recurrent threats. As technological advancements have woven computers and software intricately into the fabric of our daily lives and pivotal sectors of society, they have also become tantalizing targets for malicious actors. The repercussions of cyberattacks now transcend beyond system disruptions, threatening to wreak havoc on the lives of millions worldwide. Addressing this critical issue necessitates the exploration and development of adaptive strategies tailored to this ever-evolving digital landscape.

This dissertation zeroes in on the utilization of state information as a linchpin in shaping robust defensive measures in cybersecurity. By modeling the attacker's state — encapsulating their capabilities and objectives — alongside the defender's vulnerabilities and their dependencies, it is possible to discern potential attacker tactics that employ sequential exploits. Furthermore, it enables a deeper understanding of the magnitude of damage such attacks could inflict and paves the way to architect defenses to counter them. This approach stands out as a progressive stride, in contrast to traditional models that largely perceive attacks and susceptibilities as individual, isolated occurrences.

In furtherance of refining vulnerability detection, this dissertation introduces a technique that exposes diverse program states to fuzzers by automatically identifying state-representative variables. This methodology surmounts many of the limitations

inherent to coverage-driven strategies, by acknowledging the intricate inner-states of programs. Harnessing these innovative tactics allows stakeholders spanning security experts, academic researchers, and software engineers to gain a twofold advantage. They are equipped to thwart multifaceted cyberattacks and, more proactively, purge software of state-dependent vulnerabilities even before they pose a risk of exploitation.

REFERENCES

[1] *Cost of a data breach 2022*, Accessed April 3, 2022. [Online]. Available: https://www.ibm.com/reports/data-breach.

[2] D. Freeze, *Cybercrime to cost the world $10.5 trillion annually by 2025*, Apr. 2021. [Online]. Available: https://cybersecurityventures.com/cybercrime-damages-6-trillion-by-2021/.

[3] C. E. for International Peace, *Timeline of cyber incidents involving financial institutions*, Accessed April 3, 2023. [Online]. Available: https://carnegieendowment.org/specialprojects/protectingfinancialstability/timeline.

[4] C. Insight, *Critical insight finds healthcare data breach in 2h 2022 higher than pre-pandemic levels affecting more individuals: Critical insight*, Accessed April 3, 2023. [Online]. Available: https://www.criticalinsight.com/resources/news/article/healthcare-data-breach-2h-2022.

[5] D. Kushner, "The real story of stuxnet," *IEEE Spectrum*, vol. 50, no. 3, pp. 48–53, 2013. DOI: 10.1109/MSPEC.2013.6471059.

[6] *Unprecedented increase in cyber attacks targeting government entities in 2022: Cloudsek*, Accessed April 3, 2023. [Online]. Available: https://cloudsek.com/whitepapers-reports/unprecedented-increase-in-cyber-attacks-targeting-government-entities-in-2022.

[7] P. Shakarian and J. Shakarian, "Considerations for the development of threat prediction in the cyber domain," in *AAAI-16 Workshop on Artificial Intelligence for Cyber Security*, 2016.

[8] *Dangerous cyberattack kits cost less than $50 on the dark web*, Mar. 2022. [Online]. Available: https://www.missioncriticalmagazine.com/articles/94045-dangerous-cyberattack-kits-cost-less-than-50-on-the-dark-web.

[9] M. Hill, *Exploit chains explained: How and why attackers target multiple vulnerabilities*, Jan. 2022. [Online]. Available: https://www.csoonline.com/article/3645449/exploit-chains-explained-how-and-why-attackers-target-multiple-vulnerabilities.html.

[10] J. Robertson, A. Diab, E. Marin, *et al.*, "Darknet mining and game theory for enhanced cyber threat intelligence," *Cyber Defense Review*, vol. 1, no. 2, 2016.

[11] J. J. Robertson, V. Paliath, J. Shakarian, A. Thart, and P. Shakarian, "Data driven game theoretic cyber threat mitigation," *Innovative Applications of Artificial Intelligence*, vol. 28, 2016.

[12] W. Shim, L. Allodi, and F. Massacci, "Crime pays if you are just an average hacker," in *Cyber Security (CyberSecurity), 2012 International Conference on*, Dec. 2012, pp. 62–68. DOI: 10.1109/CyberSecurity.2012.15.

[13] M. H. Henry, R. M. Layer, K. Z. Snow, and D. R. Zaret, "Evaluating the risk of cyber attacks on scada systems via petri net analysis with application to hazardous liquid loading operations," in *Technologies for Homeland Security, 2009. HST'09. IEEE Conference on*, IEEE, 2009, pp. 607–614.

[14] T. M. Chen, J. C. Sanchez-Aarnoutse, and J. Buford, "Petri net modeling of cyber-physical attacks on smart grid," *IEEE Transactions on Smart Grid*, vol. 2, no. 4, pp. 741–749, 2011.

[15] H. Okhravi and D. Nicol, "Evaluation of patch management strategies," *International Journal of Computational Intelligence: Theory and Practice*, vol. 3, no. 2, pp. 109–117, 2008.

[16] S. Abraham and S. Nair, "Comparative analysis and patch optimization using the cyber security analytics framework," *The Journal of Defense Modeling and Simulation*, p. 1 548 512 917 705 743, 2017.

[17] L. Miao, S. Li, and Z. Wang, "Optimal dissemination strategy of security patch based on differential game in social network," *Wireless Personal Communications*, vol. 98, no. 1, pp. 237–249, 2018.

[18] F. Piessens, "A taxonomy of causes of software vulnerabilities in internet software," in *Supplementary Proceedings of the 13th International Symposium on Software Reliability Engineering*, Citeseer, 2002, pp. 47–52.

[19] K. Beck, *Test-driven development: by example*. Addison-Wesley Professional, 2003.

[20] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of unix utilities," *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.

[21] M. Zalewski, *American fuzzy lop (2.52b)*, Accessed October 26, 2021. [Online]. Available: https://lcamtuf.coredump.cx/afl/.

[22] Y. Shoshitaishvili, M. Weissbacher, L. Dresel, *et al.*, "Rise of the hacrs: Augmenting autonomous cyber reasoning systems with human assistance," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 347–362.

[23] C. Aschermann, S. Schumilo, A. Abbasi, and T. Holz, "Ijon: Exploring deep state spaces via fuzzing," in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2020, pp. 1597–1612.

[24] P. Shakarian, H. Lei, and R. Lindelauf, "Power grid defense against malicious cascading failure," in *Proceedings of the 2014 international conference on Autonomous agents and multi-agent systems*, International Foundation for Autonomous Agents and Multiagent Systems, 2014, pp. 813–820.

[25] Y. Hayel and Q. Zhu, "Resilient and secure network design for cyber attack-induced cascading link failures in critical infrastructures," in *Information Sciences and Systems (CISS), 2015 49th Annual Conference on*, IEEE, 2015, pp. 1–3.

[26] P. Paruchuri, J. P. Pearce, J. Marecki, M. Tambe, F. Ordonez, and S. Kraus, "Playing games for security: An efficient exact algorithm for solving bayesian stackelberg games," in *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 2*, International Foundation for Autonomous Agents and Multiagent Systems, 2008, pp. 895–902.

[27] J. Pita, M. Jain, J. Marecki, *et al.*, "Deployed armor protection: The application of a game theoretic model for security at the los angeles international airport," in *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems: industrial track*, International Foundation for Autonomous Agents and Multiagent Systems, 2008, pp. 125–132.

[28] M. Jain, E. Kardes, C. Kiekintveld, F. Ordónez, and M. Tambe, "Security games with arbitrary schedules: A branch and price approach.," in *AAAI*, 2010.

[29] T. Okimoto, N. Ikegai, K. Inoue, H. Okada, T. Ribeiro, and H. Maruyama, "Cyber security problem based on multi-objective distributed constraint optimization technique," in *Dependable Systems and Networks Workshop (DSN-W), 2013 43rd Annual IEEE/IFIP Conference on*, IEEE, 2013, pp. 1–7.

[30] T. Alpcan and T. Başar, *Network security: A decision and game-theoretic approach*. Cambridge University Press, 2010.

[31] S. Kusumastuti, J. Cui, A. Tambe, and R. S. John, "A behavioral game modeling cyber attackers, defenders, and users,"

[32] J. L. Peterson, "Petri net theory and the modeling of systems," 1981.

[33] C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, "*REDQUEEN*: Fuzzing with input-to-state correspondence.," in *NDSS*, vol. 19, 2019, pp. 1–15.

[34] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *2018 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2018, pp. 711–725.

[35] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: Fuzzing by program transformation," in *2018 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2018, pp. 697–710.

[36] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "Vuzzer: Application-aware evolutionary fuzzing.," in *NDSS*, vol. 17, 2017, pp. 1–14.

[37] N. Stephens, J. Grosen, C. Salls, *et al.*, "Driller: Augmenting fuzzing through selective symbolic execution.," in *NDSS*, vol. 16, 2016, pp. 1–16.

[38] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "*QSYM*: A practical concolic execution engine tailored for hybrid fuzzing," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 745–761.

[39] M. Böhme, *Aflfast*, Accessed October 26, 2021. [Online]. Available: https://github.com/mboehme/aflfast.

[40] Lafintel, *Lafintel*, Aug. 2016. [Online]. Available: https://lafintel.wordpress.com/.

[41] *Libfuzzer – a library for coverage-guided fuzz testing.* Accessed October 26, 2021. [Online]. Available: https://llvm.org/docs/LibFuzzer.html.

[42] R. Padhye, C. Lemieux, K. Sen, L. Simon, and H. Vijayakumar, "Fuzzfactory: Domain-specific fuzzing with waypoints," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 1–29, 2019.

[43] C. Lemieux, R. Padhye, K. Sen, and D. Song, "Perffuzz: Automatically generating pathological inputs," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 254–265.

[44] T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana, "Slowfuzz: Automated domain-independent detection of algorithmic complexity vulnerabilities," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2155–2168.

[45] N. Coppik, O. Schwahn, and N. Suri, "Memfuzz: Using memory accesses to guide fuzzing," in *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, IEEE, 2019, pp. 48–58.

[46] Y. Wang, X. Jia, Y. Liu, *et al.*, "Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization.," in *NDSS*, 2020.

[47] S. Zhou, Z. Yang, D. Qiao, *et al.*, "Ferry: State-aware symbolic execution for exploring state-dependent program paths," pp. 4365–4382, 2022.

[48] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3255–3272.

[49] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, and C. Zhang, "Nsfuzz: Towards efficient and state-aware network service fuzzing," *ACM Transactions on Software Engineering and Methodology*, 2023.

[50] W. Li, R. B. Vaughn, and Y. S. Dandass, "An approach to model network exploitations using exploitation graphs," *Simulation*, vol. 82, no. 8, pp. 523–541, 2006.

[51] J. Dawkins and J. Hale, "A systematic approach to multi-stage network attack analysis," in *Information Assurance Workshop, 2004. Proceedings. Second IEEE International*, IEEE, 2004, pp. 48–56.

[52] O. Sheyner and J. Wing, "Tools for generating and analyzing attack graphs," in *International Symposium on Formal Methods for Components and Objects*, Springer, 2003, pp. 344–371.

[53] E. Alata, M. Kaâniche, V. Nicomette, and R. Akrout, "An automated approach to generate web applications attack scenarios," in *Dependable Computing (LADC), 2013 Sixth Latin-American Symposium on*, IEEE, 2013, pp. 78–85.

[54] R. Schuppenies, C. Meinel, and F. Cheng, "Automatic extraction of vulnerability information for attack graphs," *Hasso-Plattner-Institute for IT Systems Engineering, University of Potsdam*, 2009.

[55] S. Roschke, F. Cheng, R. Schuppenies, and C. Meinel, "Towards unifying vulnerability information for attack graph construction," in *International Conference on Information Security*, Springer, 2009, pp. 218–233.

[56] D. E. Mann and S. M. Christey, *Towards a Common Enumeration of Vulnerabilities*, Accessed 16 Nov 2017, The MITRE Corporation, Jan. 8, 1999. [Online]. Available: http://cve.mitre.org/docs/docs-2000/cerias.html.

[57] *GNU Bourne-Again Shell (Bash) "Shellshock" Vulnerability (CVE-2014-6271, CVE-2014-7169, CVE-2014-7186, CVE-2014-7187, CVE-2014-6277 and CVE-2014-6278)*, Accessed 16 Nov 2017, 2014. [Online]. Available: https://www.us-cert.gov/ncas/alerts/TA14-268A.

[58] *CVE-2009-1185*, Accessed 16 Nov 2017, 2009. [Online]. Available: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2009-1185.

[59] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats*, Springer, 2005, pp. 247–266.

[60] L. Wang, S. Noel, and S. Jajodia, "Minimum-cost network hardening using attack graphs," *Computer Communications*, vol. 29, no. 18, pp. 3812–3824, 2006.

[61] X. Ou, W. F. Boyer, and M. A. McQueen, "A scalable approach to attack graph generation," in *Proceedings of the 13th ACM conference on Computer and communications security*, ACM, 2006, pp. 336–345.

[62] B. Knaster and A. Tarski, "Un théoreme sur les fonctions d'ensembles," *Ann. Soc. Polon. Math*, vol. 6, no. 133, p. 2 013 134, 1928.

[63] R. De La Briandais, "File searching using variable length keys," in *Papers presented at the the March 3-5, 1959, western joint computer conference*, ACM, 1959, pp. 295–298.

[64] M. Albanese and S. Jajodia, "A graphical model to assess the impact of multi-step attacks," *The Journal of Defense Modeling and Simulation*, vol. 15, no. 1, pp. 79–93, 2018.

[65] K. Durkota, V. Lisy, C. Kiekintveld, and B. Bosansky, "Game-theoretic algorithms for optimal network security hardening using attack graphs," in *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems*, International Foundation for Autonomous Agents and Multiagent Systems, 2015, pp. 1773–1774.

[66] O. Vaněk, Z. Yin, M. Jain, B. Bošanskỳ, M. Tambe, and M. Pěchouček, "Game-theoretic resource allocation for malicious packet detection in computer networks," in *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems-Volume 2*, International Foundation for Autonomous Agents and Multiagent Systems, 2012, pp. 905–912.

[67] S. Marrone, R. Nardone, A. Tedesco, *et al.*, "Vulnerability modeling and analysis for critical infrastructure protection applications," *International Journal of Critical Infrastructure Protection*, vol. 6, no. 3, pp. 217–227, 2013.

[68] F. Flammini, A. Gaglione, N. Mazzocca, and C. Pragliola, "Detect: A novel framework for the detection of attacks to critical infrastructures," *Safety, Reliability and Risk Analysis: Theory, Methods and Applications-Proceedings of ESREL'08*, pp. 105–112, 2014.

[69] T. Spyridopoulos, K. Maraslis, T. Tryfonas, G. Oikonomou, and S. Li, "Managing cyber security risks in industrial control systems with game theory and viable system modelling," in *System of Systems Engineering (SOSE), 2014 9th International Conference on*, IEEE, 2014, pp. 266–271.

[70] W. Reisig, *Petri nets: an introduction*. Springer Science & Business Media, 2012, vol. 4.

[71] A. Finkel, "The minimal coverability graph for petri nets," *Advances in Petri Nets 1993*, pp. 210–243, 1993.

[72] E. Nunes, A. Diab, A. Gunn, *et al.*, "Darknet and deepnet mining for proactive cybersecurity threat intelligence," *arXiv preprint arXiv:1607.08583*, 2016.

[73] Y. Wang, X. Jia, Y. Liu, *et al.*, "Not All Coverage Measurements Are Equal: Fuzzing by Coverage Accounting for Input Prioritization," in *Network and Distributed System Security Symposium*, 2020, ISBN: 1891562614. DOI: 10.14722/ndss.2020.24422.

[74] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, USENIX Association, Aug. 2020.

[75] FFmpeg, *Ffmpeg: Fli/flc animation video decoder*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/89ea7a2442ec3466875c81ca376bba6b6426be45/libavcodec/flicvideo.c#L220.

[76] FFmpeg, *Ffmpeg: Raw dts-hd demuxer*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/6f8b8e633205ab690a6a33b139f4e95828e4892f/libavformat/dtshddec.c#L71.

[77] FFmpeg, *Ffmpeg: Raw dts-hd demuxer*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/f5a663f2a9d05eddbd50609b45d92d2e39b3f9ed/libavformat/h264dec.c#L44.

[78] FFmpeg, *Ffmpeg: Mpeg-1/2 demuxer*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/f5a663f2a9d05eddbd50609b45d92d2e39b3f9ed/libavformat/mpeg.c#L72.

[79] B. Lepilleur and T. J. Authors, *Jsoncpp: Json reader*, Accessed August 30, 2022. [Online]. Available: https://github.com/open-source-parsers/jsoncpp/blob/a1f1613bdd81bf28289e8d3fbeb4eb78b82fb203/src/lib_json/json_reader.cpp#L238.

[80] IJG, *Libjpeg: Parse image frame header*, Accessed August 30, 2022. [Online]. Available: https://github.com/thorfdbg/libjpeg/blob/14bf94c453e5b86d712a858d4a5857db0250c886/codestream/image.cpp#L631.

[81] T. Group, *Libpcap: Read packet*, Accessed August 30, 2022. [Online]. Available: https://github.com/the-tcpdump-group/libpcap/blob/dc14a7babca1471809bee6872539ff836937840e/pcap-rpcap.c#L454.

[82] FFmpeg, *Ffmpeg: Vidvox hap encoder (counter over chunk count)*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/130d19bf2044ac76372d1b97ab87ab283c8b37f8/libavcodec/hapenc.c#L94.

[83] FFmpeg, *Ffmpeg: Vidvox hap encoder (calculate final size)*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/89ea7a2442ec3466875c81ca376bba6b6426be45/libavcodec/hapenc.c#L131.

[84] FFmpeg, *Ffmpeg:rtp h.264*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/870bfe16a12bf09dca3a4ae27ef6f81a2de80c40/libavformat/rtpdec_h264.c#L231.

[85] FFmpeg, *Ffmpeg: Raw dts-hd demuxer (read header)*, Accessed August 30, 2022. [Online]. Available: https://github.com/FFmpeg/FFmpeg/blob/6f8b8e633205ab690a6a33b139f4e95828e4892f/libavformat/dtshddec.c#L71.

[86] C. Cadar, D. Dunbar, D. R. Engler, *et al.*, "K: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *OSDI*, vol. 8, 2008, pp. 209–224.

[87] A. Fioraldi, D. C. D'Elia, and E. Coppa, "Weizz: Automatic grey-box fuzzing for structured binary formats," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 1–13.

[88] H. Salo, *Dmg2img: Adc decompress segmentation fault at adc.c:66*, Accessed February 16, 2022. [Online]. Available: https://bugs.debian.org/cgi-bin/bugreport.cgi?bug=778819.

[89] NIST, *Cve-2021-32614*, Accessed February 16, 2022. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-32614.

[90] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2329–2344.

[91] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," *IEEE Transactions on Software Engineering*, vol. 45, no. 5, pp. 489–506, 2017.

[92] M. Woo, S. K. Cha, S. Gottlieb, and D. Brumley, "Scheduling black-box mutational fuzzing," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 511–522.

[93] A. Rebert, S. K. Cha, T. Avgerinos, *et al.*, "Optimizing seed selection for fuzzing," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 861–875.

[94] Y. Chen, M. Ahmadi, B. Wang, L. Lu, *et al.*, "$MEUZZ$: Smart seed scheduling for hybrid fuzzing," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 77–92.

[95] Y. Wang, Z. Wu, Q. Wei, and Q. Wang, "Neufuzz: Efficient fuzzing with deep neural network," *IEEE Access*, vol. 7, pp. 36 340–36 352, 2019.

[96] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "Nautilus: Fishing for deep bugs with grammars.," in *NDSS*, 2019.

[97] T. Blazytko, M. Bishop, C. Aschermann, *et al.*, "*GRIMOIRE*: Synthesizing structure while fuzzing," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1985–2002.

[98] H. Han, D. Oh, and S. K. Cha, "Codealchemist: Semantics-aware code generation to find vulnerabilities in javascript engines.," in *NDSS*, 2019.

[99] R. Padhye, C. Lemieux, K. Sen, M. Papadakis, and Y. Le Traon, "Validity fuzzing and parametric generators for effective random testing," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, IEEE, 2019, pp. 266–267.

[100] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Căciulescu, and A. Roychoudhury, "Smart greybox fuzzing," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1980–1997, 2019.

[101] C. Lyu, S. Ji, Y. Li, J. Zhou, J. Chen, and J. Chen, "Smartseed: Smart seed generation for efficient fuzzing," *arXiv preprint arXiv:1807.02606*, 2018.

[102] C. Lemieux and K. Sen, "Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 475–485.

[103] B. Mathis, R. Gopinath, M. Mera, A. Kampmann, M. Höschele, and A. Zeller, "Parser-directed fuzzing," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019, pp. 548–560.

[104] B. Mathis, R. Gopinath, and A. Zeller, "Learning input tokens for effective fuzzing," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 27–37.

[105] S. Gan, C. Zhang, P. Chen, *et al.*, "*GREYONE*: Data flow sensitive fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2577–2594.

[106] S. Gan, C. Zhang, X. Qin, *et al.*, "Collafl: Path sensitive fuzzing," in *2018 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2018, pp. 679–696.

[107] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "*kAFL*: Hardware-assisted feedback fuzzing for *OS* kernels," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 167–182.

[108] C.-C. Hsu, C.-Y. Wu, H.-C. Hsiao, and S.-K. Huang, "Instrim: Lightweight instrumentation for coverage-guided fuzzing," in *Symposium on Network and Distributed System Security (NDSS), Workshop on Binary Analysis Research*, 2018.

[109] Y. Li, B. Chen, M. Chandramohan, S.-W. Lin, Y. Liu, and A. Tiu, "Steelix: Program-state based binary fuzzing," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 627–637.

[110] L. Zhao, Y. Duan, H. Yin, and J. Xuan, "Send hardest problems my way: Probabilistic path prioritization for hybrid fuzzing.," in *NDSS*, 2019.

[111] H. Huang, P. Yao, R. Wu, Q. Shi, and C. Zhang, "Pangolin: Incremental hybrid fuzzing with polyhedral path abstraction," in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2020, pp. 1613–1627.

[112] Y. Chen, P. Li, J. Xu, *et al.*, "Savior: Towards bug-driven hybrid testing," in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2020, pp. 1580–1596.

[113] P. Chen, J. Liu, and H. Chen, "Matryoshka: Fuzzing deeply nested branches," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 499–513.

[114] M. Wang, J. Liang, Y. Chen, *et al.*, "Safl: Increasing and accelerating testing coverage with symbolic execution and guided fuzzing," in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, 2018, pp. 61–64.

[115] S. Ognawala, T. Hutzelmann, E. Psallida, and A. Pretschner, "Improving function coverage with munch: A hybrid fuzzing and directed symbolic execution approach," in *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*, 2018, pp. 1475–1482.

[116] L. Zhang and V. L. Thing, "A hybrid symbolic execution assisted fuzzing method," in *TENCON 2017-2017 IEEE Region 10 Conference*, IEEE, 2017, pp. 822–825.

[117] B. Zhang, J. Ye, C. Feng, and C. Tang, "S2f: Discover hard-to-reach vulnerabilities by semi-symbolic fuzz testing," in *2017 13th International Conference on Computational Intelligence and Security (CIS)*, IEEE, 2017, pp. 548–552.

[118] A. Fioraldi, D. C. D'Elia, and D. Balzarotti, "The use of likely invariants as feedback for fuzzers," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2829–2846.